

Wavelength Meter Ångstrom WS/7 IR

Super Precision

User Manual

Introduction

The Wavelength Meter is designed for wavelength measurements of CW (continuous wave) and pulse lasers with an extremely high accuracy and measurement rate.

The Wavelength Meter consists of an optical unit, a USB connection solution for IBM PC compatible computers and the easy to use software. The optical unit is designed to create interference patterns, obtained by six Fizeau interferometers.

The signal from two photodiode arrays, found in the optical unit, is transmitted via the USB communication cable to your computer. The computer program of wavelength measurement then graphically displays the interference pattern and calculates the wavelength in comparison with a reference laser's signal, previously obtained by calibration.

An optional multichannel fiber switch unit provides the possibility of measuring up to eight lasers at the same time (two, four and eight channel versions are available).

An also optional PID regulator can stabilize up to eight lasers at a time or guide them to any desired wavelength course."

Auf der Morgenstelle 14d
72 076 Tübingen
Germany
Phone: +49 - (0)7071 - 968515
Fax: +49 - (0)7071 - 968517
www.HighFinesse.com
info@HighFinesse.com

Contents

1 Preparation of WLM for Operation 5
1.1 Installation 5
1.1.1 Requirements 5
1.1.2 Installation procedure 5
1.1.2.1 PCI DAC board for PID regulation (option) 5
1.1.2.2 Wavelength Meter 5
1.1.3 Uninstallation procedure 6
1.1.4 Update procedure 6
1.2 Device assembly 7
2 Operation procedure 8
2.1 Adjustment 8
2.2 Calibration 8
2.2.1 Calibration with He-Ne laser 8
2.2.2 Calibration with another laser 9
2.2.3 Calibration with Neon lamp 10
2.2.4 Calibration with free running He-Ne laser (option) 11
2.2.5 Calibration with SLR-1530 (option) 11
2.3 Measurement procedure in continuous wave (cw) mode 12
2.4 Pulsed laser measurements 13
2.5 Measurement with autocalibration 13
2.6 Removing blackbody radiation 14
2.7 Measurements with multichannel fiber switch (upgrade option) 15
2.7.1 Synchronization 16
2.7.1.1 Using the direct access for synchronization 16
2.7.1.2 Using the COM port for synchronization 16
2.7.2 Internal and external fibers used with the switch 17
2.8 PID laser control (upgrade option) 18
3 Program surface 19
3.1 Menu 19
3.1.1 Menu File 19
3.1.2 Menu Operation 19
3.1.3 Menu Settings 20
3.1.4 Menu Help 22
3.2 Control Elements 23
3.3 Extra Settings 28
3.3.1 Start-Settings 28
3.3.2 Replay-Settings 31
3.3.3 Recording-Settings 32
3.3.4 Various Settings 33

3.4 LongTerm graph 34

3.5 PID Laser Control Settings (upgrade option) 37

3.5.1 Reference 37

3.5.2 Regulation & Sensitivity 38

3.5.3 Bounds 39

3.5.4 Modify 40

3.5.5 Altering sensitivity 41

3.5.6 Error signals 42

3.5.7 Calibration (of analoguous voltage output) 42

3.6 PID simulator 43

3.6.1 Control and display elements 43

3.6.1.1 The graphs 43

3.6.1.2 General settings 43

3.6.1.3 Hidden General settings 45

3.6.1.4 Laser settings 45

3.6.1.5 Hidden Laser settings 45

3.6.1.6 Regulation settings 46

3.6.1.7 Named sets 46

3.6.2 Finding the best regulation parameters using the PID simulator 47

3.6.3 Limitations 47

4 External access 48

4.1 Measurement result access 48

4.1.1 Exported functions overview 49

4.1.2 Exported functions 53

4.1.2.1 General access and WLM/LSA functions 53

4.1.2.2 Measurement result access functions 64

4.1.2.3 Operation related functions 74

4.1.2.4 State and parameter functions 78

4.1.2.5 Switch functions (for the optional multichannel fiber switch) 95

4.1.2.6 Deviation control and PID regulation functions 97

4.1.2.7 Pattern data functions 104

4.1.2.8 Other functions 109

4.1.3 Error value-constants of Set…-functions 110

4.1.4 Mode constants for Callback-export, the WaitForWLMEvent-function and for saved single measurement and long-term recording files 111

4.1.5 Return error value constants of GetFrequency… and GetWavelength… 113

4.1.6 Return error value constants of GetTemperature 114

4.1.7 Importdeclarations 114

4.1.7.1 C 114

4.1.7.2 Pascal 114

4.1.7.3 Basic 114

4.1.7.4 LabView 114

4.1.8 Used constants 119

4.2 Measurement examples 125

4.2.1 Measurement Example "DataDemo" 125

4.2.2 Measurement Example "LongTerm" 127

4.2.3 Measurement Example "LongTerm" (callback style) 127

4.3 Measurement result access (over COM-Port) 128

4.4 File formats 129

5 Device information 133

5.1 Set of delivery 133

5.2 Operating conditions 133

5.3 Technical Data (Ångstrom WS/7 IR Super Precision) 133

6 HighFinesse Information-Service 135

1 Preparation of WLM for Operation

1.1 Installation

1.1.1 Requirements

For the USB device and Wavelength Meter installation you need:

- IBM PC compatible computer with MS Windows 2000, XP, Vista and Windows7
- 40 MB of free hard disk space
- 1 free USB slot
- For the optional switch unit: 1 free COM port (RS232) if this port is used in opposite to a possible direct access (optional)
- Administrator rights

1.1.2 Installation procedure

Note: *In case you are installing a system which includes an additional PCI DAC board used for a Laser Control or PID regulation option, you should install the PCI DAC board before starting the Wavelength Meter software the first time.*

1.1.2.1 PCI DAC board for PID regulation (option)

1. Switch off the computer and remove the power supply.
2. Insert the PCI DAC board to a free PCI slot on your mainboard, docking station or PCI card extender. (Note: The components of this board are sensitive. Please care for being grounded to avoid the risk of destroying the board with static electricity discharging.)
3. Plug the 1 m open ended access cable to the D-Sub female of the DAC board and fix it tightened with the screws at the male.
4. Close the housing, reestablish the net power supply and switch on your computer.
5. After startup, the OS will find the new board and display the driver installation dialog.
6. Please choose the manual installation way, selecting the driver by yourself. Guide it to the "\inf\me\" directory of your installation CD, select the found inf file and press "OK".
7. The DAC board driver now is about to be installed and the board can be used after this procedure is finished.

Note: *When the computer is powered and the D/A channels ±15 V power supply is connected and active, the D/A channels immediately put out -10 V! This only is set back to zero when the Wavelength Meter software is started. So, to protect your electronic equipment, please ensure to either do not supply the D/A channels with power (±15 V) or do not attach your equipment to the D/A channels outputs before the Wavelength Meter software is started.*
*Please also read chapter 2.8 "*PID laser control (upgrade option)*" Seite 18 before installing the PCI DAC board.*

1.1.2.2 Wavelength Meter

1. Start the setup program '<CD-Drive>:\Setup.exe'.
   This process then registers the necessary drivers in your system and installs the Wavelength Meter program.
2. Connect the Wavelength Meter with your computer using the included standard USB cable.

Now, drivers and programs are ready for work.

1.1.3 Uninstallation procedure

- Click ‘Startmenu | Programs | HighFinesse | Wavelength Meter… | Uninstall’

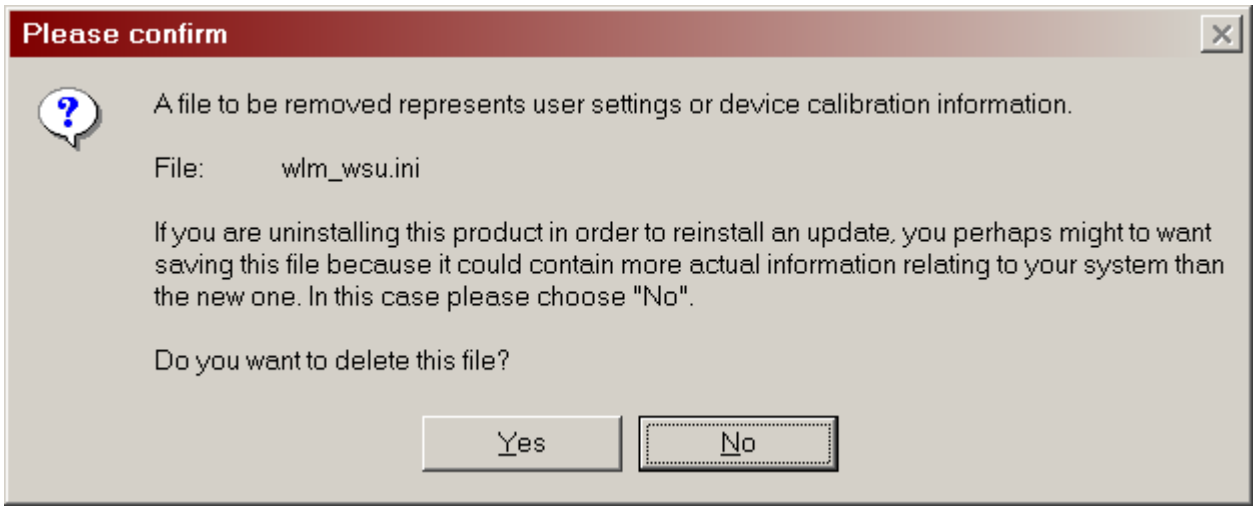
  or

- use the system control applet ‘Software’, select ‘HighFinesse Wavelength Meter…’ and click the remove button.

1.1.4 Update procedure

Updating a Wavelength Meter software simply is to be done by first manually uninstalling the old software (see paragraph 1.1.3 above) followed by a standard setup of the update.

If while using the Wavelength Meter meanwhile you have proceeded a new calibration, it is senseful to preserve the newest calibration data. Additionally you might be interested in also preserving your current software settings. On uninstallation you will be prompted for a few specific files whether they shall be preserved for a following update, this includes calibration data and software settings files. If you want to preserve the calibration data, please say "No" when you are asked to delete *.stn files and history.* files. If you want to preserve your program settings, also don't delete the *.ini files.



When later you install the update, you again will be asked whether to overwrite the files which have not been deleted beforehand. Please then do not overwrite the files if the specific information shall be preserved.

Note: *In very rare cases it might happen that the calibration format of the old software does not match the format of the updated software. In this case the calibration files have to be overwritten on installing the update. You will be notified especially when such a case occurs.*

1.2 Device assembly

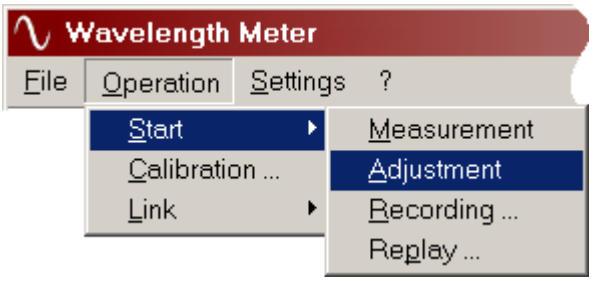
Set the collimator into the beam of the laser to be measured and plug in the 5 m long optical 50µ-fiber to the collimators' mount. Vary the height and the adjustment screws of the stand until there is a maximum emission in the optical fiber output. Now connect this fiber with the optical unit.

For the optional fiber switch there are up to eight input and one output fiber ports at the front panel of the switch. Please connect the external switch unit with a free COM-port (RS232) and to a power supply or directly to the Wavelength Meter (depends on the connection possibilities of the switch). Repeat the above adjustment of the fibers above for all lasers to be measured. Plug each laser fiber into one of the input ports and connect the output fiber to the wavemeter.

Note: *Do not apply force on fixing the fiber connectors to the devices to avoid damaging the fiber ends.*

2 Operation procedure

2.1 Adjustment



Switch on the ‘Adjustment’ operation mode using the menu option ‘Operation | Start | Adjustment’ (left figure).

Observe the interference pattern on the program window, change the exposure value and vary the adjustment screws of the fiberoptical stand til the signal amplitude is set to optimum (30 to 90 % of the chart detail height).

Therefore, please be sure to set the exposure value to a possible minimum, because it influences the measurement repetition rate.

If at the smallest possible exposure (1 ms) the interference pattern is still too large, you need to weaken the signal using optical methods. Vary the adjustment of the fiber stand or place a light filter into the ray's path, if that is appropriate relating to your measurements.

Exit the adjustment mode (using the ‘Stop’-button or the menu option ‘Operation | Stop | Adjustment’) and then start the measurement mode the same way or just use the 'Start' button. The measurement will continue until it is interrupted manually.

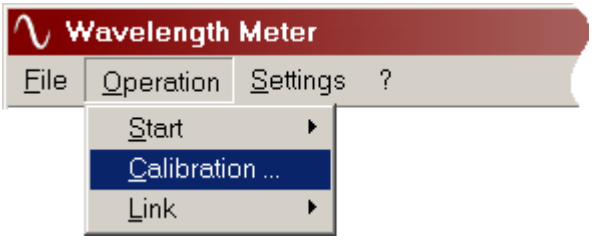
In case of a fluctuating signal level, it is possible to control the exposure period during the measurement (manually as well as in automatic mode).

The measurement program analyzes the interference pattern and, in case of signal disappearance or exceeded or weakened levels, displays an appropriate message in the status bar. The wavelength calculation misses that turn.

If you are running in 'Measurement' mode, it is not necessary to change to 'Adjustment' mode to adjust your device properly. The 'Adjustment' mode is just faster and better suited for much slower and older computers.

2.2 Calibration

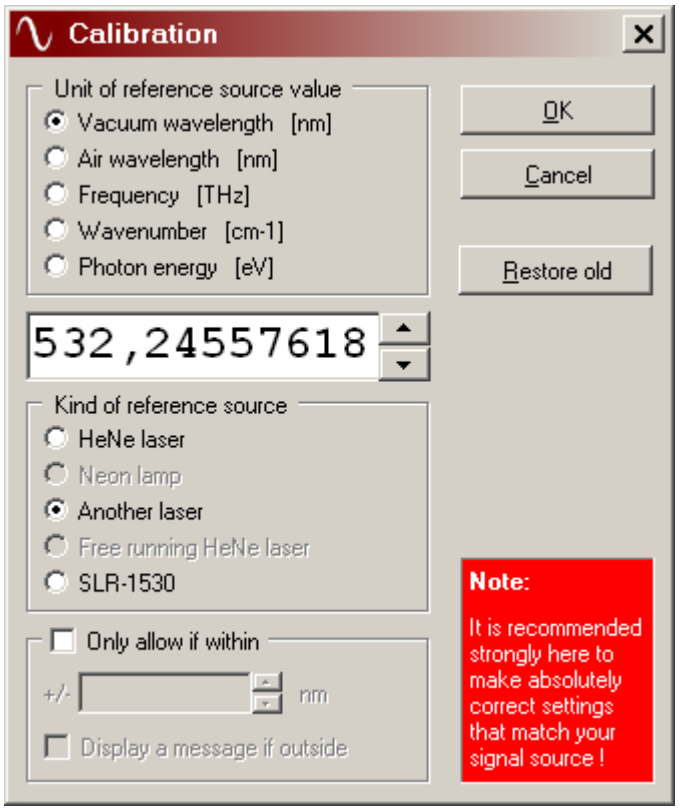
2.2.1 Calibration with He-Ne laser



To guarantee the precision indicated in the technical data, you should calibrate the device periodically on known He-Ne or other reference signals.

In order to do this, you must connect a suitable optical fiber with the reference laser emission to the optical unit.

Start the measurement mode, ensure the displayed interferometer pattern to be illuminated evenly and its amplitude to fit 60 to 90 % of the chart detail height. Also please ensure the wavelength of the reference-stabilized calibration laser to be stable.



To start the calibration, finish the measurement with the 'Stop' button and choose the 'Calibration' option in the menu 'Operation'.

Once you confirm the calibration in the appearing dialogbox (having previously entered your known reference laser's wavelength), the calibration process takes place and the interference pattern and its calculated result value will be displayed.

When the calibration laser light passes an optional multichannel fiber switch and the switch is attached and configured correctly, the calibration is performed using the active channel (in switch mode this is the channel whose result panel is marked sunken, in non switch mode the one selected with the small buttons in the result panel).

If the signal does not match your entry in any way, an appropriate errormessage occurs, saying the calibration was abandoned.

The characterizational values of the reference He-Ne laser interference pattern are written down to the file 'wlm*XXX*st.stn'[*] in the program directory. Additionally, this data will be catenated to the file 'history.*XXX*' with the corresponding timestamp. The Wavelength Meter needs these informations for correct measurements.

Note: *You should use this procedure cautiously because incorrect settings will cause further measurements to malfunction.*

The initial certification and calibration of the Wavelength Meter has been performed using a stabilized He-Ne-laser with accurate 632.990890 nm wavelength.

2.2.2 Calibration with another laser

If the corresponding option in the Calibration menu is selectable, it is possible to perform a calibration on any laser. To do so, a laser of well known wavelength is needed. The wavelength needs to be in the following range:

Device type	Calibration wavelength range
Standard	450 nm – 900 nm
UV	450 nm – 900 nm
UV2	450 nm – 800 nm
IR	600 nm – 1750 nm
IR2	632 nm – 2000 nm

This laser should be at least about 5 times more accurate as the devices absolute accuracy.

[*] *XXX* is a placeholder for the serial revision number of your Wavelength Meter. The revision number consists of three digits and can be found in the 'Contact' dialog box in the help menu.

For this purpose select the menu option ‘Operation | Calibration ...’, choose 'Another laser' and continue similar to the He-Ne-laser calibration above.

With this calibration option there also is the possibility to only allow calibration if the reference laser still is measured within a certain interval. This especially is interesting for an automatic calibration option (if any). The calibration signal is about to be checked for its quality anyway, but this option can be used to restrict this behaviour even more.

Caution:

It is imperative that you are absolutely sure about the correctness of your wavelength value.

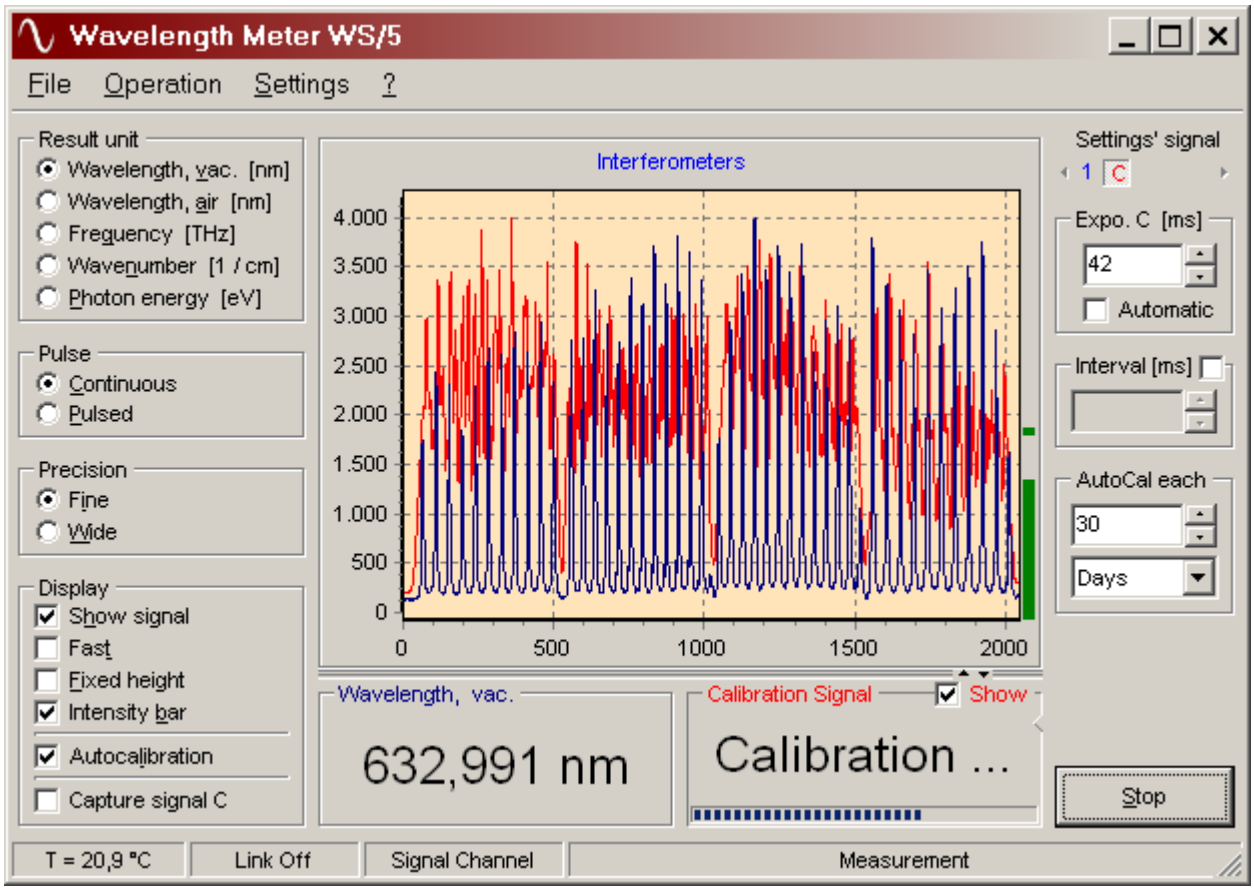
It also is possible to return to an old calbration. To do so please press the "Restore old" button and select an older working calibration.

2.2.3 Calibration with Neon lamp

Some Wavelength Meters have an option to calibrate with a special built-in Neon lamp (not available for WS/7 and WS/Ultimate). This option should be used only if the user does not dispose of a He-Ne laser or if especially the optional autocalibration option (see chapter "Measurement with autocalibration" below) of this device is used.

The interference pattern of the calibration lamp will be taken and correlated to a HeNe laser calibration pattern.

This calibration source is designed for the autocalibration mode (see chapter "Measurement with



autocalibration'' below).

For manual calibration using the built-in Neon lamp, please switch to the internal Neon lamp channel using the small coloured channel button ''C'' on the top right corner of the result value panel. Start a usual measurement and adjust an optimum of signal amplitude (60 to 90 % of the chart detail height, the more the better) using the exposure controls. If the automatic exposure is not long enough (stops at the value set in the 'Extra Settings'-dialog box reached by menu 'Settings'), you can directly enter a higher exposure time after unchecking the automatic exposure checkbox. Then sequentially stop the measurement and start the calibration in the 'Operation' menu. Select ''Neon lamp'' in the Calibration dialog box and press OK. After successful calibration the message ''Cal. ready'' will be displayed.

Note: *Some older devices have been delivered with an external Neon lamp. If you dispose of such a lamp, please set it directly onto the light input of the device, plug in the FC-connector tightly and connect it with power.*

The file 'wlm*XXX*st.stn'* will be updated as on the He-Ne laser calibration procedure.

It is possible to observe and adjust the signal of an internal lamp after switching to channel C (the small button at the upper border of the result panel)

Do not keep a builtin Neon lamp running in measurement mode more then 2 to 3 minutes (the lamp has limited life time), as well do not keep an external lamp powered more then 2 to 3 minutes.

2.2.4 Calibration with free running He-Ne laser (option)

If this device (WS/7 only) is shipped with a special free running He-Ne laser, its periodical wavelength course is characteristic to this laser and can be taken for high precision calibration.

To use this option, choose menu ‘Operation | Calibration ...’, select 'Free running HeNe laser' and continue similar to the HeNe-laser (stabilized) calibration above after entering the known characteristic calibration wavelength.

Important notes:

Push the red Button on the front panel of the free running HeNe Laser. Now wait 2 minutes for the Laser to warm up before you proceed with the calibration. Calibrating to early will reduce the accuracy of the calibration.

The calibration duration of this option can take several minutes and must not be interrupted! The laser will turn off automatically after some minutes.The fan will run until the power supply is disconnected.

It is imperative that you are absolutely sure about the correctness of your wavelength value. The calibration wavelength can be found on the identification label of the laser, together with the serial number.

To achieve the best calibration result, use single mode fibers at 630nm only.

2.2.5 Calibration with SLR-1530 (option)

If the corresponding option in the Calibration menu is selectable, it is possible to calibrate on the given wavelength of a HighFinesse reference laser (SLR).

For this purpose select the menu option ‘Operation | Calibration ...’, choose 'SLR-1530' and continue similar to the He-Ne-laser calibration above.

Caution:

It is imperative that you are absolutely sure about the correctness of your wavelength value. It also is possible to return to an old calbration. To do so please press the "Restore old" button and select an older working calibration.

2.3 Measurement procedure in continuous wave (cw) mode

To start the cw-measurement, first set the wanted physical unit, the desired signal channel (switch channel ;if an optional fiber switch is attached), and the continuous wave mode, then there’s nothing to do but pressing the ‘Start’ button to change to the operation mode ‘Measurement’. Instead of this you also can use the menu option 'Operation | Start | Measurement'.

Note: *For the best results, make sure the device has been properly adjusted and calibrated beforehand. The calibration process especially should be undertaken periodically (and if remarkable environmental conditions have changed) to reach the specified precision.*

On successive measurements, the recorded interference patterns will be displayed in the corresponding charts to each interferometer group and the calculated wavelength in the panel below. In case of bad signals the wavelength panel will indicate this instead of displaying a calculated result.

The measurement repetition rate but also the signal amplitude depend on the actual exposure value. If you set down the exposure time slice to improve a higher repetition rate, the signal amplitude will drop … and vice versa. You can leave the device to detect the smallest working exposure setting. To do so, set the exposure to the lowest possible value and then switch on the automatic exposure mode (to be found under the exposure value controls or in menu ‘Settings | Exposure’).

Additionally it is possible to influence the measurement rate by adjusting the measurement interval. The settable resolution is 10 ms, but it only shows effect if chosen larger than the exposure. The resulting measurement rate always is limited by the maximum of exposure and measurement interval. The measurement interval setting is an overchannel setting. So, in switch mode (optional fiber switch required), it is taken for all channels together.

You can use the ‘Fast’ mode setting to speed up redrawing of the interference patterns by drawing less pixels or choose 'Show Signal' to completely dis- or enable the updating of the graphs, switching off ’Show Signal’ provides the highest possible speed.

The calculated result and all other additional values and settings can be accessed during measurement as described in 'Measurement result access'. Additionally it is possible to receive the wavelength over COM-port of an extra computer.

To transfer the wavelength over the COM-port to another computer use the menu option ‘Operation | Link | Connect COM-Port’. It is also possible to run this option automatically with and while measuring. Adjust the necessary settings in the start-settings dialog (menu ‘Settings | Extra Settings …’).

If you want to start another application with the measurement, you can set this using start-settings, too. We deliver examples for detailed external control ('<InstallationPath>\Projects\DataDemo\Delphi') and for less detailed ('<InstallationPath>\Projects\DataDemo\C', '…\Visual Basic' and '…\LabView'), also you will

find a sample of long-term measurement graphing at <InstallationPath>\Projects\LongTerm\Delphi. The corresponding executable files can be used to demonstrate this application launching.

2.4 Pulsed laser measurements

Measuring pulsed lasers in cw mode

Pulsed lasers can be measured in the continuous wave (cw) mode, too, if you set the exposure large enough to cover at least one pulse signals' cycle, better two or more. This way it is possible to measure each pulse separately (e.g. 100 Hz rep. rate, 10 ms exposure time) or to average (optically, without using the average control right hand side) by choosing a higher exposure time (100 Hz rep. rate, 50 ms exposure time: averaging 5 times).

This technique only works if the periodic cycle of your lasers pulses does not exceed 1000 ms. It is advisable to set the exposure to be controlled manually, an automatic exposure control here doesn't guarantee stable measurements. To avoid 'Big Signal' with large exposure values you may have to adjust less light to the collimator.

Measuring pulsed lasers with the built-in optical synchronization

Measurement of a pulsed laser with the built in triggering mechanism is provided similar to the continuous wave mode. Having switched on the 'Pulsed laser' mode setting, the device detects the peaks independently and raises its own TTL-level triggering pulse controlled by a photodiode when the lasers' peak is rising. Then the program will calculate the wavelength once per pulse.

Note: *To avoid fibre damage, do not let the input energy exceed 300 µJ when measuring ns-laser pulses and respectively less with even shorter pulses.*

2.5 Measurement with autocalibration

Besides the WS/7 series all our actual Wavelength Meters and Spectrum Analysers have the possibility to perfom frequent autocalibrations. The autocalibration mode can be switched on in the control group "Display" or via menu "Settings | Display“. The calibration rate can be set in the group 'AutoCal each' on the right hand side of the main application window. It is possible to adjust it to calibrate once on each measurement start or with a given rate or interval (once per number of measurement shots, days, hours or minutes).

Once this mode is entered and running, both signals and their results can be observed in the pattern charts and the corresponding result panels below if they selectively are enabled using the options buttons in the belonging signal result panels.

For testing purpose it also is possible to run the autocalibration signal in non autocalibration mode if you switch to channel C (with the small button in the result panel) and start a measurement. This channel does not calculate a wavelength (except in combination with a multichannel switch box, see below).

When the device calibrates in autocalibration mode, the flux of the measurement signal is interrupted. The duration of the interruption depends on the exposure setting of the calibration channel. When the calibration signal intensity is out of bounds, the exposure in this case will be adjusted automatically regardless of its autoexposure setting.

When the calibration results in an error message, the calibration was not proceeded and thus won't take effect.

Laser Spectrum Analyzers and all devices of the WS/5 and WS/6 series have a special neon lamp built in, used as calibration source. This source internally works as a separated channel and can also be used to serve with the option to calibrate the device automatically.

Wavelength Meters of the Ultimate series come with a second channel that can be used for autocalibration.

Note: *The second fiber input channel internally is equipped with an additional singlemode fiber best suited for this device (by default with a cut-off wavelength of 570 nm for calibrations with a red HeNe laser, in case of other fibers this is noted in the shipped "Measurement and Calibration Certificate" or in the delivery list). For calibrations using this input channel, please only use a wavelength matching the included fiber.*

WS/7 series do not have a built-in autocalibration feature.This type of device only can be autocalibrated with the multichannel switch option. This option is also possible for all other device series. The switch unit provides the possibility to measure more than just one laser at a time (see chapter measurement with switch option). This circumstance is used to serve with the option to calibrate the device automatically based on the signal guided to the autocalibration channel.

Note: *It is absolutely imperative for all calibration methods that the wavelength value entered for calibration is correct and accurately meets the reference signal!*

2.6 Removing blackbody radiation

The included ccd arrays for all types of devices are sensitive to blackbody radiation (BBR). Especially IR and IR2 devices are very sensitive to BBR. By this reason IR and IR2 devices are cooled to a stabilized working temperature to reduce the blackbody radiation influence. But for stable measurements it additionally is needed to remove the lasting radiation as well as possible. For standard range devices this procedure only is recommended for very high exposure times (e.g. for lasers with very low input power), but for any IR devices always. Each time after software startup one needs to store the lasting background which then successively will be subtracted from each following measurement shot. This allows better measurement stability.

To store the signal to be subtracted, stop any measurements, use the menu option "Operation | Blackbody radiation | Store" or the "Store dark" button (available for IR devices only) in the bottom right corner of the main window and follow the instructions. This procedure will take a few minutes.

After having stored the blackbody radiation data under special conditions, it might happen that the possible exposure range is limited additionally because above a specific exposure setting the blackbody radiation itself would be too high.

For any subsequent call to store, the stored data first needs to be cleared again.

This procedure might be required to be repeated from time to time, especially when operational or environmental conditions have changed.

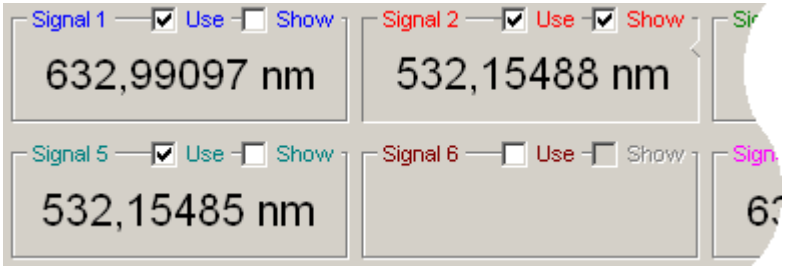
Note: *Please ensure IR and IR2 devices to get sufficient time to warm up before this procedure is performed.*

For measurements with stored background we recommend to switch on the intensity bar. The intensity bar on the right hand side of the pattern charts s split into two parts: A small upper block indicates the complete signal amplitude including the subtracted background. It is possible to obtain an overexposure warning while the pattern still appear well illuminated, because signal plus background can saturate the ccd arrays.

2.7 Measurements with multichannel fiber switch (upgrade option)

In order to measure the frequencies of more than just one laser at a time, we offer an opto-mechanical multichannel fiber switch. The combination of our highspeed Wavelength Meters with one of the quickest fiber switches available allows the measurement of up to eight channels almost simultaneously. Exposure and other parameters can be defined independently for each light source. The switch is controlled by our software via a COM port or by the wavelength meter directly.

The unit switches the output fiber periodically through the input channels (but also unperiodically if desired by varying the exposure parameters channel by channel. It also can be triggered arbitrarily by external access to the software control library). Each input channel is measured separately by the wavemeter.



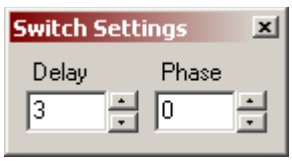
The figures show clippings of an 8-channel switch software surface.

In switch mode (settable in control group "Display" or via menu "Settings | Display") the software displays the interferometer pattern of one or more signals, separated by colour. The signals to display can be chosen by the corresponding checkboxes "Show" in the belonging result panel. Each signal can be (de-)activated by the checkbox "Use" aside. The exposure time and automatic mode as well as the result unit, the precision and the averaging settings can be set independently for each signal. This is done by selecting the corresponding coloured signal/channel number above the exposure control group. Alternatively it also can be done by checking the "Show" button or by changing the active result panel (the active result panel is the one displayed sunken with a small triangle right hand side, it controls which signals' pattern intensity bar is shown). The general ability of the exposure, the result unit, the precision and the averaging settings to behave channel dependent can be set in the "Extra Settings" dialog (Menu "Settings | Extra Settings …", sheet "Various").



The switch mode works reliably in combination with the cw mode only!

Even in non switch mode it is possible to use the switch box to statically switch to one of the attached signal fibers. Statically switching (thus selecting the active channel) simply is performed by selecting one of the small channel buttons in the result panel.



The channel separation of the switch is preadjusted ex works before delivery, but in case of different computer speeds or non neglectable CPU usage of other processes, it nevertheless might happen that the channels superpose. In such case, please modify the channel separation (to be reached by menu "Settings | Switch Settings …"). Please switch on a measurement in switch

mode, connect different lasers and optimize their input power to reach maximum possible ccd pattern graphs (until "overexposed" is shown but). Now adjust the "Delay" and "Phase" values inside the Switch Settings dialog box. "Delay" is an extra separation delay between two succeeding exposure windows, in some cases it might be needed to be set up to 10 ms (and with slow computers even higher). "Phase" tries to iron OS inserted extra delays (or caused by other programs' or mouse activities) by - if the best (precalculated) switching moment can't be reached anymore caused by non CPU allotment of the WLM process by the OS - trying to switch in advance (considered logically). This value usually is of meaning with slow computers only and can be set up to 20 ms. Once the separation for a given computer and CPU usage of other processes is working with big intensity signals, it should be left on that value even if with lower intensity the separation would visually seem to also work with smaller delays, else the channel interference could influence the measurement results even if the interferometer pattern do not look like.

As an additionally available upgrade option the multichannel switch option provides the possibility for autocalibration by switch. Any well known laser source within the calibration range of the Wavelength Meter (450 - 900 nm with standard range devices) connected to the switch can be used for this task. For WS/7 and WS/U this should be used with singlemode switches only.

Note: *The switch has a warranted reliability of 5 ·10^8 switching cycles.*

2.7.1 Synchronization

There are two different ways of synchronization communication between the switch and the host computer. One possibility is via COM port (RS232) and with the second one the switch is controlled by the Wavelength Meter directly over a LEMO connection. Which of these is available depends on the age of the switch and the Wavelength Meter. Newer switch models contain both connectors, older ones may only be equipped with one of them. And Wavelength Meters that are not equipped with a LEMO port also don’t support the direct access.

Note: *If the switch needs an additional external power supply be sure to use the delivered power supply only (5 V, 1 A).*

2.7.1.1 Using the direct access for synchronization

This option is available if both, the switch and the Wavelength Meter provide the direct connection 5 pin LEMO females. It is the usual synchronization interface with modern Wavelength Meters and switches. Please connect the external switch with the Wavelength Meter using the delivered 5 pin LEMO cable. And if the switch needs external power please also plug in the external power supply. If an older Wavelength Meter without LEMO female shall be upgraded with a switch, it needs to be shipped to implement the required interface.

2.7.1.2 Using the COM port for synchronization

This option is suitable for Wavelength Meters that don’t come with a 5-pin LEMO female. That way it is also possible to upgrade your older Wavelength Meter with a fiber switch without changes at the Wavelength Meter. Please use the included serial RS232 cable to connect the switch with a COM port of your computer. If your computer lacks of a COM port (many notebooks and modern PC’s), there also are USB adapters available that provide a COM port.

Note: *Before connecting the switch to power the RS232 data cable needs to connect the switch with the computer. And vice versa: The power adapter must be disconnected in advance to the data line.*

Additionally it is required to correctly adjust the COM port (RS232) parameters. Especially the port number depends on the specific port the switch is connected to. If you want to use the switch and the functionality to export the measurement results via COM port to another computer (see paragraph 4.3 "Measurement result access (over COM-Port)" Seite 128), you need to dispose of two COM ports and set the functionalities to different ports. If the ports are identical, the first used option causes the other to be disabled, saying that the port is occupied. The baudrate depends on the switch itself, there are versions with 9600 or 57600 baud, a wrong set baudrate causes the switch to not work at all. The other handshaking settings required are: 8 data bits, 1 stop bit, no parity and no flow control.

With PCI and USB1.1 Wavelength Meters the switch timing behaviour needs to be calibrated before first use of the switch. This but is done automatically. This procedure can take a few minutes, please do not use your mouse or press any keys while it is running. It also is recommended to close all additional tasks before.

2.7.2 Internal and external fibers used with the switch

Our switches can be equipped with 50 µm multimode fibers or with selected singlemode fibers. All fibers but need to be of the same type, it is not possible to equip different channels with different fiber types because all need to match the common output fiber. Available fiber types are:

- 50 µm multimode fibers, no working range limit
- SM405 singlemode fibers, working range 370 - 500 nm
- SM460 singlemode fibers, working range 440 - 630 nm
- SM630 singlemode fibers, working range 610 - 820 nm
- SM780 singlemode fibers, working range 750 - 1000 nm
- SM980 singlemode fibers, working range 900 - 1300 nm
- SM1300 singlemode fibers, working range 1000 - 1800 nm

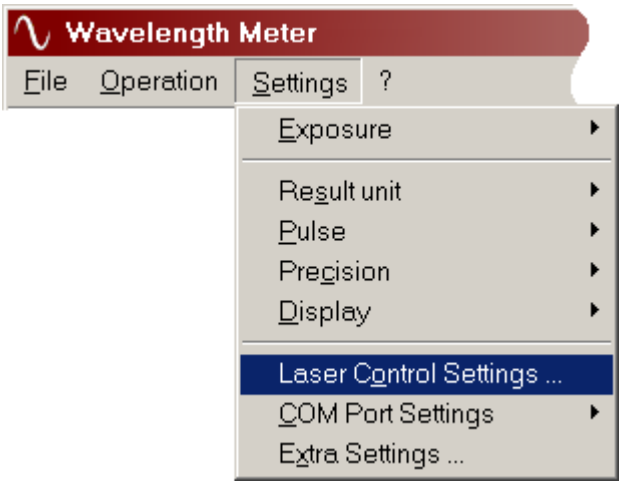
Additionally also the externally attached fibers are required to be of the same type as the internal ones. Very especially it is not possible to use external multimode fibers as input to a singlemode switch as well as external singlemode fibers as worker fibers between a multimode switch and the Wavelength Meter.

Generally please note that the usage of any multimode fiber in the light path will limit the measurement accuracy to the so called quick coupling accuracy. With a WS/7 this is 150 MHz. And if any singlemode fiber is used, this will limit the possible measurement range to the working range of the specific singlemode fiber.

The ends of each fiber attached to the switch or to the Wavelength Meter need to be FC/PC type (non angled; marked blue on fibers supported by us). So, the worker fiber between switch and Wavelength Meter needs to be FC/PC:FC/PC. Please use the delivered FC/PC:FC/PC fiber only (marked blue on both sides). FC/APC fiber ends (angled; marked green) only should be used at the laser side.

2.8 PID laser control (upgrade option)

During measurement it is possible to write the results to a plotter or to control up to eight different lasers simultaneously with analog PID regulated signals.



Having switched on this feature, the software calculates the difference between the measured and a previously set reference wavelength (controllable by a mathematical function input) and exports the belonging signal in relation to a specific preset range and sensitivity. Also it is possible to export error values and step informations as a voltage signal and to calibrate the output ports.

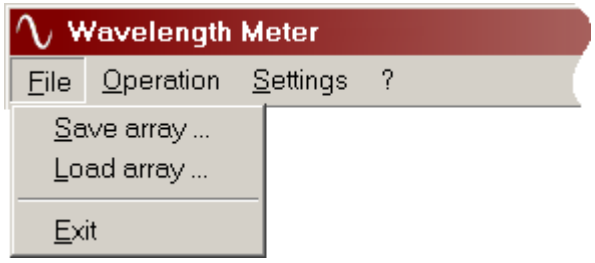
Adjusting all settings according to this feature can be done in a dialog box accessible via menu "Settings | Laser Control Settings ...". For detailed information please have a look at chapter 3.5 "PID Laser Control Settings (upgrade option)".

The output signals are enduring between single measurement calculations, not pulsed. The output voltage has a maximum covering range of -0 to 0 V with a step resolution of 0 mV (principally), according to a 0 bit DAC. The signals can be accessed using the LEMO connectors at the rear of the Wavelength Meter.

3 Program surface

3.1 Menu

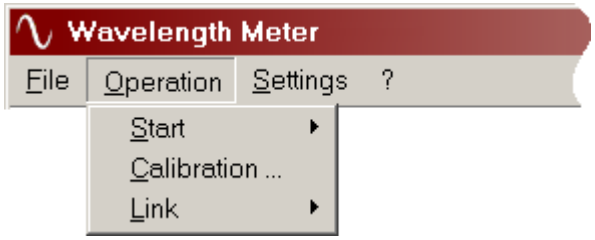
3.1.1 Menu File



- **Save Array …** stores a pattern snapshot of a finished measurement for further processing. You can choose a single measurement array format (*.smr) or the some larger text-based format (*.smx) if you want to workaround with an external editor. The files' formats are described in chapter 4.4 "File formats".
- **Load Array …** loads previously recorded arrays of wavelength measurements and displays it like as it was measured right now.

- **Exit** the program.

3.1.2 Menu Operation



- **Start/Stop** **Measurement:** Starts or stops the measurement.
  Adjustment: Starts or stops the adjustment mode. This has to be done as initial preparation of the Wavelength Meter and if considerable conditions have changed, like mechanical device movements or environmental changes.
  Recording …: Starts or stops a recording of an entire measurement with all its data. This is simply the measurement mode, which additionally saves all necessary measurement information to a file that you have determined previously. This recording can be launched later using the Replay option. Notice to serve a big amount of disk space cause on very small exposure values the measurement can take about 500 kB and more per second !

Note: *This option does not include a graphical representation of a long-term wavelength measurement! To obtain a long-term stability-graph of the wavelength, you can use the also available program "LongTerm" (see "Measurement Examples"). It can be called manually from within the installed directory or via "Startmenu | Programs | HighFinesse | Wavelength Meter… | LongTerm". It is also possible to get this program launched automatically. How to perform this, please have a look at "Extra Settings …" and there at "Start-behaviour | Extra program" and "Command line".*

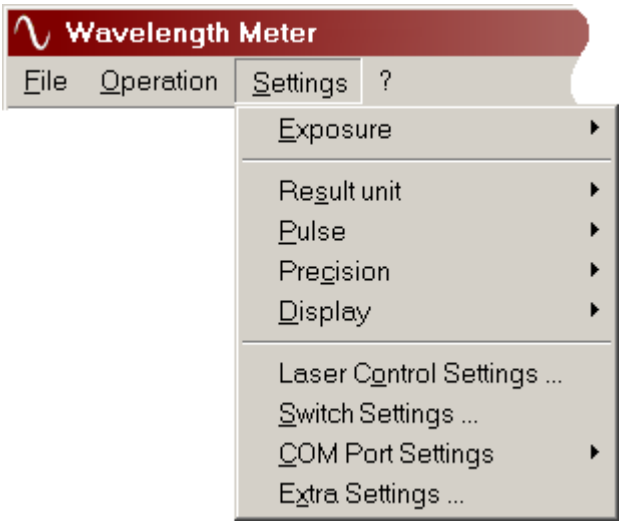
Replay …: Starts and stops a previously recorded measurement. For additional settings please have a look at "Replay behaviour" (menu "Settings | Extra Settings …").

- **Calibration …** Should be used periodically on known He-Ne laser signals to provide the measurement precision as indicated in the technical data manual. Or use Another laser periodically if other than He-Ne laser signals are wanted to be taken for calibration. This Wavelength Meter also especially can be calibrated using a free running He-Ne laser.

- **Blackbody radiation** **Store:** Each time after reaching working temperature after the 2 hour warm up (IR2 devices only) one needs to store the lasting radiation which then successively will be subtracted from each following measurement shot. But other than IR2 devices also should perform this procedure above a distinct exposure when the caracteristic noise becomes too big.
  Clear: For any subsequent call to store, the stored data first needs to be cleared again. Please also have a look at chapter "Removing blackbody radiation"

- **Link** **Connect COM-Port:** Connects the program to the RS232 port. This offers the possibility to transfer the measured wavelength with a serial cable to another computer.

3.1.3 Menu Settings



- **Exposure** To switch exposure control between automatic and manual. Values inserted in manual mode remain constant. In automatic mode, on measurement or adjustment the exposure value will be set by the program to fit a proper

measurement intensity. Please also have a look at chapter 3.2 paragraph "Exposure".

- **Result-Mode** Determines in which physical unit the calculated result value is to be displayed. This has no effect on exported values to externally attached user programs.

- **Pulse** **Continuous:** Measuring the wavelength of a Continuous Wave laser.
Pulsed: For wavelength measurements of pulsed lasers with the built-in optical synchronization method in pulsed measurement mode (instead of measuring pulsed lasers in continuous mode) please have a look at "Pulsed laser measurements".

- **Precision** **Fine:** To measure with full accuracy.
Wide: For wavelength measurements of wide-line lasers. Using this method the measurement is performed only by main Interferometers 1…5 with a precision of ± 0.001 nm.

- **Display** **Show Signal:** Controls whether the interference pattern shall be redrawn on a new measurement. Deactivating this option speeds up the calculation and in combination with small exposure slices this provides the highest speed of measurement.
Fast: Speeds up the calculation by redrawing fuzzier. In combination with small exposure slices this provides a little bit higher speed of measurement.
Fixed height: Stretches or diminishes the interference pattern that the maximum amplitude always appears attached to the allowed maximum amplitude. In some cases this can improve the visible pattern stability. If this option is used the "Intensity bar" option (see below) should be used also to not get the amplitude control lost.
Intensity bar: Displays the maximum amplitude of the interferometer pattern of the corresponding chart as vertical coloured bar. The bar is coloured green if the measurement intensity is alright and red if the intensity is out of bounds. The bar height always belongs to the interferometer pattern of the unzoomed chart with non height-fixed pattern even if the chart is zoomed or the pattern are fixed.
Swich mode (optional): Activates the settings and display options needed for usage with the fiber switch.
Capture signal: Captures the last shown signal on the screen.

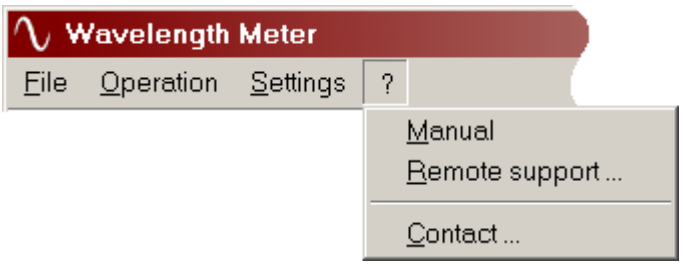
- **Laser Ctrl. Settings** Displays a dialog box to adjust the analog PID regulation signals. This option allows the measurements to be written to a plotter, e.g., or to control a laser to be locked or adjusted anyway.

- **Switch Settings …** Displays a small dialog box for fine tuning the switch channel separation. (optional fiber switch required)

- **COM Port Settings** **Switch Port …:** Displays a dialog box for setting the COM-port (RS232) parameters needed for proper access of the fiber switch. Please also have a look at paragraph 2.7 "Measurements with multichannel fiber switch (upgrade option)" Seite 15. (optional fiber switch required)
Link Port …: Displays a dialog box for setting the COM-port (RS232) parameters what is needed for wavelength export to another computer. For furher information please see paragraph 4.3 "Measurement result access (over COM-Port)" Seite 128.

- **Extra Settings …** Shows a dialog box where the program startbehaviour and the behaviour on replaying and recording of measurements can be adjusted. There are also additional options for the behaviour at the start of measurements. For detailed information please look at paragraph 3.3 "Extra Settings" Seite 28.

3.1.4 Menu Help

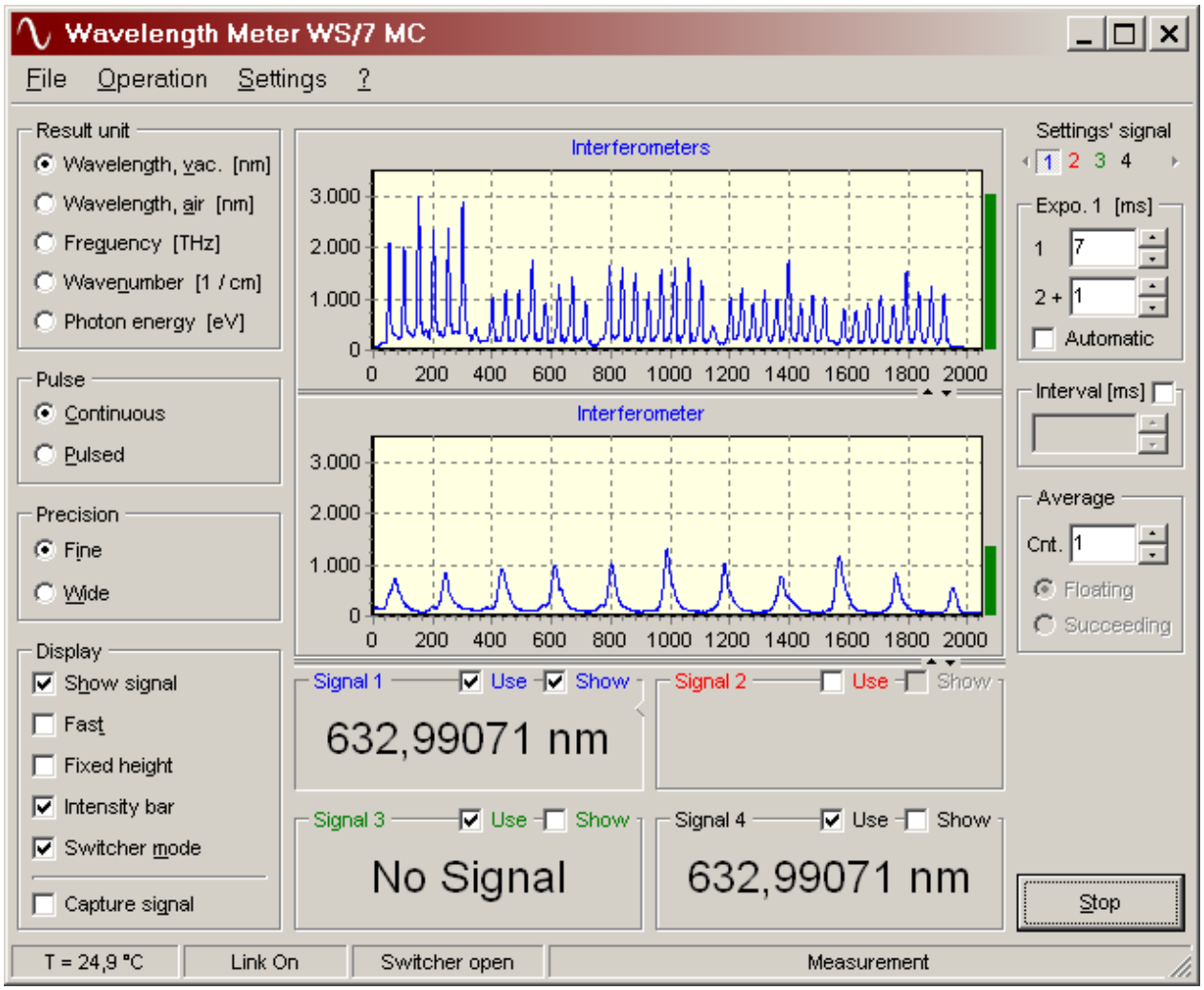


- **Manual** Displays this manual.

- **Remote support …** Using this function, you can establish a direct connection to our support team, obtaining help and troubleshooting.
  By the remote support, our service team is able to see the content of your screen and - if you agree - to work on your computer. During the session, every action is logged and could be provided to you if desired.
  In order to use the remote support, an internet connection is necessary and you need to have a personal consultant number which may be retrieved by our service team directly.
  You can reach our service team by phone, skype or by email (the actual data and addresses at printing time can be found at chapter 6 "HighFinesse Information-Service" Seite 135).

- **Contact …** Displays the Wavelength Meter version and enables access to the HighFinesse-homepage.

3.2 Control Elements



Group Result unit

Program 7-113nrsw4.bmpUse the controls in this group to select the desired result unit:

- vacuum wavelength, nm
- air* wavelength, nm
- frequency, THz
- wavenumber, cm^{-1}
- photon energy, eV

When more than one signal channel is available, it is possible to set each signals' (optional fiber switch and/or builtin channels') result unit independently. This possibility can be adjusted in the "Extra settings" dialog (Menu "Settings | Extra Settings …", sheet "Various").

* Air wavelength is calculated for the standard dry air (15 °C and 760 mm Hg).

Group Pulse

- **Continuous:** Measuring the wavelength of a Continuous Wave laser.
- **Pulsed:** For wavelength measurements of pulsed lasers with the built-in optical synchronization method in pulsed measurement mode (instead of measuring pulsed lasers in continuous mode) please have a look at "Pulsed laser measurements".

Group Precision

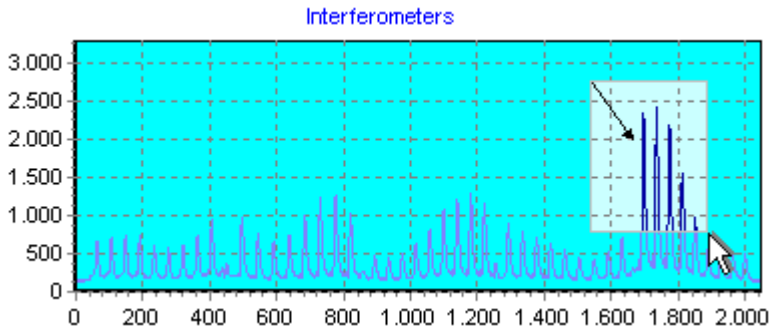
- **Fine:** Switch to this mode to measure with full accuracy.
- **Wide:** For wavelength measurements of wide-line lasers. Using this method the measurement is performed only by main Interferometers 1…5 with a precision of ± 0.001 nm.

When more than one signal channel is available, it is possible to set each signals' (optional fiber switch and/or builtin channels') precision setting independently. This possibility can be adjusted in the "Extra settings" dialog (Menu "Settings | Extra Settings …", sheet "Various").

Group Display

- **Show signal:** Controls whether the interference pattern shall be redrawn on a new measurement. Deactivating this option speeds up the calculation and in combination with small exposure slices it provides the highest speed of measurement.
- **Fast:** Speeds up the graphical pattern representation by drawing fuzzier. In combination with small exposure values this provides a little bit higher speed of measurement.
- **Fixed height:** Stretches or diminishes the interference pattern that the maximum amplitude always appears attached to the allowed maximum amplitude. In some cases this can improve the visible pattern stability. If this option is used the "Intensity bar" option (see below) should be used also to not get the amplitude control lost.
- **Intensity bar:** Displays the maximum amplitude of the interference pattern of the corresponding chart as vertical coloured bar. The bar is coloured green if the measurement intensity is alright and red if the intensity is out of bounds. The bar height always belongs to the interference pattern of the unzoomed chart with non height-fixed pattern even if the chart is zoomed or the pattern are fixed.
- **Switch mode:** Activates the settings and display options needed for usage with the optional fiber switch.
- **Rear channel:** Toggles between the front and rear input of the Wavelength Meter
- **Capture signal:** Captures the last shown signal on the screen.

Interferometer pattern charts



In enlarged mode there will two different interference patterns be displayed, the upper one represents the recording of 5 Fizeau interferometers and the lower one of an additional long Fizeau interferometer which together result in the high measurement precision.

The recorded interference graphs can be resized (zoomed) and moved inside the charts by mouse dragging.

To zoom the graph, the area to be enlarged must be selected with the mouse from its upper left corner direction bottom right with pressed left mouse button. Selecting diagonally upwards sets back the graph to its original state.

To suppress redrawing of the interference pattern, what leads to faster recalculation, you can switch on the ‘Fast’ mode setting (using the checkbox in control group ‘Mode’ or the entry in menu ‘Settings | Mode’).

You can move the graph in any direction by pressing the right mouse button while moving the mouse and if two charts are displayed you can resize them using the splitter control between them.

Any recorded interference patterns (and the calculated results, too) will be saved on the screen even if the measurements have been finished.

Between the graphs there's a little double-lined splitter control. Make use of it with your mouse to resize the charts mutually.

Result value

On successive measurements, the recorded interference patterns will be displayed in the corresponding charts to each interferometer group and the calculated wavelength in this result-panel below.

If an optional fiber switch is available and the switch mode is set, the display is changed to show up to eight single result panels, each of which can be selected independently to be used for measurement, for displaying its corresponding interference pattern in the charts above and, if marked active, for controlling the signal number whose pattern intensity bar is shown. A result panel can be marked active by simply clicking to it or by checking one of its checkboxes, the active panel is displayed sunken with a small triangle right hand side. The active panel does not necessarily correspond to the active settings' signal, toggling the active panel also sets the active settings' signal, but not versa. In non switch mode the active channel is set by selecting one of the small channel buttons in this result panel.

In case of a bad or disappearing signal the result-panel will indicate this instead of displaying a calculated result. The result panel can be resized using the thin splitter control above to fit the displayed values' font size matching your needs.

Settings’ signal (optional fiber switch required)

In switch mode you will be served with up to eight buttons to control the signal/channel which the exposure settings, the result unit, the precision and the averaging settings actually display the settings of and can be changed for. Different exposure values and automatic modes can be set to be able to adjust each signals' magnitude independently and different result units and precision settings channel by

channel if desired, additionally each channel can be averaged controlled by its own settings. The settings' signal also can be altered by changing the active result panel (see ''Result value'' above), changing the settings' signal but does not change the active result panel. The general ability of the exposure, the result unit, the precision and the averaging settings to behave channel dependent can be set in the ''Extra Settings'' dialog (Menu ''Settings | Extra Settings …'', sheet ''Various'').

In non switch mode the same functionality is available. The active channel is selected by pressing one of the small channel buttons in the result panel.

Exposure

Exposure control for interferometers. The displayed value is interpreted in milliseconds. Using the vertical arrows you can set the exposure value from 1 up to 9999 ms, also you can enter the required value directly into the edit control.

Higher exposure values force the measurement to a higher sensibility but also to a smaller repetition rate. You can leave the program to decide about the best exposure setting with selecting ''Automatic'' exposure here or in menu ‘Settings | Exposure’. The borders for this process can be set in menu 'Settings | Extra Settings …' within the register tab 'Various'. The automatic control is preset to work up to 2 seconds at most to not leave the measurements feel too unresponsive.

The mechanism of the exposure values is as follows: The ccd displayed as the upper graph is exposed using the upper exposure value (no. 1; at least 1 ms), the lower one is exposed with both the exposure values in addition. The automatic exposure control adjusts trying to set the maximum of the ccd intensity to a range between 1000 and the maximum of the unzoomed graphs (~ 3000). To avoid oscillations, the values will be left unchanged if both the intensities work in proper range.
If an optional fiber switch is available, you will be served with up to eight different exposure value sets to be able to adjust each signals' (switch channels') magnitude independently (see ''Settings' signal'' above).

Interval

The minimum measurement interval. It limits the possible measurement rate. The settable resolution is 10 ms, but it only shows effect if chosen larger than the exposure. The resulting measurement rate always is limited by the maximum of exposure and measurement interval (and additionally by some other speed related settings, like displaying of the pattern, and by computer speed and the cpu usage of other processes). The exposure but is not influenced by the interval setting, for sure.

The measurement interval setting is an overchannel setting. So, in switcher mode (optional fiber switch required), the same value is taken for each channel. If a channels exposure is bigger than the interval setting, the belonging real interval is bigger, too.

Average

This option allows to average the displayed and exported result.

The averaging is switched on by a value larger than 1, and switched off again at 1. 'Floating' ever averages the last n measurements, except if there are less than n available and 'Succeeding' waits for n measurements and then hands out the average. If with the 'Succeeding' option after an errormessage has occurred and the count of measurements still is not fulfilled to average, the last valid wavelength value will be redisplayed but not exported.

When the “Pattern” switch is checked, this option averages the recorded interference pattern prior to calculation. The so calculated wavelengths in this case but are not further averaged. This especially can help with low power signals with a bad signal to noise relation. It but can also be counterproductive in case of heavily instable or tuned signals. With tuned or moving signals it will create a wavelength shift of

half the averaging count in best case. In worst case it also can dish the original interference when the signal is moving too fast.
With unchecked “Pattern” the intererence pattern are left as they were read, the wavelength of this original pattern is calculated and later the wavelength values’ series is averaged.

When more than one signal channel is available, it is possible to set each signals' (optional fiber switch and/or builtin channels') averaging settings independently. This possibility can be adjusted in the "Extra settings" dialog (Menu "Settings | Extra Settings …", sheet "Various").

Start / Stop

Starts the measurement. After pressing the 'Start' button, its caption changes to 'Stop' and vice versa.

Status Bar

The following information is reported in the status bar:

- Temperature inside the optical unit (Fizeau interferometer versions and Laser Spectrum Analyser only);
- Result exporting COM-port state;
- Switcher COM-port state (if the optional fiber switch is available and only if it is connected via an external RS232 COM-port; Note: if you use a switch which is connected with the Wavelength Meter directly, “no switch” is displayed);
- Current state of the Wavelength Meter

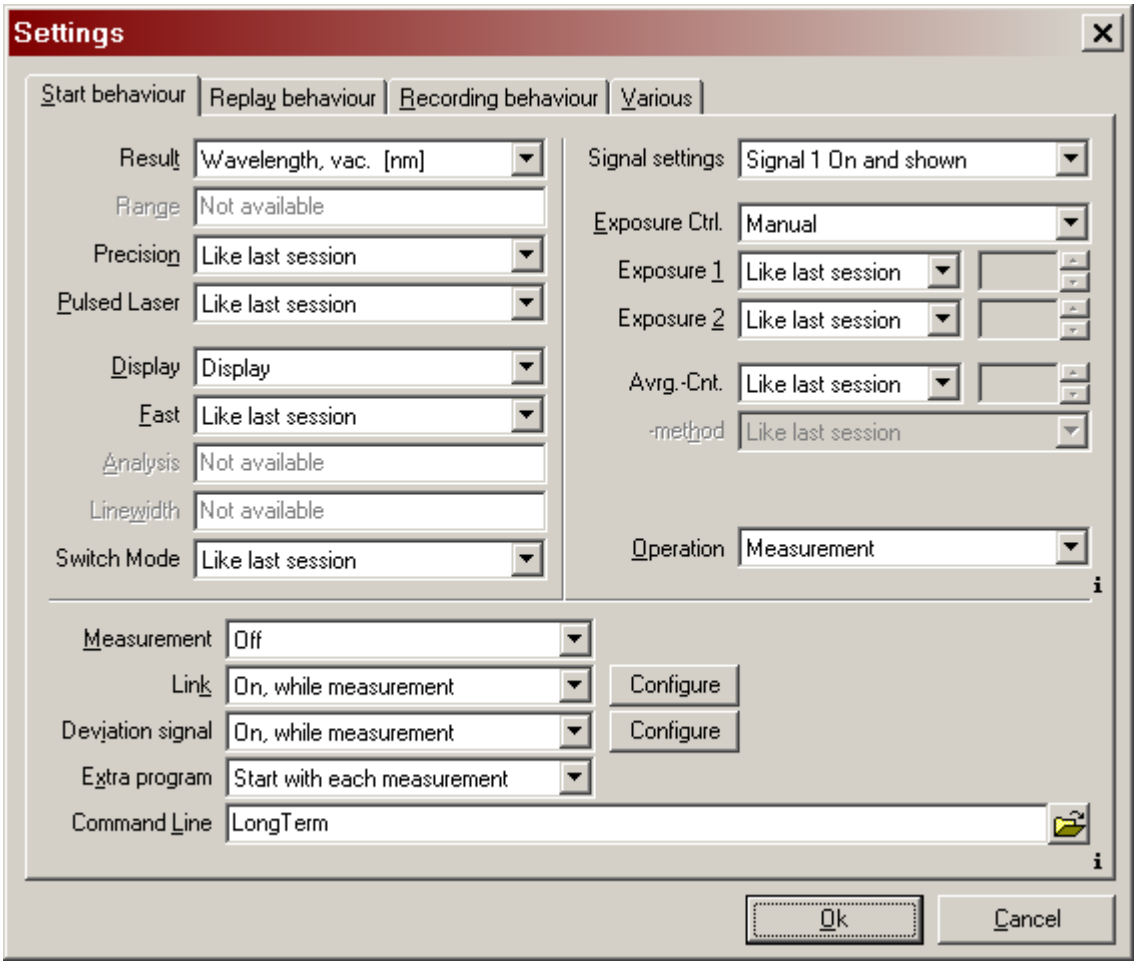
3.3 Extra Settings

There are some additional program settings available to be made. The following options can be accessed with the menu 'Settings | Extra Settings …':

- To find information about influencing the programs' start-behaviour, please look at 3.3.1 Start-Settings.
- To find information about settings of the behaviour of replaying recorded measurements, please look at 3.3.2 Replay-Settings.
- How to adjust automatically recorded measurements you can find at 3.3.3 Recording-Settings.
- To find information about additional settings, please look at 3.3.4 Various Settings.

3.3.1 Start-Settings

This Start-Settings dialog box, accessible via menu 'Settings | Extra Settings …', enables you to set the programs' start-behaviour. For all possible program settings you can decide whether they shall represent a special state on program start or be like as the program was closed the last time ('Like last session'). Additionally it is possible to set some automatically starting activities.



Note: *In various Wavelength Meter versions, some parameters are not accessible or not available. This is indicated with 'Automatic Detection' or 'Not Available'.*

Below, the existing possibilities are listed.

Options

Result: *(mode of the calculated result value displayed on the result panel)*
- Like last session
- Wavelength, vac. [nm]
- Wavelength, air [nm]
- Frequency [THz]
- Wavenumber [cm^{-1}]
- Photon energy [eV]

Range: *(Automatic Detection)*

Precision:
- Like last session
- Fine
- Wide *(the interferometer/grating with the smallest FSR is ignored)*

Pulsed laser:
- Like last session
- Continuous Wave
- Pulsed *(triggered internally.)*

Show signal:
- Like last session
- Don't display *(fastest calculation setting)*
- Display

Fast:
- Like last session
- Off
- On *(Interference pattern will be drawn with less pixels \ faster)*

Analysis: *(Not Available)*

Linewidth: *(Not Available)*

Switch Mode: *(optional fiber switch required)*
- Like last session
- Off
- On

Signal Settings: *(optional fibr switch or double pulse mode required)*
- Like last session
- Signal 1 On and shown
- Signal 1 On; not shown
- All signals On and shown
- All signals On; none shown

Exposure control:
- Like last session
- Manual
- Automatic *(Exposure will be fit by program while measurement)*

Exposure 1 / 2: *(No. 2 is version dependent.)*
- Like last session
- Following value *(Explicitly entered exposure value, as shown)*

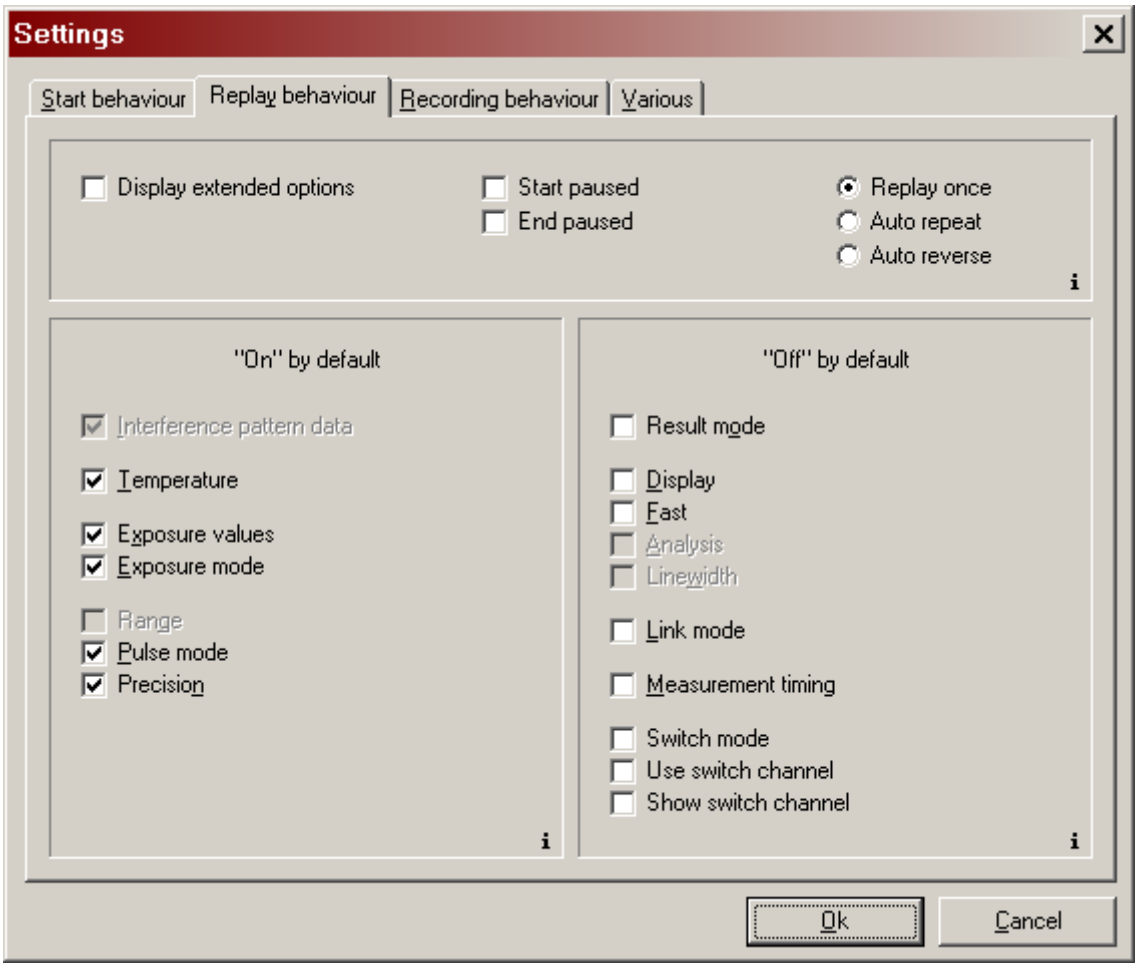
Average Count:
- Like last session
- Following value *(Explicitly entered averaging count, as shown)*

Averaging method:
- Like last session
- Floating *(After every measurement each "count" values are about to be averaged)*
- Succeeding *(Always "count" measurements are collected before an average is displayed)*

Operation:	• Like last session • Adjustment • Measurement
Measurement:	• Off • On, start with program
Link *(with COM-port)***:**	• Off • On, start with program • On, while measurement
Deviation/PID signal:	• Like last session • Off • On
Extra program: *(possibility to start an extra program on start of the Wavelength Meter or measurement)*	• None • Start with program • Start with measurement once *(only with first measurement activity)* • Start with each measurement *(control yourself to close the called program each time before)*
Command line:	*The command line string to be executed if 'Extra program' is selected*

3.3.2 Replay-Settings

The Replay behaviour dialog box can be reached by way of menu 'Settings | Extra Settings …'. It enables the setting of the behaviour on replaying recorded files.



In this picture you can see all the information that is implemented in a recorded long-term measurement. On replaying a recorded file the program will set the corresponding values according to the checked options in this dialog.

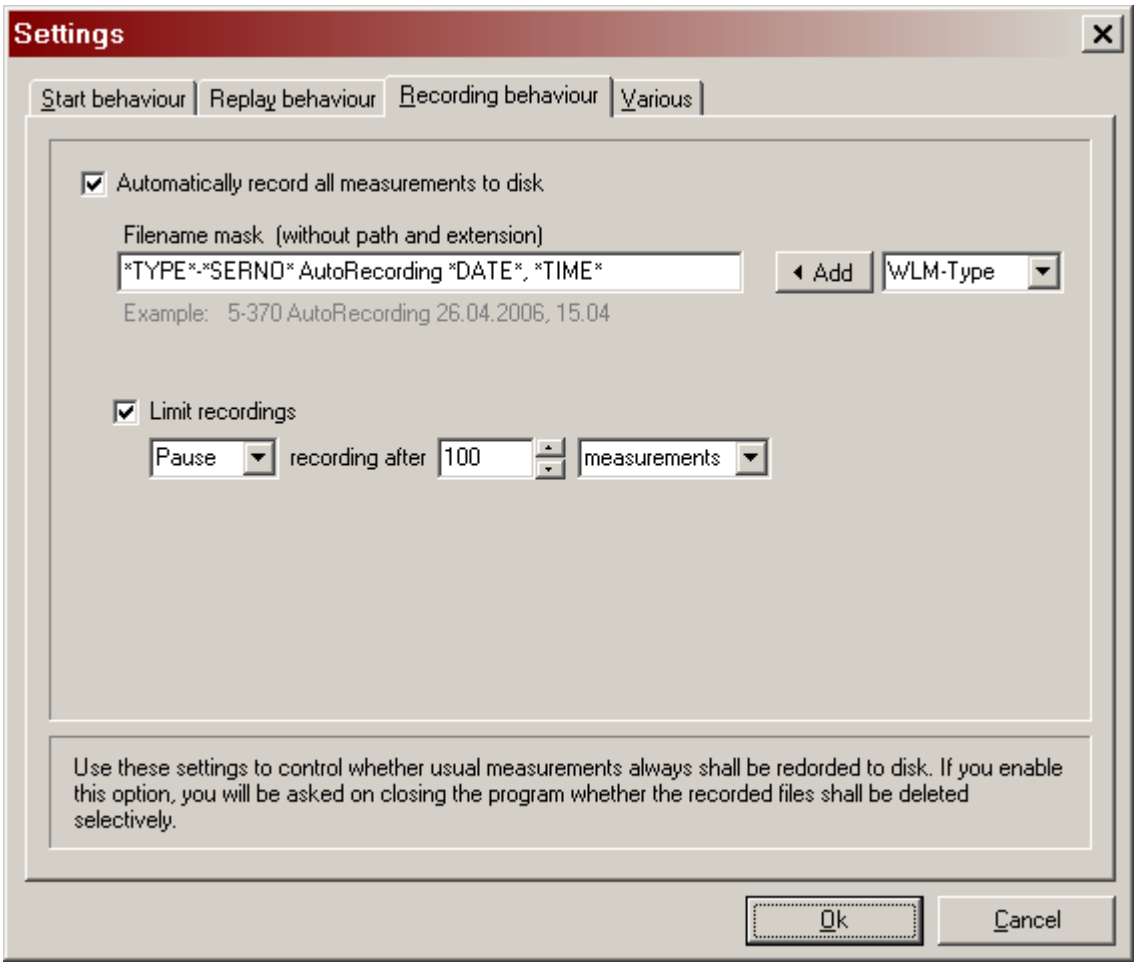
The options in the left area ("On" by default) are recommended to be checked.

Additionally it is possible to influence the running behaviour in replay mode. With 'Start paused' selected, the replay just starts when the 'Play' button is pressed. 'End paused' only takes effect if 'Replay once' is selected, it causes the replay dialog to stay on screen when the last measurement is replayed. It also is possible to replay a recording non-stop. 'Auto repeat' restarts from the beginning after each pass and 'Auto reverse' changes the replay direction when the end or the starting point is reached. To also display these options in the replay navigation dialog, check "Display extended options", this setting but requires a program restart to take effect.

3.3.3 Recording-Settings

In this section it is possible to adjust whether recorded measurements shall calculate live or not. If chosen to not calculate live, this provides the highest possible measurement repetition rate and the recorded data can be calculated analyzed later on replaying the file.

Additionally it is possible to switch on and adjust automatically performed recordings in usual measurement mode. When this option is used, all measurements are recorded to disk automatically even if one does not explicitly select the recording mode.



In this dialog box you can set the filename mask for the recorded files (since you are not being asked about a name on normal measurements) and whether the recordings shall stop when a specific value is reached.

The filename mask is the name of the recording files without their path and extension portions. In case of file name repeatings new created files will be equipped with individual numbers. Using the button "Add", you can insert the value selected in the options box on the right hand side. On measurement these mask identifiers (encircled by asterisks) will be resolved to values like visible in the grayed example below. Change the string right as you please.

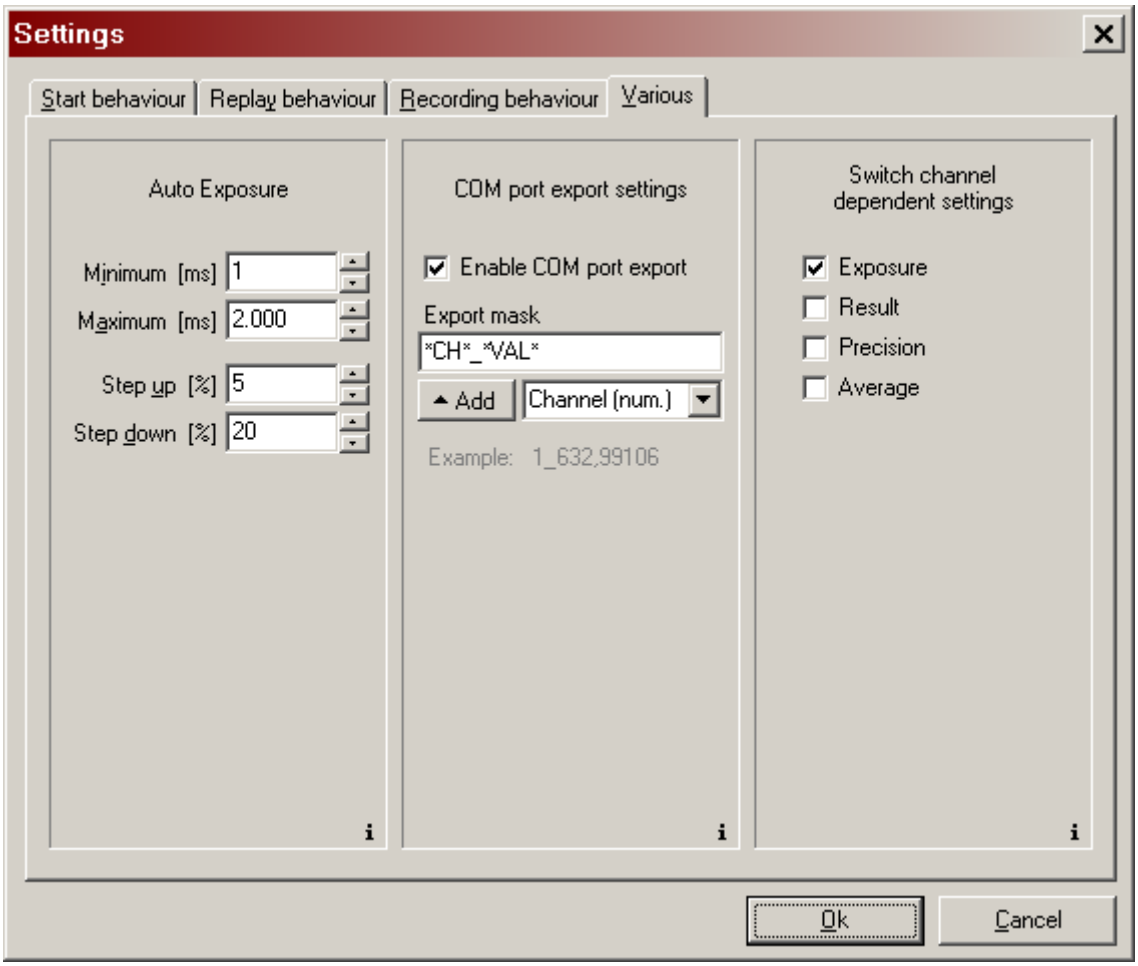
It is possible to limit the recorded files automatically to a specific count of measurements, a maximum file size, or to a predefined recording time. When the recordings are paused, the measurement still will go on and you can decide when to stop it using the button "Stop" in the main program window or in the recording status window. When you decide to stop the recording after a specific value has been reached, also the corresponding measurement activity will be stopped. (Note: When chosing to limit to a maximum

file size, the file will be a few kB bigger in size than given because the size overrun will be detected not before the last measurement shot has been written.)

Before closing the program you will be asked to delete several or all of this instances' automatically recorded files. A dialog appears where one can select the files selectively and delete them.

3.3.4 Various Settings

The fourth sheet of the Extra-Settings dialog allows you to set the behaviour in automatic exposure mode. This option is useful especially for largely exposed measurements and for measurements with varying input energy, both to minimize the sluggishness once the exposure has been set too high. With Minimum and Maximum you can set the range in which the exposure is adjusted. Step up and Step down represent the change in percent of the actual exposure value.



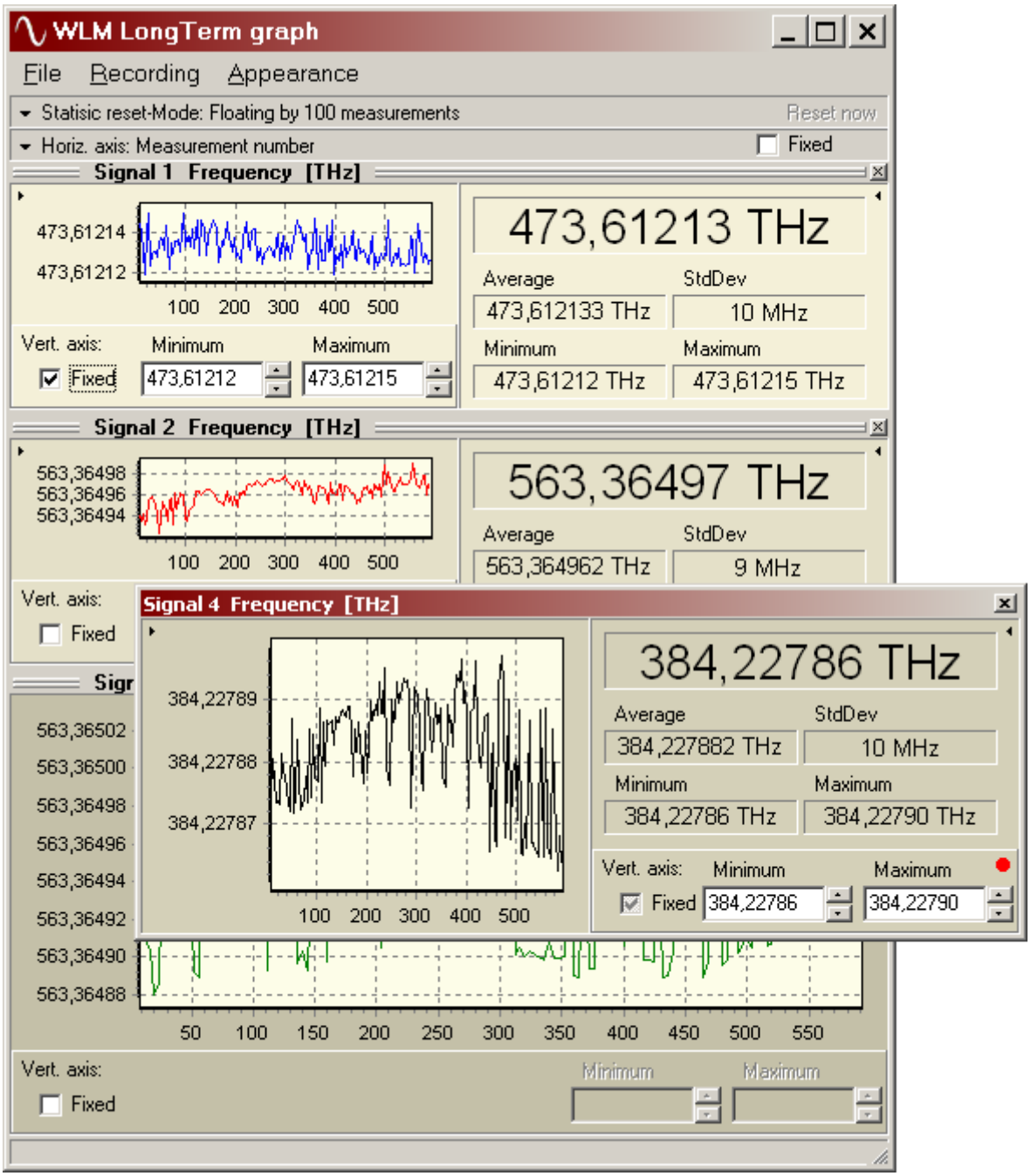
It also is possible to dis-/enable the wavelength export over COM port and vary the style the resut values pass along the COM port. It is possible to use any character but the asterisk. The asterisks themselves are used to encapsulate variables. There are three variables able to be used: *VAL* represents the value in the true sense to be exported, and with *CH* and *CHALPH* the belonging fiber switch channel (if any) can be added in numerical and alphanumerical notation. The necessary final carriage return is added automatically and can not be inserted here.

If an optional fiber switch is available, it additionally is possible here to adjust whether the exposure settings, the result unit, the precision and the averaging settings shall behave channel independent or not. If set dependent, it is possible to set different exposure values and automatic modes, result units,

precision and averaging settings signal by signal. If one of the checkboxes is unchecked, the corresponding option always will be valid for all signals/channels.

3.4 LongTerm graph

Beneath the small longterm charting demonstration programs (with sourcecode, see chapter 4.2) there also is a fully functional longterm charting program included which separates each signal (for multi channel switches also) to an own chart and also displays a few statistical values beneath the charts.



The image shows the longterm graph of a Wavelength Meter WS/7 in 4 channel switch mode.

In Switch Mode the LongTerm program provides the possibility to display every channel of up to 8 different laser sources in its own LongTerm window each. Additionally (depending on the Wavelength Meter version) it serves with the ability to display the linewidth, the signal distance, the analoguous input and output for laser deviation or PID control and the temperature.

Each chart can be switched on and off in the menu "Appearance | Charts". The individual charts can be docked to the main LongTerm window or displayed independently using the "drag and drop" bar at the top of the chart.

It is possible to store the results to disk and to reload from disk (menu File). The so stored lta (long term array) files consist of wavemeter information, information about the number and the settings of the displayed charting frames and the measurement result data itself. The data is arranged in columns and each line starts with recording timestamp and additionally consists of the single values of one or more of the charting frames, separated by tabulators. The data values appear as exported by the Wavelength Meter and represent the values as the corresponding Get-function would return (see chapter 4.1.1 "Exported functions and constants"). In case of special values like for "Big Signal" these values are stored to file, too (in this case ErrBigSignal for instance, what is -4). The lta files but do not include raw measurement data, only the final results, it is not possible to conclude from lta file data to the measurement itself. The lta files are useful for further processings of the final results. To better be able to reanalyze your measurements with all its conditions and raw data, record the measurements to ltr files (longterm recording files by menu “Operation | Start | Recording …” in the Wavelength Meter main application, see section 3.1.2 “Menu Operation”).

In menu recording one can start and stop the current data acquiration process and also clear all collected data. On start of the program the data acquiration is active.

Group Statistic Reset

This group controls the behaviour of the calculations whose results are shown in the windows. Expand to the options' view with clicking the small arrow on its left hand side. The value "Floating by…" sets the number of measurements from which the statistics are calculated. To display the statistic values, please click the small arrow right hand side beneath the chart. The additional visible but disabled options will be completed soon.

Group Horiz. Axis

Expanded using the small arrow, the radio buttons on the left hand determine what is displayed on the horizontal axis. At this moment only the measurement number and time are available. With the checkbox "Fixed" one can choose whether the right end of the horizontal axis floats with the measurement counts or is fixed, so that the whole displayed section jumps half the way when reaching the right end.

Chart Windows

If the small arrow on the upper right is clicked, each LongTerm window displays the wavelength measurement as well as the calculated mean value, minimum, maximum and the standard deviation of the lasers and other signals. It then is possible to hide the chart section of this window to only display the statistics section. To do so, please click the small arrow on the upper left hand. It is not possible to hide both sections: if the statistics portion is hidden using the arrow on the right hand, at least the chart section will be displayed.

With the check box "Vert. Axis Fixed" the upper and lower limits of the vertical axis are kept fix and can be edited with the "Minimum" and "Maximum" spinboxes.

As in the main program window it is also possible here to zoom and move the charts directly with the left and right mouse button respectively. A small red point appears subsequently on the lower right indicating

that the range zoomed by the mouse differs from the absolute fixed range (Checkbox „Axis Fixed"). Click that red button to store the displayed range as default. After a restart this range is loaded.

Settings (menu "Appearance")

General: If measurements return a logical error value senseless to be displayed in a long-term chart or if a chart lacks of a specific value in spot to another chart, it here is possible to adjust how to visualize this. It is possible to display such an occurrence as a gap or to simply connect the surrounding points.

Idler (Data collection of measurements with more than just one Wavelength Meter and/or Laser Spectrum Analyser): If more than one device is started at a time, each available option of each of the devices is displayed in the submenu "Charts" of the"Appearance" menu. Additionally there are options to display the Idler values taken from simultaneous pump and probe experiments.

For this Idler calculation especially there is an option available to adjust the interpretation of the arriving signals order. This is important for triggered pulsed measurements as well as for cw measurements. Since just one cpu is involved, it principally is impossible to get both signals simultaneously and to export them. The order in which simultaneous measurements can be retrieved by the devices is some kind of accidentally that way and needs a further interpretation.

The following three options can be found at menu "Appearance | Settings …" sheet "Idler":

- The software assumes that the result signal always follows the pump signal.
- The software assumes that the pump signal always follows the result signal.
- The software handles values to only match if they appear within a specified interval independent of their order. The interval in [ms] can be entered along with the option.

However, we recommend to trigger simultaneous signals with different trigger settings that they appear with a specific phase shift and the order that way is forecastable.

AutoSignals: Adjust here whether new arriving signals shall open the corresponding chart automatically on first occurrence. When a new measurement only information is detected by the application, the corresponding chart will be displayed automatically when this option is selected. After closing this chart it must be opened manually to display it again, it is not going to be redisplayed automatically again until the software is restarted.

AutoSave: Here you can set whether the current data shall be saved to disk and whether you want to be asked beforehand. It is possible to save automatically on closing the program, on resetting the current data (menu “Operation”) and when the computer enters a power saving mode like suspending to disk or standby. Switch on “Ask before” if you want to be asked before saving and getting the possibility to modify the filename.

3.5 PID Laser Control Settings (upgrade option)

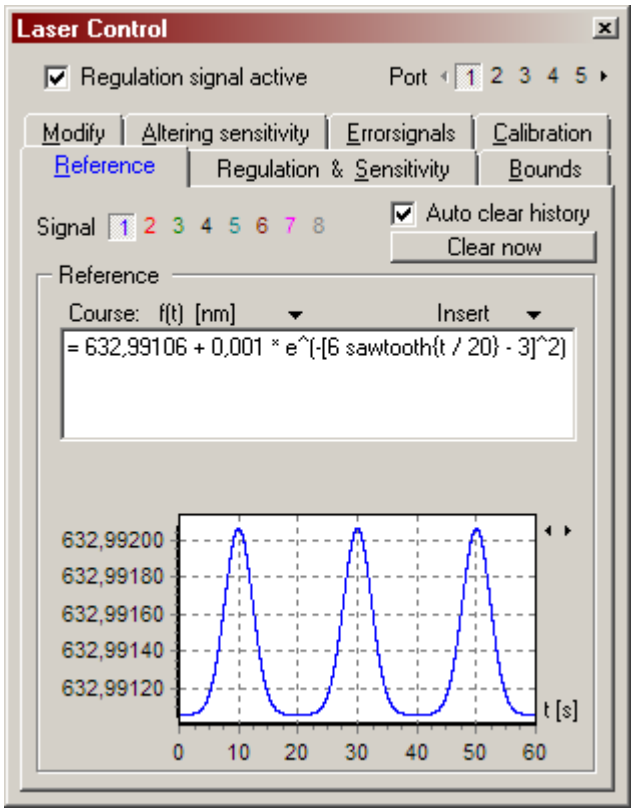
These settings control the options available for the PID regulated laser control signals. They can be reached over menu “Settings | Laser Control Settings ...".

All settings but the main switch “Regulation signal active” made in the following dialog box are resident between program calls. This also means that the settings of the last session are active when the program is loaded even if this dialog box is not opened newerly. (This also has a special effect for the settings of the sheet "Altering Sensitivity" if the deviation control generally is switched on and you have chosen a large signal delay.) All settings but those in the sheets "Altering sensitivity" and "Errorsignals" are made for each available output port selectively. To switch between the ports’ settings, use the small numbered "Port" buttons on the upper right.

All voltage values able to be entered but those in the sheet "Calibration" and “shot-per-shot change” in the sheet “Bounds” have a range of -0 to +0 V.

Please also have a short look at paragraph 2.8 "PID laser control (upgrade option)".

3.5.1 Reference



If a multichannel fiberswitch is available, use the small coloured "Signal" buttons to adjust the switch channel to be used with the selected output port.

Here the reference wavelength course can be entered. It is possible to enter any arbitrary mathematical function to leave the regulated laser follow it (the only limitation is that the laser principally must be regulatable to this function course). The function takes the time (t) as variable and its result is interpreted in the unit selected above. (Note that the so selected unit also applies to the sensitvity settings in the "Regulation & Sensitivity" sheet. So, a change of the unit very probably also requires at least changing the sensitivity leading factor or the regulation parameters P, I and D, even if the different courses for different units would match each other exactly.) Whenever the function is changed, it will be proven for syntactical and logical errors as well as for range exceedings. Under the function edit box there is a field displaying error and exceeding messages. Additionally it is possible to watch the function result in the graph below. On any error and range exceeding result, the main regulation switch will be unchecked and deactivated. Later reactivation needs to be done manually. When the function is changed without any errors inbetween, it directly will be applied for a possibly active regulation, but please note that the regulation history memory of the selected port will be cleared and reinitialized in this case if the "Auto clear history" box is checked. If unchecked the history will be cleared only on function syntax or range errors and on manual restart of the main regulation switch "Regulation signal active". Using the "Clear now" button the history can be cleared and reinitialized manually any time, even during an active regulation.

All mathematical terms able to be used are listed in the box reached by the "Insert" button. Selecting an item of this list causes the item to be inserted to the function at the current selection. Currently marked selections will we placed as argument of new inserted functions.

Possible function ingredients are the function variable **t**, the functions **sin**, **cos**, **tan**, **cot**, **asin**, **acos**, **atan**, **acot**, **exp**, **ln**, **lg**, **lb**, **sqr**, **sqrt**, **rectangle**, **triangle**, **sawtooth**, **frac**, **fracvar**, **isotrapezoid**, **thimble**, **stairs**, **stairsrelax**, **motdrive**, **heaviside**, **abs**, **sign**, **round** and **trunc**, the constants **pi**, **e** and **c0** (vacuum speed of light), the operators **+**, **-**, *****, **/**, **^**, **(**, **)**, **[**, **]**, **{** and **}**, and all numeric values.

rectangle, triangle, sawtooth, frac, fracvar, isotrapezoid and thimble are 1-periodic functions and (except for fracvar) have a result range of [0, 1]. fracvar has a result range of [-0.5, 0.5]. The bracket signs (, [and { do not differ as well as),] and }, so it is possible to close a bracket by } which was opened by [. And additionally the colon and the dot do not differ, they both are taken as decimal separator and there is no thousands separator allowed. The multiplication operator usually is not necessarily needed between terms, except in front of e and exp to enable scientifically entered powers of 10 like 1e5 (100000). e^x is the same as exp(x) and for x > 0 sawtooth(x) is the same as frac(x).

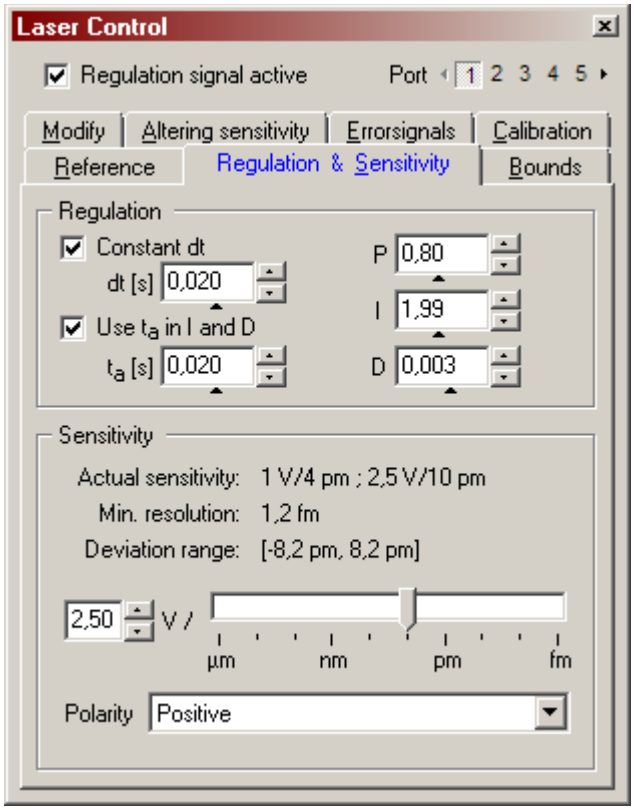
3.5.2 Regulation & Sensitivity

If a multichannel fiberswitch is available, use the small coloured "Signal" buttons to adjust the switch channel to be used with the selected output port.

***Regulation*:** This is the heart of the laser regulation. The P (proportional), I (integral) and D (derivative) parameters must be set to properly match the entire measuring and controlling system. They follow the general PID regulation function

$$Output(t) = S \cdot \left[P \cdot err + I' \cdot \int_0^t err \cdot dt + D' \cdot \frac{derr}{dt} \right]$$

where S is the sensitivity (or signal amplification, see below). t_a does not need to be used imperatively, it is meant to reduce the system timing influence of the I and D parameters. Used with t_a, it means that I' = I / t_a, and D' = D * t_a and the parameters I and D stay comparable and are easier to interprete. Used without t_a, it means I' = I, and D' = D, and with a small system time constant the parameter I usually would be very large, whereas D would be very small. (The number of resolution digits of t_a, P, I and D can be modified by the small triangles in case they are too or too less accurate for proper handling.) Setting dt constant (so, changing to Δt), the live timing behaviour can be eliminated completely, and does not disturb on drastic timing changes, such as adding or removing additional measurement channels or switching on/off interference pattern drawing. If dt is not taken constant, the real timing is used.



Changes of the regulation and sensitivity parameters will be applied to the current regulation process (if any) immediately. But unlike with changes to the reference function course

(see sheet "Reference") in this case the regulation history memory will not be cleared.

***Sensitivity*:** The sensitivity can reach from 1 V/µm to 9.99 V/fm (resp. 1 V/PHz to 9.99 V/MHz, 1 V/(1e4/cm) to 9.99 V/(1e-5/cm) or 1 V/eV to 9.99 V/neV, selectable in the sheet "Reference"). It will be set compound by a leading factor (1 to 9.99) multiplied with a guiding factor wich has 10 possible positions from 1V/µm to 1V/fm (logarithmic scale) (or in Hz, 1/cm or eV). When you use sensitivity altering (sheet “Altering sensitivity”), note that only the guiding factor is about to be altered.

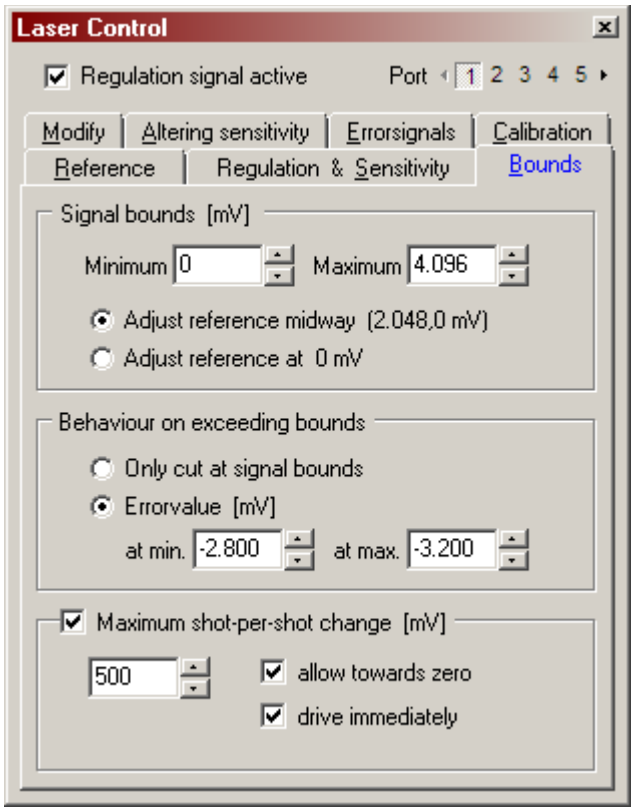
With Polarity you can turn around the signal sign if needed with your electronic equipment arrangement.

The range and resolution informations displayed reflect the actual settings. The "Deviation range" information as well as the exported voltage signal also depend on the signal bounds within the sheet "Bounds".

An example with simple proportional settings (P = 1, I = D = 0):
The reference wavelength is set to 632.99106 nm and the currently measured laser wavelength is 632.99110 nm, 40 fm above. With a positive polarity and the sensitivity set to 1 V/100 fm the deviation signal would be 400 mV if the reference was adjusted at 0 V and the other settings in the sheet "Bounds" allowed that.

Finding good regulation parameters sometimes can be a tricky thing. Please also have a look at the PID simulator (see paragraph 3.6 "PID simulator"), an application to help getting a better understanding the PID regulation parameters (and there especially paragraph 3.6.2 "Finding the best regulation parameters using the PID simulator"). This extremely can reduce the time and the uncertainty on finding good parameters and helps getting a better *feeling* with handling the regulation parameters.

3.5.3 Bounds



***Signal Bounds*:** Sets the range that shall be accessible for the reference deviation, you can access the entire range (-0 to 0 V) or any other in between. If the absolutes of both the values are different, it is possible to adjust the reference wavelength to "0" or to the center of both the values.

***Behaviour on exceeding bounds*:** If the signal exceeds the signal bounds' range, there's a possibility to leave the signal simply being cut or to apply an other specific entered value. The values to be entered here make no sense to overlap with the signal bounds interval. Also they should vary from all the other used voltage values in any of the other sheets. Otherwise you would not be able to recognize the signal.

The sheet "Altering sensitivity" has a delay and a finalizing functionality. These are not used for the signal in the true sense, not here and not in the sheet "Errorsignals".

***Maximum shot-per-shot change*:** Enables to limit the maximum output voltage change in order to protect very sensitive electronic equipment. Depending on the adjusted parameters and on the laser behaviour it might be possible that the

output voltage changed from one end to the other end of its range, what is more than 0 V (delta, absolute) with this WLM. To not have to restrict the maximum allowed output voltage range, this option allows to separate large changes to smaller parts per change.

The maximum allowed change value can be set from 1 to 10000 mV. Also it is possible to allow larger changes than the given one for changes direction to 0 V. If the “allow towards zero” option is checked, so a change towards 0 V is enabled nevertheless, independent of the adjusted limitation value. If for instance the previously output voltage was 3 V, and the calculated new voltage would be 1 V, these 1 V would be put out even if the change was limited to 100 mV only. If, in this example, the calculated new voltage would be below 0 V, only 0 V would be put out.

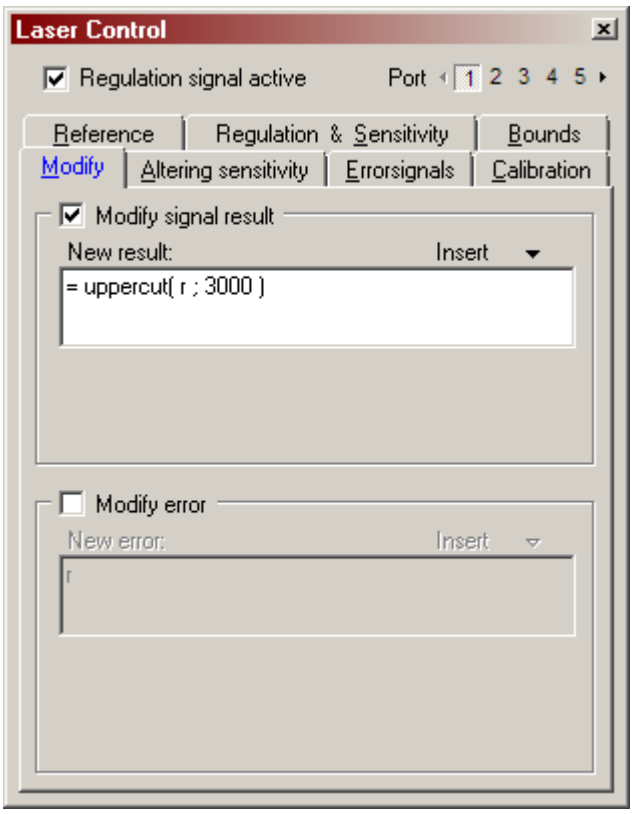
“drive immediately” controls whether on measurement the voltage is limited and put out only once per measurement event or driven to the calculated voltage in limited steps immediately after the measurement event. If this option is not selected and the voltage is changed once per measurement event, large regulation changes can cause a lazy regulation if occurring in combination with tightly limited output changes. The minimum delay between successive changes during measurements in this case is the measurement repetition time, usually in 10 ms range (device- and interface-dependent, and depending on the exposure, the main measurement interval and on speed relating parameters like GUI interferometer pattern updating). If this option is checked, the voltage is driven stepped to the desired value immediately. Each step roughly takes 200 µs. After reaching the final value the measurement flow continues. The voltage output of the calibration and test options in the “Calibration” sheet is driven immediately in 200 µs steps, always, independent of this setting.

Please note that the so limited and not immediately driven voltage result is not treated for API access. The exported voltage (see chapter 4.1.1 "Exported functions and constants", function `CallbackProc`, `Mode` parameter `cmiAnalogOut`) is the one which would occur without this limitation. This also is reflected by data collections with the LongTerm graph (see chapter 3.4 LongTerm graph).

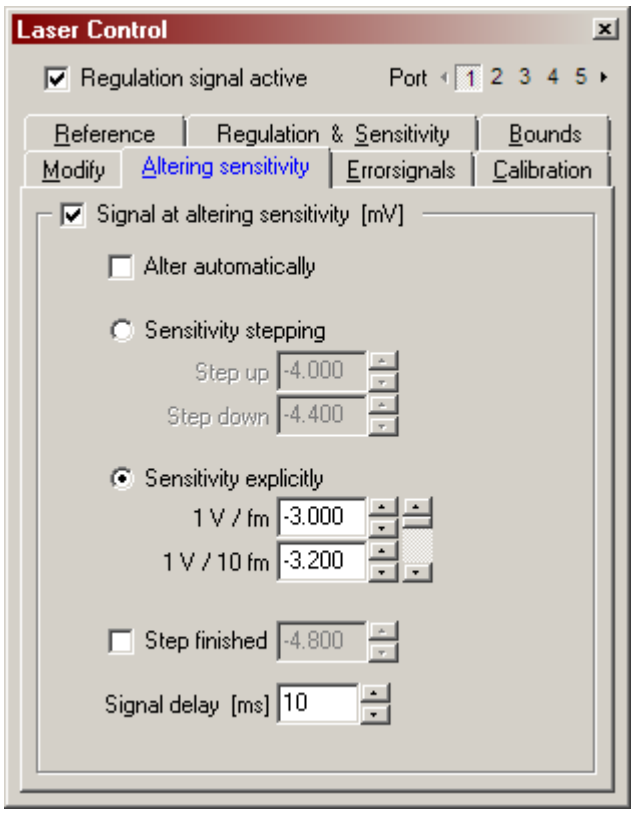
3.5.4 Modify

***Modify signal result*:** This is a possibility to modify the output voltage before it finally is applied. It allows to cut runners for instance or modify the output in any desired way. The variable is "r" here and you can use all the functions that also can be used to form the reference course (see description for sheet “Reference”).

***Modify error*:** Similar to modifying the signal result above, this allows to modify the linear signal error (measurement result – reference value) before it enters in the PID formula.



3.5.5 Altering sensitivity



This is a possibility to leave your electronics know about the sensitivity and its changes.

***Alter automatically*:** Fulfills a sensitivity change by a factor of 10 if the resulting signal exceeds the range set in the sheet bounds. The sensitivity only will be automatically set up, not down.

***Sensitivity stepping*:** Raises an information voltage that the sensitivity has changed one step up or down.

***Sensitivity explicitly*:** Raises a specific voltage for jumping to a specific sensitivity.

"Stepping" or "Explicitly" work exclusive.

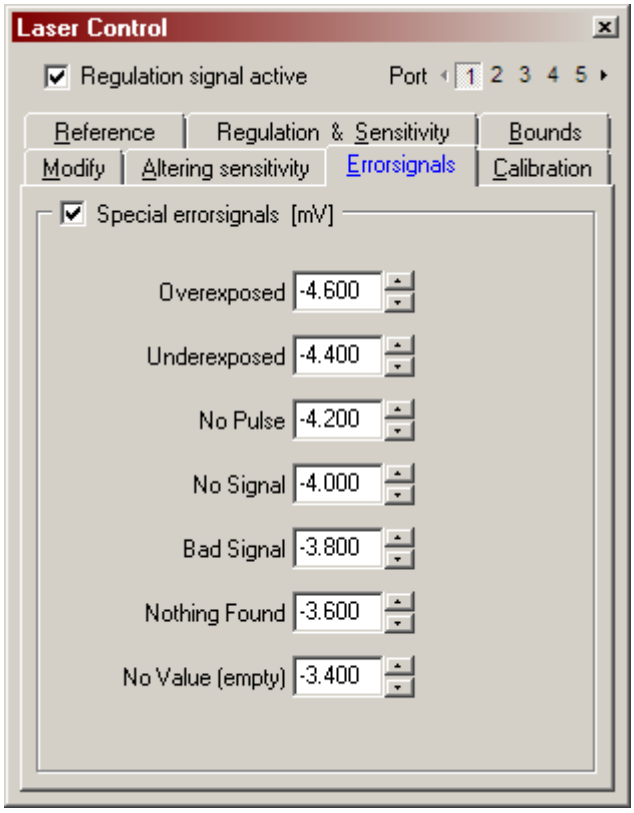
***Step finished*:** If wished, raises an additional finalizing signal after each sensitivity stepping.

***Signal delay*:** Each sensitivity altering signal can be delayed for a time of 0 to 1000 ms with 10 ms steps.

Specific note:
Before applying the last sessions' values, on start the application has a predefined sensitivity of "0", what is 1V / nm. When the stored user-defined sensitivity is set, the device exports the signal like described here, in order to a laser electronics that is plugged on and running directly is informed about the new situation. This has a side effect: If "Use deviation signal" is switched on and you use a large signal delay, no measured wavelength is exported during this delay. This is valid for the moment the deviation signal mechanism first is switched on, what can be automatically on start of the program or of a measurement (see menu "Settings | Extra Settings ...", sheet "Start behaviour", option "Deviation signal") or manually at any time.
You wouldn't notice the delay if "Signal delay" was set only up to 100 ms, but an extremum example of what could be the largest delay possible is:
If the delay is set to 1000 ms and "Sensitivity stepping" and "Step finished" are switched on, "finish" would be raised after each single step and both followed by a 1 second timeout. If the sensitivity from the last session to reapply is "-6" (1V / fm), this would cause six single steps, each of 2 seconds (including the "finish" signal). This would make 12 seconds!

3.5.6 Error signals

Here specific signals can be set to appear if the measurement has lead to an errorvalue.



3.5.7 Calibration (of analoguous voltage output)

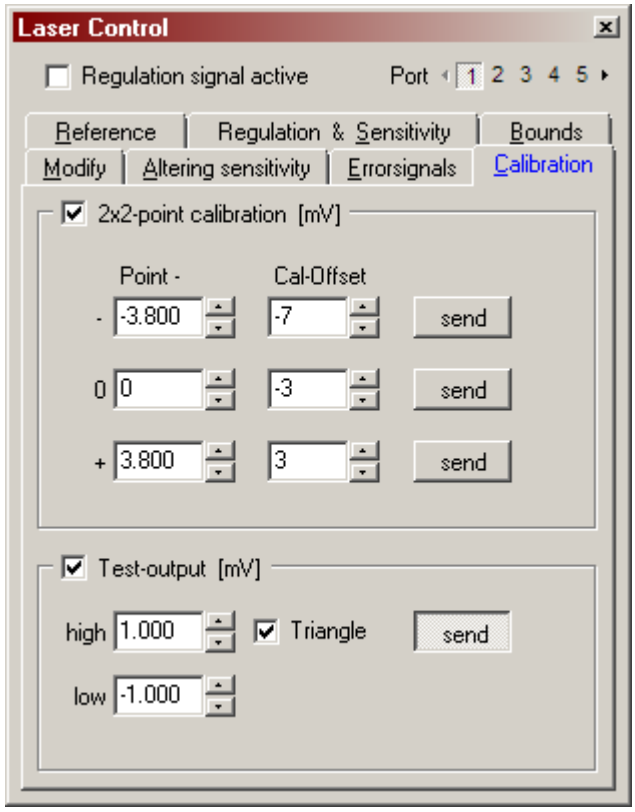
***2x2-point calibration*:** Serves with the possibility to adjust the output voltage.

Set the points "-", "0" and "+" to specific values you want to measure. And then adjust the Cal-Offset values while measuring the output, one after another. The procedure is finished when the measured signals correspond with the values of "Point -", "...0" and "...+".
Use the "send" buttons to send the belonging signal to measure. You need to stop any running measurements before.

Please choose the distances between "Point -" and "...0" and between "...0" and "...+" as far as possible, as far as your equipment allows to measure. This guarantees the best accurracy. In contrast to the settable voltages of the other sheets within this dialogbox, the absolute values of "Point -" and "...+" are allowed for a range of 200 to -200 mV only, Point - negative and Point + positive. Point 0 shall reflect zero voltage (-1 to +1 mV).

Afterwards please leave the checkbox checked, if you want to use the settings.

***Test-output*:** Allows to generate specific voltage values for testing purpose, e.g. for detecting a mode hop free working range. Against to the calibration option above this also works during measurement (to be able to observe the laser behaviour and the measurement results) but here the regulation signal must be switched off before. As long as the regulation signal (the main switch on the upper left hand) is active, this option is disabled. If "Triangle" is selected, the voltage between "low" and “high” is driven automatically in triangle course. If “Triangle” is off, the voltage needs to be changed manually to see an effect. The test output is transferred when the "send” button is pressed.



3.6 PID simulator

The PID simulator PIDSim2 is a helper tool to understand the PID regulation and to find the best regulation parameters for your tunable laser and the desired wavelength course you want to drive it. It is intended to help understanding how the P, I, D and t_a parameters act on an ideal laser (without any noise and special answer bahaviour) regulation and how they influence a regulation. The simulator can be found in the subdirectory …\Projects\PID of the Wavelength Meter installation directory.

Each parameter change is processed immediately and so is the (virtual) regulation answer. It is possible to drive the wavelength to any desired course over the time by a freely adjustable mathematical formula like with the real regulation in the main measurement program. All important regulation parameters are available to be set as well as some Wavelength Meter and laser characterizing parameters.

Beneath observation of manual changes to any parameters it also is possible to let the program automatically calculate the best PID parameters. Additionally it is possible to "connect" the simulator to the Wavelength Meter application to synchronize all changes to the regulation parameters immediately. And some parameters (the measurement cycle, the laser wavelength at an output voltage of 0 V and the amplification factor of the laser system) can be estimated live by the Wavelength Meter.

All values can be stored to named sets which later can be loaded on demand. If the simulator is connected to a Wavelength Meter application this also can be used as a regulation database including different courses and parameters.

3.6.1 Control and display elements

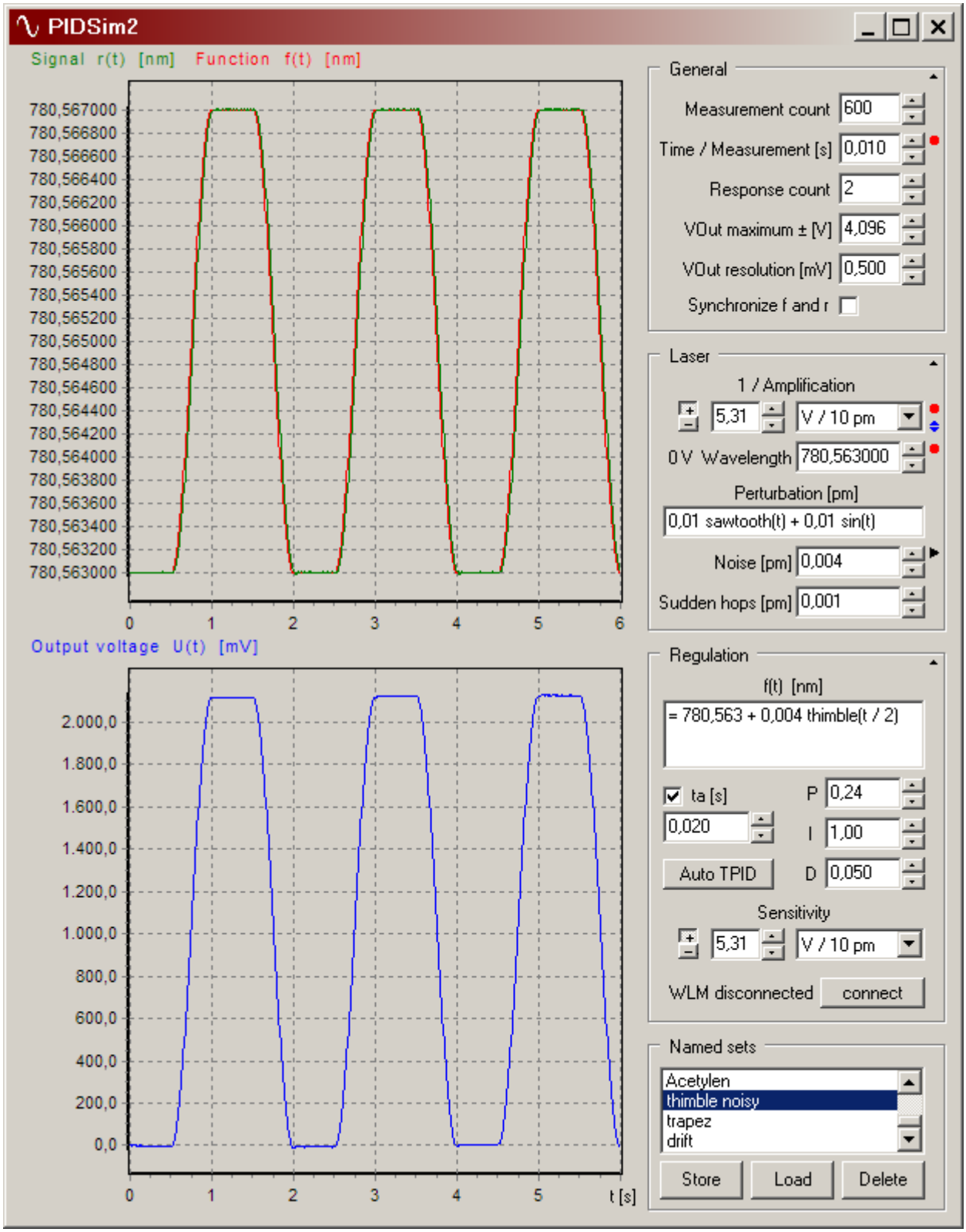
3.6.1.1 The graphs

The upper graph shows the desired regulation function course *f*(*t*) in red colour together with the virtual regulation response signal *r*(*t*) (green) as an answer to the virtual regulation output voltage *U*(*t*) which is displayed blue in the bottom graph. Like with the main application, the graphs can be moved and zoomed with pressed left and right mouse button. With this the time scale is synchronized between the graphs.

3.6.1.2 General settings

Measurement count: is used for displaying the function but it also defines the range used for the automatic PID parameters estimation. Use whatever you want and what is needed to properly display your formula (100 to 10000 possible) showing the characteristics of your entered formula. With periodic functions at least a few periods should be visible this way. It is possible to set any count between 100 and 10000, the bigger this value, the more accurate (but also the slower) the automatic parameters estimation "Auto TPID" will work.

Time / Measurement: is the measurement cycle time in [s]. For correct PID parameter estimation this value should meet the reality. To determine it ***live*** you can use the red button on the right hand side (this only is available when a Wavelength Meter application is running). A regulated measurement will be started to determine the cycle. Wait until the displayed time has stabilized and ***apply*** the value to the PID simulator with the green hook. To ***cancel*** the live evaluation you can use the black *x* button.



This image shows the PID Simulator PIDSim2 with the best regulation- and live evaluated device parameters for a real regulation to a thimble function (an isotrapezoid with sinoidal edges).

3.6.1.3 Hidden General settings

The following controls usually are hidden on program start, they are reachable via the small expand triangle in the topright corner.

Response count: Is the number of measurement cycles until the response wavelength of a regulation action is known. This is a characterizing parameter of the Wavelength Meter and usually it is 2 in non switcher mode, only some IR CCD arrays have a slower readout behaviour and cause a value of 3. In switcher mode it usually can be set to 1.

VOut maximum: This is the maximum voltage range (in [mV]) the WLM can put out. With this Wavelength Meter it is ± 0 mV.

VOut resolution: Is the DAC output resolution in [mV]. With this device it is 0 mV.

Synchronize f and r: Only removes the response delay in the displayed graph for better visual comparison of the regulation function *f* and the response signal *r*.

3.6.1.4 Laser settings

1 / Amplification: This reflects the real amplification of the laser system. It is displayed inverse here to better be comparable to the regulation output sensitivity. You can calculate the laser amplification by recording the wavelength changes on output voltage changes made using the Test-output in the calibration sheet of the regulation window of the Wavelength Meter main application. Or you can use the simulator to evaluate this value: When a Wavelength Meter application is active, there is a little red "***live***" button shown on the right hand side, you can use this button to evaluate the amplification automatically. This process can take up to minutes, depending on the speed settings of the Wavelength Meter application (exposure, display mode, …) and on the load of other processes. The Wavelength Meter will be set to measurement mode without regulation, please wait for a minimum set of data to be collected (counts up to 100%) until an amplification is displayed. When the displayed value is stable, use the small green hook button to ***apply*** the value to the program. Or ***cancel*** the procedure using the small x button.

Note: *The automatic estimation of the laser system amplification is not aware of mode hops and does not try to find the best working range. It simply assumes that the given output voltage (by default ± 1 V) at least is possible without mode hops. The bounds of this output voltage but also can be changed (lower and upper border independently) in case the used laser requires other range boundaries. To change the boundaries please use the small blue double triangle button on the right hand side. There you also can adjust the amplification check speed by modifying the count of readings per cycle.*

0 V Wavelength: This is the laser wavelength while the laser amplifier is driven with 0 V output. For real regulated measurements you again can find this value manually by using the Test-output in the Wavelength Meter main application (s. 1 / Amplification) or you can leave the simulator finding this value automatically. For automatic evaluation please use the little red "***live***" button on the right hand side. Use the green "***apply***" hook to adopt the received average value when it appears stabilized or ***cancel*** the procedure with the *x* button.

3.6.1.5 Hidden Laser settings

The following controls usually are hidden on program start, they are reachable via the small expand triangle in the topright corner.

Perturbation: Can be used to add an arbitrary formula as external perturbation (in [pm]). Can also be used for simulation of drifts for instance and other behaviour.

Noise: Can be used to show the behaviour with laser and measurement noise with am amplitude of the given value (in [pm]). When a noise different from zero is given, a small black triangle "***run continuously***" button appears right hand side which can be used to run the noise continuously.

Sudden hops: Adds four single runaways of the given value (in [pm]) in different directions to the laser answer.

3.6.1.6 Regulation settings

f(t): The regulation course in [nm]. This is the course displayed in red colour in the graphs on the left hand side. Use whatever you desire within a mode hop free wavelength range. The same formula elements as within the main application are available.

t_a: The regulation time constant in [s]. Switch it on to use the given value for the ***I*** and ***D*** consideration of the regulation. If switched off, 1 s is used.

P, I and D: The regulation parameters in the true sense. You can modify them to see how an ideal system (without any noise and special answer bahaviour, but with all other given parameters in this program, especially the response delay) would answer.

Auto TPID: Use this button to leave the simulator estimate the best *P*, *I* and *D* parameters for the given constellation. First, the use of t_a will be switched on and set to *Response count * Time / Measurement*, then the simulator calculates. Depending on the given *Measurement count* this process can take some seconds, while as long as not finished the button is disabled and this process is not interruptable. You can use the found parameters in the laser control settings window and maybe adjust them a bit to optimize your feedback loop if needed. If the controller does not work properly with the found parameters, you might need to repeat the procedure with slightly different initial values because the conditions might have changed due to an unstable laser or amplifier. The better you know your conditions and the laser properties, the better the estimated values are.

Sensitivity: The voltage output sensitivity *S* simply is the final multiplication factor of the calculated wavelength which the laser system is intended to be driven to. It should compensate the real laser amplification and therefore it should equal to the *inverse amplification 1/A* (S * A = 1). However, when not connected to a WLM, it is possible to adjust *S* and *1/A* different to see how it influences the result. When connected to a WLM, *S* and *1/A* are forced treated synchronized, this requires that the amplification value *A* needs to reflect the real laser amplification.

connect / disconnect: When a Wavelength Meter application is active, it is possible to connect the simulator with it, what means that all changes in one of the applications to the regulation parameters PID formula, t_a, *P*, *I*, *D*, and the *sensitivity* (which in this mode reflects the laser *amplification*) are synchronized between each other and it is not needed to make these changes twice manually. Please note that on connecting the existing PID parameters in the PID simulator are updated with those of the WLM main application immediately and without request.

3.6.1.7 Named sets

It is possible to store all parameters to a named set which later can be loaded again. This possibility also can be taken as a small database for the main program.

Store: Use this button to store all parameters shown in the simulator except for *Synchronize f and r*. You will be asked for a name where it is not possible to use the same name twice. This set is stored to disk and can be loaded again after the next simulator startup.

Load: Press this button to load the set which is selected in the list above. You can also load a set by doubleclicking an item in the list.

Delete: Using this button the selected set in the list will be deleted.

3.6.2 Finding the best regulation parameters using the PID simulator

To find the right parameters and to optimize the PID-controller to have the most accurate and fastest feedback control of your laser we here describe a quick procedure finding the parameters P, I, D and the time constant ta for the I- and D-calculation as well as the laser parameters. Having evaluated the needed physical parameters of the entire measurement system, the PID Simulator then calculates the best regulation parameters according to your setup- and laser-conditions.

For further synchronization of PID simulator and Wavelength Meter please “connect” the applications (see “connect / disconnect” in parapgraph 3.6.1.6 ”Regulation settings“ above).

Set an appropriate exposure time. Depending on the maximum available measurement speed (itself depends on the computer speed, the speed influencing settings of the Wavelength Meter and on additional CPU load of other programs) it is not needed to set the exposure to the smallest possible value, it is more senseful to adjust it to the biggest value where it does not diminish the possible measurement speed. This could provide a more stable measurement result since the calculation is better with a higher interference pattern amplitude and the pattern itself is more stable with a higer exposure. You can switch on the live evaluation of the measurement cycle time (see “Time / Measurement” in parapgraph 3.6.1.2 ”General settings“ above) and slowly step the exposure from 1 to 15 ms. Wait a moment until the displayed time has stabilized for each exposure setting. Later take the biggest exposure where you found the smallest measurement cycle time and also apply the Time / Measurement you found with this exposure setting. Now adjust the laser intensity and the fiber coupling to optimize the amplitude of the recorded interference pattern roughly to 60 to 90 % of the chart detail height.

Find a working range where the laser is operating mode hop free. This working range later must cover the wavelength range you want to work/regulate at. Start a measurement without regulation, open of the "Calibration"-tab in the laser control settings window, activate Test-output in triangle mode and press "Send". Increase the triangle output voltage borders step by step and observe the maximum mode hop free positive range. It later might also be necessary to limit the maximum/minimum voltage of the regulation output in order not to cause the laser to jump during the regulation (mode hop). Therefor set these values in the "Bounds"-tab in the laser control settings window.

Now let the PID simulator estimate the laser amplification automatically (please see “1 / Amplification” in paragraph 3.6.1.4 ”Laser settings“) and do the same for the 0 V Wavelength (see “0 V Wavelength” right there). Use a voltage range slightly smaller than the mode hop free tuning range you previously evaluated (using the small blue double triangles button right hand side).

Now if you have entered all desired values you can press "Auto TPID" to start the calculation and wait until the new parameters take place and the "Auto TPID"-button is enabled again. The simulator has tried to find the best settings corresponding to your initial parameters. You can use these parameters in the laser control settings window and maybe adjust them a bit to optimize your feedback loop.

If the controller does not work properly with the found parameters, you might need to repeat the procedure with slightly different initial values because the conditions might have changed due to a unstable laser or amplifier. The better you know your conditions and the laser properties, the better the estimated values are.

3.6.3 Limitations

The synchronization with the WLM is made for output port 1 only (interesting only for versions with more than one DAC output ports).

The unit of measurement of the PID simulator is of [nm] (wavelength vacuum) only and can not be changed. Therefore the TPID estimation works correct only if the regulation course is of vacuum wavelength, too.

4 External access

4.1 Measurement result access

The measurement results and all program values and settings are accessible for user's programs via calls of wlmData.dll-routines. You can find complete sourcecode examples in the subdirectory 'Projects' of your Wavelength Meter installation path. These examples exist for C/C++, Visual Basic, Delphi and LabView. A complete control sample (DataDemo.exe compiled using Delphi 6) is available in the program path and over the startmenu and also a long-term measurement demo (LongTerm.exe representing the Delphi 6 project "LongTermCB"). A very simple callback demonstrating project can be found in \Projects\ComExampleCB\Delphi, the compiled program can be used to access the Wavelength Meter over a COM-port (RS232) connection at another computer.

Note: *The wlmData.dll uses shared memory for interprocess communication. This requires that the server (the Wavelength Meter application) and the clients (any access and control programs, your own ones for instance) access the same dll file, not just identical copies of this file. If the Wavelength Meter application for instance accesses the dll in the System32 directory, your application but uses its private copy inside its own directory, the connection can't establish and called functions will return one of the error indicators* `ErrWLMMissing` *or* `ResERR_WlmMissing` *(see functions "*`GetFrequencyNum, GetWavelengthNum`*" on page 64 and chapter 4.1.3 "*Error value-constants of Set…-functions*" on page 110). On installation, the wlmData.dll file is installed to the System32 directory. We recommend not to create any other copies of this file on the computer where the Wavelength Meter is installed.*

The offered functions are separated into two groups, Get…-functions and Set…-functions and additionally callback mechanism- and base data-routines. The Get…-functions return the requested values in the return parameter, the over handed parameters of the same datatype are for future use and can be ignored the time being. The Set…-functions return a success-indicating value of type 'long' and take the same datatype for parameter as the corresponding Get…-function, this parameter is used to set and change the Wavelength Meter states. The callback (`CallbackProc`/`Ex`) and wait-event mechanisms (`WaitForWLMEvent`/`Ex`) offer methods for generalized and faster measurement acquisition, and the base data-routines `GetPattern`/`Data`/`Num` and `GetAnalysis`/`Data` for obtaining the interferometers' and gratings' pattern and resolved spectrum analysis (if available) data for further data processing. `WaitForWLMEvent`/`Ex` was included to provide a method of fast data acquisition for programming environments that are not able to work with the `CallbackProc` procedure, and the same task is dealed by `GetPatternData`/`Num` and `GetAnalysisData` to support the `GetPattern`/`Num` /`GetAnalysis` functionality of obtaining the interferometer pattern and analysis data for development environments which are not able to handle pointers.

Look at the next chapters to see a detailed description of all exported functions and constants or jump to chapter 4.1.7 "Importdeclarations" (page 114) to find import declaration notes for the programming environments C/C++, Pascal, Basic and LabView.

4.1.1 Exported functions overview

In chapter 4.1.2 the available function are described in detail. For a better overview they here are listed in the order of appearance with a short hint about what they are doing:

Function and chapter	*page*
Chapter** 4.1.2.1 **"General access and WLM/LSA functions"	***53***
Instantiate checks whether the Wavelength Meter server application is running, changes the return mode of the measurement values, installs or removes an extended exporting mechanism or changes the appearance of the server application window or starts or terminates the server application.	*53*
ControlWLM, ControlWLMEx start, hide or terminate the WLM server application.	*55*
SetMeasurementDelayMethod sets the method and the delay how the WLM server application in measurement mode hands off processor time to fulfill the sleep policies' needs or personal desires.	*57*
SetWLMPriority sets the process priority class of the WLM server application in measurement mode.	*58*
CallbackProc, CallbackProcEx are application-defined callback procedures. They receive any measurement results and WLM state changing information.	*59*
WaitForWLMEvent, WaitForWLMEventEx are called to receive any measurement results and WLM state changing information. These functions return if a new measurement result is available or any of the Wavelength Meters' states has changed.	*61*
GetWLMVersion returns the Wavelength Meter or Laser Spectrum Analyser version.	*62*
GetWLMIndex returns the Wavelength Meters' or Laser Spectrum Analysers' handling index number.	*62*
GetWLMCount returns the count of started Wavelength Meter and Laser Spectrum Analyser server applications.	*63*
PresetWLMIndex sets the active dll-internal server handling index.	*63*
GetChannelsCount returns the count of measurement channels the specific Wavelength Meter or Laser Spectrum Analyser possesses.	*63*
Chapter** 4.1.2.2 **"Measurement result access functions"	***64***
GetFrequency, GetFrequency2, GetWavelength, GetWavelength2 return the main results of the measurement.	*64*
GetFrequencyNum, GetWavelengthNum return the main results of the measurement of a specified signal.	*64*
GetCalWavelength returns the measurement result of the calibration laser before and after calibration.	*65*
GetLinewidth returns the calculated linewidth (FWHM) in various units of measurement. This function only works with "R"- or "L"-versions that are able to calculate the linewidth.	*65*
GetDistance returns the resolved distance between the two main lines. This functionality is available for multimode resolvers only.	*66*
GetTemperature returns the temperature inside the optical unit.	*66*
GetPressure returns the pressure inside the optical unit or of an externally preset pressure mode.	*67*
SetPressure activates a specific pressure mode and sets the pressure of an external pressure mode.	*67*
GetDeviationSignal returns the analog voltage output of the DAC channel 1 in Wavelength Meter versions with Deviation output or PID regulation function.	*68*

Function and chapter	***page***
SetDeviationSignal raises an analog voltage output at DAC channel 1 in Wavelength Meter versions with Deviation output or PID regulation function.	*69*
GetDeviationSignalNum returns the analog output voltage of a specified DAC channel in Wavelength Meter versions with Deviation output or PID regulation function.	*69*
SetDeviationSignalNum returns the analog output voltage of a specified DAC channel in Wavelength Meter versions with Deviation output or PID regulation function.	*69*
GetAnalogIn returns the external analog voltage input to the ADC of the Wavelength Meter.	*70*
GetPowerNum returns the power of the current measurement shot.	*70*
GetAmplitudeNum returns the extremum points of the interferometer pattern, the absolute minimum and maximum as well as the fringes average amplitudes.	*71*
GetMinPeak, GetMaxPeak, GetMinPeak2, GetMaxPeak2 return the extremum points of the interferometer pattern of channel number 1. These functions principally are obsolete and included for backward compatibility only.	*71*
GetAvgPeak, GetAvgPeak2 return the average amplitude of the interferometer pattern. These functions principally are obsolete and included for backward compatibility only.	*72*
SetAvgPeak dis-/ or enables the calculation and export of the interference pattern amplitude average.	*72*
GetExternalInput returns data that previously has been transferred to the Wavelength Meter, either directly or from recorded files.	*73*
SetExternalInput transfers up to 64 external values to the Wavelength Meter server that can later be fetched synchronized with the measurements, on replaying recorded files for instance.	*73*
Chapter** 4.1.2.3 **"Operation related functions"	***74***
GetOperationState returns the actual state of measurement.	*74*
Operation sets the program measurement action mode and enables loading and recording files.	*74*
SetOperationFile tells the Wavelength Meter server application about the file name used with the next called `Operation` function.	*75*
Calibration calibrates the Wavelength Meter or Laser Spectrum Analyser based on the light of a reference source.	*75*
TriggerMeasurement interrupts, continues or triggers the measurement loop.	*76*
SetBackground sets the Wavelength Meters background consideration state.	*77*
Chapter** 4.1.2.4 **"State and parameter functions"	***78***
GetResultMode returns the result mode which actually is shown on the program surface.	*78*
SetResultMode sets the unit of the displayed result on the program surface.	*78*
GetRange returns the wavelength range state of the program.	*78*
SetRange sets the wavelength range state of the program.	*79*
GetPulseMode returns the programs pulse mode setting if available.	*79*
SetPulseMode sets the pulse or cw measurement mode.	*80*
GetWideMode returns the Wavelength Meters' measurement precision mode indicator.	*80*
SetWideMode sets the measurement precision.	*80*
GetDisplayMode returns the programs display mode setting if available.	*81*
SetDisplayMode sets the interference pattern display modes.	*81*
GetFastMode returns the Wavelength Meters fast mode state.	*81*
SetFastMode sets the fast mode state.	*82*
GetBackground returns the Wavelength Meters background consideration state.	*82*
GetAnalysisMode returns the Wavelength Meters analysis mode state.	*82*

Function and chapter	*page*
SetAnalysisMode sets the Wavelength Meters analysis mode state.	*83*
GetLinewidthMode returns the Wavelength Meters linewidth estimation mode state.	*83*
SetLinewidthMode sets the Wavelength Meters linewidth estimation mode state.	*83*
GetDistanceMode returns the Wavelength Meters lines distance mode state.	*83*
SetDistanceMode sets the Wavelength Meters lines distance mode state.	*84*
GetIntervalMode returns the Wavelength Meters interval mode state.	*84*
SetIntervalMode sets the interval mode state.	*84*
GetInterval returns the Wavelength Meters measurement interval.	*85*
SetInterval sets the Wavelength Meter measurement interval.	*85*
GetAutoCalMode returns the Wavelength Meters autocalibration mode state.	*85*
SetAutoCalMode sets the autocalibration mode state.	*86*
GetAutoCalSetting receives the calibration rate parameters of the autocalibration option.	*86*
SetAutoCalSetting sets the calibration rate parameters of the autocalibration function.	*87*
GetActiveChannel returns the currently active measurement channel.	*87*
SetActiveChannel sets the currently active measurement channel in non switch mode.	*88*
GetExposureNum returns the actual valid exposure values of a specific signal.	*89*
GetExposure, GetExposure2 return the actual valid exposure values of signal 1.	*89*
SetExposureNum sets the interferometers exposure values of a specific signal.	*90*
SetExposure, SetExposure2 set the interferometers exposure values of signal 1.	*90*
GetExposureMode returns the state of exposure control of the first or only signal channel.	*91*
GetExposureModeNum returns the state of exposure control.	*91*
SetExposureMode sets the mode of exposure control.	*92*
SetExposureModeNum sets the mode of exposure control of a specific signal channel.	*92*
GetExposureRange returns the maximum and minimum of the possible exposure values of the active Wavelength Meter or Laser Spectrum Analyser version.	*92*
GetLinkState returns the link state to the COM-Port.	*93*
SetLinkState sets the link state to the COM-Port.	*93*
LinkSettingsDlg displays a dialog box for setting up the COM-Port parameters for the wavelength exporting (server) and importing (client) capability.	*93*
GetReduced returns the reduction state of the program surface.	*94*
SetReduced sets the surface reduction state.	*94*
GetScale is obsolete. It still is implemented for compatibility means only.	*94*
SetScale is obsolete. It still is implemented for compatibility means only.	*94*
Chapter** 4.1.2.5 **"Switch functions (for the optional multichannel fiber switch)"	***95***
GetSwitcherMode returns the general switch mode of the optional multichannel switch.	*95*
SetSwitcherMode generally switches the switch mode on or off.	*95*
GetSwitcherChannel returns the currently active channel of the optional multichannel switch.	*95*
SetSwitcherChannel sets the currently active channel of the optional multichannel switch in non-switch mode.	*96*
GetSwitcherSignalStates returns the usage and display state of a specified switch channel.	*96*
SetSwitcherSignal, SetSwitcherSignalStates set the usage and display state of a specified switch channel.	*97*
Chapter** 4.1.2.6 **"Deviation control and PID regulation functions"	***97***
GetDeviationMode returns the activity state of the Deviation output (Laser Control) or PID regulation functions.	*97*

Function and chapter	*page*
SetDeviationMode sets the activity state of the Deviation output (Laser Control) or PID regulation functions.	*98*
GetDeviationReference returns the reference value for the Deviation output function.	*98*
SetDeviationReference sets the constant reference value for the Deviation output or PID regulation function.	*98*
GetDeviationSensitivity returns the dimension of the output sensitivity for the Deviation output function. This function is obsolete but still implemented for backward compatibility.	*99*
SetDeviationSensitivity sets the dimension of the output sensitivity for the Deviation output function. This function is obsolete but still implemented for backward compatibility.	*99*
GetPIDCourse, GetPIDCourseNum receive the reference value or PID regulation course of the PID regulation function.	*99*
SetPIDCourse, SetPIDCourseNum set the reference value or PID regulation course of the PID regulation function.	*100*
GetPIDSetting receives the PID and sensitivity parameters of the PID regulation function.	*101*
SetPIDSetting sets the PID and sensitivity parameters of the PID regulation function.	*102*
ClearPIDHistory clears the PID integral history of a given regulation port.	*103*
Chapter** 4.1.2.7 **"Pattern data functions"	***104***
GetPatternItemCount, GetAnalysisItemCount return the count of exported items per array accessible with `GetPattern`/`Num` and `GetPatternData`/`Num`, resp. `GetAnalysis` and `GetAnalysisData`.	*104*
GetPatternItemSize, GetAnalysisItemSize return the size of exported arrays' items accessible with `GetPattern`/`Num` and `GetPatternData`/`Num`, resp. `GetAnalysis` and `GetAnalysisData`.	*105*
GetPattern, GetPatternNum, GetAnalysis return the memory location of an exported array.	*106*
SetPattern, SetAnalysis control whether measured pattern or spectral analysis data arrays will be exported.	*107*
GetPatternData, GetPatternDataNum, GetAnalysisData copy the interferometer pattern or spectral analysis data into an array or a reserved memory location.	*108*
Chapter** 4.1.2.8 **"Other functions"	***109***
ConvertUnit converts a given wavelength (or other) value into a representation of an other unit.	*109*
ConvertDeltaUnit converts a given delta value relative to a base value into a representation of an other unit.	*110*

4.1.2 Exported functions

4.1.2.1 General access and WLM/LSA functions

Instantiate

The `Instantiate` function checks whether the Wavelength Meter server application is running, changes the return mode of the measurement values, installs or removes an extended exporting mechanism or changes the appearance of the server application window or starts or terminates the server application.

`long Instantiate(long RFC, long Mode, long P1, long P2)`

Parameters

`RFC`

ReasonForCall-parameter. Following values are possible:

`cInstResetCalc`, `cInstReturnMode`:	Is used to alter the return mode of `GetFrequency1`/`2`/`Num`, `GetWavelength1`/`2`/`Num`, `GetLinewidth`, `GetDistance` and `GetAnalogIn`, using the `Mode`-parameter.
`cInstCheckForWLM`:	Simply used to check for the presence of a running instance of the WLM server application. Any other parameters may be 0 here.
`cInstNotification`:	Installs or removes the callback or the wait for event exporting mechanism.
`cInstControlWLM`, `cInstControlPriority`, `cInstControlDelay`:	Supported for backward compatibility. Please have a look at the "Obsolete call possibilities" section below in the description of this function.

`Mode`

Used with `cInstResetCalc` (resp. `cInstReturnMode`) as `RFC`-parameter to control the return mode of `GetFrequency1`/`2`/`Num`, `GetWavelength1`/`2`/`Num`, `GetLinewidth`, `GetDistance` and `GetAnalogIn`. `Mode = 0` here means, that on each call of these functions the latest calculated value is returned. With `Mode = 1`, the return value of these functions is set to zero after each first request to any recalculated value. The standard behaviour is of `Mode = 0`.

If `RFC` is `cInstNotification`, this parameter controls the extended exporting mechanisms callback exporting and wait for event-exporting. Following value settings are available:

`cNotifyInstallCallback`, `cNotifyInstallCallbackEx`:	Installs the callback mechanism, which calls a userdefined callback procedure on each calculation or if any other WLM state value has changed. The parameter `P1` needs to obtain the starting address of your callback procedure.
`cNotifyRemoveCallback`:	Removes a perhaps installed callback usage. The associated thread will be terminated.
`cNotifyInstallWaitEvent`, `cNotifyInstallWaitEventEx`:	Installs an alternative to the callback mechanism for programming environments that are not capable of publishing pointers to functions. To use this technique,

you must call the function `WaitForWLMEvent/Ex` in a loop. For further information see below and at the declaration of `WaitForWLMEvent/Ex` inside this chapter.

`cNotifyRemoveWaitEvent`: Removes a possibly installed wait-event usage.

`P1`

This parameter has a meaning only if the callback or wait-event mechanism shall be installed (`RFC = cNotification`; `Mode = cNotifyInstallCallback/Ex` or `cNotifyInstallWaitEvent/Ex`).

In case of callback installation, `P1` takes the starting address of your self-defined callback procedure (for further information, please look at the declaration of the `CallbackProc` procedure prototype below inside this chapter). In any other cases you can set `P1` to 0.

If installing the wait-event mechanism, `P1` has to be used to set the timeout interval of the `WaitForWLMEvent` function and is interpreted in milliseconds. If `P1` is negative, the timeout is set to infinite and `WaitForWLMEvent` never returns if not one of the events occurs that are explained in the description of the `WaitForWLMEvent` function (resp. `CallbackProc` procedure). If `P1` is positive, `WaitForWLMEvent` returns if it has received WLM measurement or state changing information or if the timeout interval elapses. If `P1` is 0, the function checks for an event and returns immediately.

`P2`

This parameter is only used for installing the callback mechanism (see `P1` above) and serves with the opportunity to set the callback-threads' priority. Set this parameter to 0 if you want to leave the thread to be of standard priority. If not for callback means, you can set `P2` to 0.

Return Values

If the function succeeds and at least one Wavelength Meter or Laser Spectrum Analyser is active or terminated due to this instantiating operation the function returns a value greater than 0, else 0.

Remarks

In older versions of the Wavelength Meter (only), the dll had to be instantiated before any further use, this is not necessary any more.
In alternative to poll for each value with one of the `Get…`-functions, it is possible to install a mechanism that leaves your code more clear and quite faster, thus avoiding the usage of timers (that limit the possible repeat rate of your polling calls and risk to lose some measurement results) or infinite loops (that waste too much processor time by running incessant and superfluous counts of calls to the `Get…`-functions). On using this callback- or the wait-event technique, you should consider the additional information in the declaration remarks of the `CallbackProc` procedure prototype and the `WaitForWLMEvent` function.

Obsolete call possibilities

Earlier versions of this function had a few additional call possibilities which all are still supported for backward compatibility:

To start, hide or exit the server, now the `ControlWLM/Ex` functions should be used. The following lines but still are equivalent:

```
Instantiate(cInstControlWLM, Action, App, Ver);
ControlWLM(Action, App, Ver);
ControlWLMEx(Action, App, Ver, 0, 0);
```

To adjust the method the WLM main measurement loop uses for handing off processor time to other applications, now the `SetMeasurementDelayMethod` function should be used. The following two

lines but still are equivalent:

```
Instantiate(cInstControlDelay, iDelayMode, iDelay, 0);
SetMeasurementDelayMethod(iDelayMode, iDelay, 0);
```

To set the WLM server process priority class for the measurement mode, now the `SetWLMPriority` function should be used. The following two lines but still are equivalent:

```
Instantiate(cInstControlPriority, 0, iPriority, 0);
SetWLMPriority(iPriority, 0, 0);
```

ControlWLM, ControlWLMEx

The `ControlWLM/Ex` functions start, hide or terminate the WLM server application.

```
long ControlWLM(long Action, long App, long Ver)
long ControlWLMEx(long Action, long App, long Ver, long Delay, long Res)
```

Parameters

`Action`

This parameter determines whether the server is to be shown, hidden, started or terminated:

`cCtrlWLMShow`:	Displays the server window in its most recent state if it was hidden using `cCtrlWLMHide` before. Starts the WLM server application if it is not yet running and makes it the foreground window. Alternatively it also is possible to start any other certain executable file.
`cCtrlWLMHide`:	Hides the server window and its associated taskbar icon. Starts the WLM or LSA server application if it is not yet running and hides it. If this function is called for another than a WLM or LSA server application, the `Hide` aspect is ignored and the function behaves as called with `cCtrlWLMShow`.
`cCtrlWLMExit`:	Terminates the WLM or LSA server application. Is ignored for other applications.

`cControlWLMShow` and `cControlWLMHide` can be combined with one of the following values in order to suppress error and information messages:

`cCtrlWLMStart-Silent`:	On start no error and information messages will be displayed.
`cCtrlWLMSilent`:	On start but also while running no error and information messages will be displayed.

And with the `ControlWLMEx` function it additionally can be combined with the following value in order to wait until the operation is finished:

`cCtrlWLMWait`:	Causes the call not to return until the operation is finished or the specified timeout (`Delay` parameter) has elapsed.

`App`

Alternatively to start the WLM server application detected automatically it also is possible to start the server software (or any other) with filling `App` with a pointer to a zero terminated file name string of a certain executable file. This only works if `Action` is `cCtrlWLMShow` or

`cCtrlWLMHide`.

`Ver`

Can be used to spot to a distinct installed WLM or LSA server when more than one is installed. `Ver` represents the version number (serial number or version information detail 1, see `GetWLMVersion`) of the WLM or LSA. This parameter is only considered if not `App` is specified. If `Ver` is set 0, it spots to the last accessed (see `GetWLMIndex` and `PresetWLMIndex`) or only active server. If no server is active, and `Ver` is 0, the last installed WLM or LSA is started (when `Action` does not contain `cCtrlWLMExit`).

`Delay`

Can be used to specify a timeout to return before the operation is completed. May help if a server is deadlocked or waits for an input to continue. Has effect only if `Action` contains `cCtrlWLMWait`. A negative value sets an infinite delay.

`Res`

Can be set to 1 to get extended return information on starting a Wavelength Meter or Spectrum Analyser in combination with the `cCtrlWLMShow` or `cCtrlWLMHide` and the `cCtrlWLMWait` `Action` flags. Otherwise set it to 0.

Return Values

If the function succeeds and at least one Wavelength Meter or Laser Spetrum Analyser is active or terminated due to this instantiating operation the function returns a value greater than 0, if another but the WLM server application was started the return value is 1 else 0.
If `ControlWLMEx` was called with `cCtrlWLMShow` or `cCtrlWLMHide` and with `cCtrlWLMWait` and `Res` set to 1 to start a Angstrom Wavelength Meter or Spectrum Analyser, the return value can consist of the following flags:

`flServerStarted`:	Doesn't represent an error, it simply indicates that the software server is started.
`flErrDeviceNotFound`:	No Angstrom Wavelength Meter or Spectrum Analyser was found.
`flErrDriverError`:	The driver was not loaded correctly or caused the device to not start properly.
`flErrUSBError`:	A USB error occurred and the device could not be started properly.
`flErrUnknownDeviceError`:	An unknown device error occurred and the device could not be started properly.
`flErrWrongSN`:	The device started has an other serial number than expected.
`flErrUnknownSN`:	The device started has an unknown serial number.
`flErrTemperatureError`:	An error occurred on initialization of the temperature sensor. The device was started, but will not be able to report correct wavelength measurements.
`flErrCancelledManually`:	The device initialization was cancelled manually.
`flErrUnknownError`:	An unknown (hardware or software) error occurred. The device was started but may not work properly.

Remarks

After installation the corresponding WLM server application is published to the system and will be called here if `App` is zero. If this file or the entire directory afterwards is renamed or moved manually, this function will not be able to find the server application file automatically and will not succeed. Do not move or rename the installation path or the executable files after installation.

If this function is called with `cCtrlWLMWait`, the calling thread is not responsive to other tasks until the function returned. If called from the main thread, your application will be unresponsive during the call. A possibility to avoid this behaviour is to use the `CallbackProc` procedure instead and listen to `cmiServerInitialized` and/or `cmiDLLAttach`/`Detach`.

SetMeasurementDelayMethod

The `SetMeasurementDelayMethod` function sets the method and the delay how the WLM server application in measurement mode hands off processor time to fulfill the sleep policies' needs or personal desires.

`long SetMeasurementDelayMethod(long Mode, long Delay)`

Parameters

`Mode`

This parameter is used to control the method how the polling data acquisition loop hands off processor time to let the system (resp. other applications) be fairly actionable beneath the WLM application. The possibilities for '`Action`' are:

`cCtrlMeasDelayRemove`:	Removes any possibly set delay mode and restores the usual, builtin sleeping method with a delay of the minimum system timeslice.
`cCtrlMeasDelayDefault`:	Like with `cCtrlMeasDelayRemove` the usual builtin method is used with a settable delay but.
`cCtrlMeasDelayGenerally`:	Interrupts the loop for the given delay after each poll, successive or not.
`cCtrlMeasDelayOnce`:	Interrupts the loop once for the given delay after a successive measurement event occurred.
`cCtrlMeasDelayDenyUntil`:	This is not a delay truly, the polling thread denies control until the given delay has been reached. Please also have a look at the "Remarks" section below.
`cCtrlMeasDelayIdleOnce`:	Uses the WaitForInputIdle mechanism with a timeout of the given delay once after each successive measurement event.
`cCtrlMeasDelayIdleEach`:	Like `cCtrlMeasDelayIdleOnce`, but after each poll, successive or not.

`Delay`

This parameter determines the corresponding delay in [ms], it can be set from 0 to 9999 ms.

Return Values

This function returns one of the values described in "Error values of `Set`…-functions" below.

Remarks

Except for calls with `cCtrlMeasDelayDenyUntil` the system will treat all entered `delay` values with the systems' minimum time slice granularity. The usual system time slice granularity is of 10 ms, so the real value can be expressed by

((*Delay* - 1) adiv 10 + 1) * 10

where adiv is the asymmetric integer division operator. So only when `Delay` is 0, the real value is smaller than the minimum timeslice.

With `cCtrlMeasDelayDenyUntil` the polling thread denies control until the given delay has been reached. Against all other methods here this allows to interpret the delay in single milliseconds and not just with the minimum system timeslice granularity.

With all methods but both the builtin (`cCtrlMeasDelayRemove` and `cCtrlMeasDelayDefault`) the polling thread generally switches to `REALTIME_PRIORITY_CLASS` directly before handing off its lasting processor time and switches back to the priority class which was active before. This rigid behaviour allows the application to jump to a front position when the system schedules processor time.

For special tasks with needs for synchronization of other control applications with the Wavelength Meter application it also might be useful to trigger the Wavelength Meter measurements by software with calls to `TriggerMeasurement`. This guarantees to have the Wavelength Meter measurement loop blocked completely without wasting any processor time. Please have a look at the `TriggerMeasurement` function description.

SetWLMPriority

The `SetWLMPriority` function sets the process priority class of the WLM server application in measurement mode.

```
long SetWLMPriority(long PPC, long Res1, long Res2)
```

Parameters

`PPC`

The servers' measurement mode process priority class to set. All system base process priority class values can be used.

`Res1, Res2`

Reserved for future use and should be 0.

Return Values

This function returns one of the values described in "Error values of `Set`…-functions" below.

Remarks

The so set priority class is active only when the WLM is in measurement mode, on leaving the measurement mode the priority class automatically is set back to `NORMAL_PRIORITY_CLASS`. Please be very careful with setting high priority class values, the system might be deadlocked if the WLM server application for any reason would not return or crash in measurement mode. Especially better completely avoid using `REALTIME_PRIORITY_CLASS` for this reason.

CallbackProc, CallbackProcEx

The CallbackProc and CallbackProcEx procedures are application-defined callback procedures. They receive any measurement results and WLM state changing information. The third parameter of Instantiate defines a pointer to these callback functions. CallbackProc and CallbackProcEx are placeholders for the application-defined procedure names.

```
void CallbackProc(long Mode, long IntVal, double DblVal)
void CallbackProcEx(long Ver, long Mode, long IntVal, double DblVal,
                    long Res1)
```

Parameters

Ver

The device version number which has called this procedure. This is important only when more than one Wavelength Meter or Laser Spectrum Analyser is running.

Mode

Indicates the meaning of the IntVal and DblVal parameters. Here is a set of some of the possible constants for the Mode parameter:

Value in DblVal

cmiTemperature	cmiWavelength1 to 9	cmiLinewidth
cmiDistance	cmiAnalogIn	cmiAnalogOut
cmiPID_P, …I, …D, …T, …dt	cmiDeviation-SensitivityFactor	cmiExternalInput

Value in IntVal

cmiExposureMode	cmiAnalysisMode	cmiMin11 to 29
cmiExposureValue11 to 19	cmiSwitcherMode	cmiMax11 to 29
cmiExposureValue21 to 29	cmiSwitcherChannel	cmiAvg11 to 29
cmiResultMode	cmiPIDCourse	cmiPatternAnalysis-Written
cmiRange	cmiPIDUseT	
cmiPulseMode	cmiPIDConstdt	cmiVersion0
cmiWideMode	cmiPID_-AutoClearHistory	cmiVersion1
cmiFastMode	cmiDeviationPolarity	cmiDLLAttach
cmiDisplayMode	cmiDeviationUnit	cmiDLLDetach
cmiReduced	cmiDeviation-SensitivityDim	cmiServerInitialized
cmiLink		
cmiOperation		…

The complete list of the possible cmi…-constants can be found in chapter 4.1.4 "Mode constants for Callback-export, the WaitForWLMEvent-function and for saved single measurement and long-term recording files" on page 111 (those marked by an “e” in the final bracket), declared in the header files "wlmData.*" that are enclosed with the demo examples in the "Projects" subdirectory:

IntVal

If Mode is one of the values listed under "Value in DblVal" above, IntVal holds the exact measurement time stamp in and rounded to milliseconds. The timestamp reflects the point of time the Wavelength Meter just has received the interferometer pattern, what is before calculation of the wavelength and exporting.

For values listed under "Value in IntVal" above, IntVal receives the integer value that would be returned on a successive call to the corresponding Get…-function. For Get…-functions that return boolean values, IntVal is the casted integer. In case of cmiDLLDetach, IntVal is 0.

With cmiPatternAnalysisWritten (a notification about the publishing of new interference

pattern raw data and the spectrum of analysis results which can be accessed with the `GetPattern`/`Data`/`Num` or `GetAnalysis`/`Data` functions), `IntVal` is the channel number of the multichannel switch the data belongs to, in versions without switch `IntVal` is 1 always.

`DblVal`
Has a meaning only for `Mode` values listed under "Value in DblVal" above. With these, `DblVal` is the corresponding double value. For any other values for `Mode`, `DblVal` receives 0.

`Res1`
With `cmiSwitcherChannel` (occurs when the optional multichannel switch is being switched, the belonging channel number is held by `IntVal`) `Res1` holds the switching timestamp. With other `Mode` values `Res1` is without meaning.

Return Values
This is a procedure and you cannot return anything.

Remarks
If installed, this procedure is called each time a new measurement result is available or any of the Wavelength Meters' states - indicated by the `cmi`…-constants listed in the description of the `Mode` parameter - has changed.

This callback will be installed and removed with calls to `Instantiate`. Please have a look to `Instantiate` to see how.

Once installed, the `CallbackProc`/`Ex` procedure is part of an additional thread the wlmData.dll has created, so please see to synchronize possible competing memory accesses with your application to avoid memory access violations. Also note that this procedure just runs once a time, so if it runs too long, you might lose some measurement results. The easiest way it can be helped is to just leave the code inside this procedure collect the received data and return. Another thread then can work with it where it just has to synchronize accesses to the collected data. For this topic please have a look to the example program in the programs subdirectory ".\Projects\LongTermCB\Delphi\".

The difference between `CallbackProc` and `CallbackProcEx` is that the `CallbackProcEx` variant is able to handle more than just one WLM server application at a time and receives the sending WLM version in the `Ver` parameter.

Another way to get the data on the run is to install a wait-for-event mechanism rather than this callback one, useful for development languages that are not able to deal with function pointers. This is done simply by calls to the `WaitForWLMEvent`/`Ex` function in a loop. The `WaitForWLMEvent`/`Ex` function (see below) only returns for the same reasons for which `CallbackProc`/`Ex` is called or if a previously specified timeout interval has elapsed.

Example
This line prepares the dll for usage with your self defined callback procedure `MyCallbackProc`/`Ex`:

```
Instantiate(cInstNotification,
            cNotifyInstallCallback,
            Integer(@MyCallbackProc),
            0);
```

resp.

```
Instantiate(cInstNotification,
            cNotifyInstallCallbackEx,
            Integer(@MyCallbackProcEx),
            0);
```

This line removes the callback announcement:

```
Instantiate(cInstNotification, cNotifyRemoveCallback, 0, 0);
```

This is a possible implementation (modelled on the contents of the Delphi LongTerm-Demo):

```
procedure MyCallbackProcEx(Ver, Mode, IntVal: Integer; DblVal: Double;
                           Res1: Integer);
  begin
    CriticalSection.Acquire; // possibility to synchronize accesses
                             // to your code and objects
    case Mode of
      cmiWavelength1:
        begin
          // Access measurement results here using the DblVal parameter
        end;

      cmiTemperature:
        begin
          // Access the temperature here using the DblVal parameter
        end;

      cmiWideLine, cmiVersion: // or any other cmi...-constants
        begin
          // Do something with additional state information using the
          // IntVal parameter here
        end;

      cmiDLLDetach:
        begin
          // The WLM or LSA exited
        end;

      cmiDLLAttach:
        begin
          // Another WLM or LSA server has announced
        end;

      cmiServerInitialized:
        begin
          // A WLM or LSA server has announced and is completely ini-
          // tialized (IntVal = 1) or it has been quit and is uninitia-
          // lized (IntVal = 0) that way
        end;
    end;
    CriticalSection.Release;
  end; // CallbackProc.
```

WaitForWLMEvent, WaitForWLMEventEx

The `WaitForWLMEvent/Ex` functions are called to receive any measurement results and WLM state changing information. These functions return if a new measurement result is available or any of the Wavelength Meters' states has changed.

```
long WaitForWLMEvent(long &Mode, long &IntVal, double &DblVal)
long WaitForWLMEventEx(long &Ver, long &Mode, long &IntVal, double &DblVal,
                       long &Res1)
```

Parameters

Ver, **Mode**, **IntVal**, **DblVal**, **Res1**

These parameters act identically to that described in the `CallbackProc` procedure above.

Return Values

These functions return 1 if they have received measurement or state changing information and there still are other results available, 2 if they have received measurement or state changing information and there are no additional results available at the moment, -1 if the timeout interval specified with the call to `Instantiate` has elapsed, 0 if the mechanism has been removed or not been installed before and -2 if they have received an unknown event or an error has occurred.

Remarks

If installed, these functions only return if a new measurement result is available, a Wavelength Meters' state has changed (please see the description of the `CallbackProc`/`Ex` procedure) or the specified timeout interval has elapsed.

The functionality of these functions has to be installed with a call to `Instantiate`. Please look at `Instantiate` to see how.

To not affect the flow of your application, it is recommended to use these functions inside a self created secondary thread. Otherwise a large timeout interval might cause your application to be unresponsive to user actions during this interval. Similar to the usage of the `CallbackProc`/`Ex` procedures, the easiest way is to just leave these functions' calling threads collect the received data and loop. The main thread or another one then can work with the data where it just has to synchronize accesses to the collected data.

These functions are useful for development languages that are not able to deal with function pointers. Another way to get the data without wasting processor time is to install a callback procedure instead of using this function. For this purpose please have a look at the `CallbackProc`/`Ex` procedure above.

GetWLMVersion

The `GetWLMVersion` function returns the Wavelength Meter or Laser Spectrum Analyser version.

long GetWLMVersion(long Ver)

Parameters

Ver

Specifies the version information detail, 0 means the Wavelength Meter type (can be 5 to 8) or Laser Spectrum Analyser type (always 5), 1 addresses the version number, 2 the revision number of the software, and 3 a floating software compilation number.

Return Values

If the function succeeds and at least one Wavelength Meter or Laser Spectrum Analyser is active the function returns the requested version information detail of the Wavelength Meter, else `ErrWlmMissing`.

GetWLMIndex

The `GetWLMIndex` function returns the Wavelength Meters' or Laser Spectrum Analysers' handling index number.

long GetWLMIndex(long Ver)

Parameters

Ver
Specifies the WLM/LSA version number (version information detail 1, see `GetWLMVersion`).

Return Values
If the function succeeds and at least one Wavelength Meter or Laser Spectrum Analyser is active the function returns the dll internal handling number of the specified Wavelength Meter/Laser Spectrum Analyser, else `ErrWlmMissing`.

GetWLMCount

The `GetWLMCount` function returns the count of started Wavelength Meter and Laser Spectrum Analyser server applications.

```
long GetWLMCount(long V)
```

Parameters

V
Reserved for future use and should be zero.

Return Values
The function returns the count of different active Wavelength Meter and Laser Spectrum Analyser server applications (while it is not possible to start the same server twice).

PresetWLMIndex

The `PresetWLMIndex` function sets the active dll-internal server handling index.

```
long PresetWLMIndex(long Ver)
```

Parameters

Ver
Specifies the dll-internal handling index or the corresponding Wavelength Meter or Laser Spectrum Analyser version information detail 1 (see `GetWLMVersion`).

Return Values
If the function succeeds and at least one Wavelength Meter or Laser Spectrum Analyser is active the function returns the dll-internal handling index of the specified Device, else `ErrWlmMissing`.

Remarks
If you are using more than one Wavelength Meter and/or Laser Spectrum Analyser at a time, it is required to preset the WLM/LSA handling index to access the correct device with any of the functions that try to obtain data from or set states of a Wavelength Meter or Laser Spectrum Analyser server instance. This setting is consistent between other calls, so this function only needs to be called again when you want to change the WLM or LSA to access to or if another new server has been started inbetween.

The `Ver` parameter can manage both, the handling index as well as the Wavelength Meter/Laser Spectrum Analyser version detail 1. Their ranges can't overlap.

GetChannelsCount

The `GetChannelsCount` function returns the count of measurement channels the specific Wavelength Meter or Laser Spectrum Analyser possesses.

long GetChannelsCount(long C)

Parameters

C
Reserved for future use and should be zero.

Return Values
The function returns the count of channels that can measure, calculate and return wavelength. These can be single or a number of switch channels on the front and rear entry (if any), as well as the second pulse channel in double pulse versions. An also available channel of a possibly builtin neon lamp is not considered with this function since it does not measure a wavelength.

4.1.2.2 Measurement result access functions

GetFrequency, GetFrequency2, GetWavelength, GetWavelength2

The `GetFrequency` and `GetWavelength` functions return the main results of the measurement.

```
double GetFrequency(double F)
double GetFrequency2(double F2)
double GetWavelength(double WL)
double GetWavelength2(double WL2)
```

Parameters

F, **F2**, **WL**, **WL2**
Reserved for future use and should be left zero.

Return Values
The functions return the same values as `GetFrequencyNum` and `GetWavelengthNum` do.

Remarks
For Wavelength Meters with multi channel switch or double pulse option (MLC) these functions are obsolete and should be replaced by `GetFrequencyNum` and `GetWavelengthNum` in your code. The following pairs are equivalent:

```
GetWavelength(0);        and   GetWavelengthNum(1, 0);
GetWavelength2(0);and    GetWavelengthNum(2, 0);
GetFrequency(0);         and   GetFrequencyNum(1, 0);
GetFrequency2(0);        and   GetFrequencyNum(2, 0);
```

GetFrequencyNum, GetWavelengthNum

The `GetFrequencyNum` and `GetWavelengthNum` functions return the main results of the measurement of a specified signal.

```
double GetFrequencyNum(long num, double F)
double GetWavelengthNum(long num, double WL)
```

Parameters

num
Indicates the signal number (1 to 8) in case of a WLM with multi channel switch or with double pulse option (MLC). For WLM's without these options 1 should be overhanded.

F, **WL**
Reserved for future use and should be left zero.

Return Values

If the function succeeds and the Wavelength Meter is active, the function returns the last measured frequency value in THz or wavelength in nm, else one of the following error values:

`ErrNoValue`:	If the dll is instantiated with return mode = 1 and you yet have requested the latest measured value, the functions return `ErrNoValue`.
`ErrNoSignal`:	The Wavelength Meter has not detected any signal.
`ErrBadSignal`:	The Wavelength Meter has not detected a calculatable signal.
`ErrLowSignal`:	The signal is too small to be calculated properly.
`ErrBigSignal`:	The signal is too large to be calculated properly, this can happen if the amplitude of the signal is electronically cut caused by stack overflow.
`ErrNoPulse`:	The detected signal could not be divided into separated pulses.
`ErrWlmMissing`:	The Wavelength Meter is not active.
`ErrNotAvailable`:	This called function is not available for this version of Wavelength Meter.

GetCalWavelength

The `GetCalWavelength` function returns the measurement result of the calibration laser before and after calibration.

`double GetCalWavelength (long ba, double WL)`

Parameters

`ba`

Used with `ba = 0` the result of the calibraton laser before the last calibration is returned. With `ba = 1` the result of the same signal is retuned concerning the calibration effect.

`F`, **`WL`**

Reserved for future use and should be left zero.

Return Values

If the function succeeds and the Wavelength Meter has calibrated at least once before, the function returns the last measured wavelength of the calibration laser before or after calibration in [nm], else one of the error values described in the ‘Return Values’ section of the `GetWavelengthNum` function above.

GetLinewidth

The `GetLinewidth` function returns the calculated linewidth (FWHM) in various units of measurement. This function only works with "R"- or "L"-versions that are able to calculate the linewidth.

`double GetLinewidth(long Index, double LW)`

Parameters

`Index`

Determines the unit of measurement to be returned. The following values are possible:

`cReturnWavelengthVac`: Linewidth in [nm] corresponding to the vacuum wavelength.

`cReturnWavelengthAir`: Linewidth in [nm] corresponding to the wavelength in air.

`cReturnFrequency`: Linewidth in [THz] corresponding to the vacuum frequency.

`cReturnWavenumber`: Linewidth in [cm^{-1}] corresponding to the vacuum wavenumber.

`cReturnPhotonEnergy`: Linewidth in [eV] corresponding to the photon energy.

LW

Reserved for future use.

Return Values

If the function succeeds and the Wavelength Meter is active the function returns the linewidth in a unit of measurement depending on the `Index` parameter and with the accuracy displayed on the server surface, else one of the error values described in the `GetWavelength`/`GetFrequency`-functions.

GetDistance

The `GetDistance` function returns the resolved distance between the two main lines. This functionality is available for multimode resolvers only.

`double GetDistance(double D)`

Parameters

D

Reserved for future use and should be left zero.

Return Values

The function returns the distance between the two main lines if the Wavelength Meter is in analysis mode.

GetTemperature

The `GetTemperature` function returns the temperature inside the optical unit.

`double GetTemperature(double T)`

Parameters

T

Reserved for future use.

Return Values

If the function succeeds and the Wavelength Meter is active the function returns the temperature (in °Celsius) inside the optical unit with a precision of three digits, else one of the following values:

`ErrTempNotMeasured`: Starting value. The device has still not measured the temperature until now.

`ErrTempNotAvailable`: This device does not support measuring the temperature. The optical unit does not consist of a temperature sensor.

`ErrTempWLMMissing`: The Wavelength Meter server instance meanwhile has been terminated.

GetPressure

The `GetPressure` function returns the pressure inside the optical unit or of an externally preset pressure mode.

`double GetPressure(double P)`

Parameters

`P`
Reserved for future use.

Return Values

If the function succeeds and the Wavelength Meter is active the function returns the pressure (in mbar) of the preset pressure mode (either the possibly builtin sensor or one of two external modes), else one of the following values:

`ErrTempNotMeasured`:	Starting value. The device has still not measured the pressure until now.
`ErrTempNotAvailable`:	This device has no builtin pressure sensor and no external pressure mode was set via `SetPressure`.
`ErrTempWLMMissing`:	The Wavelength Meter server instance meanwhile has been terminated.

Remarks

Not every Wavelength Meter consists of a builtin pressure sensor. If but there is a pressure sensor or a pressure value is given (via `SetPressure`) by an external sensor, the software can consider the pressure to align the wavelength reading. Devices without builtin sensor don't possess pressure compensation factors ex works. If needed, you can activate an external pressure mode, route the current pressure to the software (see `SetPressure`), record measurements (ltr files) of stable signals and with varying pressure (100 mbar at least) and let us calculate and implement the required parameters into your software.

SetPressure

The `SetPressure` function activates a specific pressure mode and sets the pressure of an external pressure mode.

`long SetPressure(long Mode, double P)`

Parameters

`Mode`
Can be used to preset a specific pressure mode: Mode = -1 activates the builtin sensor mode (also if this sensor is physically not available), Mode = -2 activates the external sensor mode 1 and Mode = -3 the external sensor mode 2. Always only one pressure mode can be active and the mode can be changed only in non measurement mode. `P` will be ignored on calls changing the active pressure mode.
It also can be used to specify the pressure mode on setting the pressure `P` of an external pressure mode: 2 specifies the external pressure mode 1 and 3 means the external pressure mode 2. If a pressure `P` is to be set, the specific pressure mode needs to be specified. If a pressure is set this way with a pressure mode that is not active, the value is accepted and stored in the software (as well as in recorded files) but not used for wavelength calculation since it doesn't belong to the currently active pressure mode.

P

The pressure in mbar for one of the external pressure modes. `P` only can be within 10 and 1200 mbar and it is ignored if `Mode` is not 2 or 3.

Return Values

This function returns one of the values described in "Error values of `Set`…-functions" below.

Remarks

Using this function one can (e.g. for devices without builtin pressure sensor) activate the internal or an external pressure mode, and, once an external mode activated, route a pressure to the Wavelength Meter server software.

Not every Wavelength Meter consists of a builtin pressure sensor. If but there is a pressure sensor or a pressure value is given (via `SetPressure`) by an external sensor, the software can consider the pressure to align the wavelength reading. Devices without builtin sensor don't possess pressure compensation factors ex works. If needed, you can record measurements (ltr files) of stable signals and with varying pressure (100 mbar at least) and let us calculate and implement the required parameters into your software.

There are three possible modes, one builtin sensor mode and two for external sensors, just only one mode can be active at a time. Ex works the internal sensor mode is preset (also if there's no sensor built in).

If a pressure is set with a pressure mode that is not active, the value is accepted and stored in the software (as well as in recorded files) but not used for wavelength calculation since it doesn't belong to the currently active pressure mode. Additionally, the pressure is accepted and shown in the status bar of the graphical user interface, but will not be used either if the last calibration was performed without pressure value (resp. sensor). So, one first needs to activate a specific pressure mode, hand over the belonging pressure, calibrate (just once, not each time), and from that time the measurement is performed with pressure consideration. Also, vice versa, if the last calibration was performed with pressure sensor, the measurement but later without, the calibration pressure simply is ignored. Once calibrated using a special pressure sensor, all following measurements should be performed using the same sensor.

GetDeviationSignal

The `GetDeviationSignal` function returns the analog voltage output of the DAC channel 1 in Wavelength Meter versions with Deviation output or PID regulation function.

`double GetDeviationSignal(double DS)`

Parameters

DS

Reserved for future use and should be left zero.

Return Values

The function returns the last exported analog voltage of the DAC channel 1 in [mV].

Remarks

For Wavelength Meters with more than one DAC channel this function is obsolete and should be replaced by `GetDeviationSignalNum` in your code.
The following calls but still are equivalent:

```
GetDeviationSignal(0);   and   GetDeviationSignalNum(1, 0);
```

SetDeviationSignal

The `SetDeviationSignal` function raises an analog voltage output at DAC channel 1 in Wavelength Meter versions with Deviation output or PID regulation function.

`long SetDeviationSignal(double DS)`

Parameters

`DS`
The voltage to put out in [mV].

Return Values

For return-information, please look at "Error values of `Set`…-functions".

Remarks

Using this function you can control the analog output in non-regulation mode by yourself. Principally you can create your own regulation mechanism this way.

For Wavelength Meters with more than one DAC channel this function is obsolete and should be replaced by `SetDeviationSignalNum` in your code.
The following calls but still are equivalent:
`SetDeviationSignal(V);` and `SetDeviationSignalNum(1, V);`

GetDeviationSignalNum

The `GetDeviationSignalNum` function returns the analog output voltage of a specified DAC channel in Wavelength Meter versions with Deviation output or PID regulation function.

`double GetDeviationSignalNum(long Num, double DS)`

Parameters

`Num`
The DAC output channel number. Must be set to 1 for devices with only one DAC channel.

`DS`
Reserved for future use and should be left zero.

Return Values

The function returns the last exported analog voltage of the belonging DAC channel in [mV].

SetDeviationSignalNum

The `SetDeviationSignalNum` function returns the analog output voltage of a specified DAC channel in Wavelength Meter versions with Deviation output or PID regulation function.

`double SetDeviationSignalNum(long Num, double DS)`

Parameters

`Num`
The DAC output channel number. Must be set to 1 for devices with only one DAC channel.

`DS`
The voltage in [mV] to put out.

Return Values

For return-information, please look at "Error values of `Set`…-functions".

Remarks

Using this function you can control the analog output in non-regulation mode by yourself. Principally you can create your own regulation mechanism this way using the builtin DAC channels.

GetAnalogIn

The `GetAnalogIn` function returns the external analog voltage input to the ADC of the Wavelength Meter.

```
double GetAnalogIn(double AI)
```

Parameters

`AI`

Reserved for future use and should be left zero.

Return Values

If the dll is instantiated with return mode = 1 and you yet have requested the latest measured value, the function returns -1000000, else the function returns the last detected analog voltage in [mV].

GetPowerNum

The `GetPowerNum` function returns the power of the current measurement shot.

```
double GetPowerNum(long num, double P)
```

Parameters

`Num`

Indicates the signal number (1 to 8) in case of a WLM with multi channel switch or with double pulse option (MLC). For WLM's without these options 1 should be overhanded.

`P`

Reserved for future use and should be left zero.

Return Values

If the function succeeds and the Wavelength Meter is active, the function returns the power of the last measured cw or quasi cw signal in µW or the energy in µJ in case of pulse mode measurements, else one of the following error values:

`ErrNoValue`:	If the dll is instantiated with return mode = 1 and you yet have requested the latest measured value, the functions return `ErrNoValue`.
`ErrNoSignal`:	The Wavelength Meter has not detected any signal.
`ErrBadSignal`:	The Wavelength Meter has not detected a calculatable signal.
`ErrLowSignal`:	The signal is too small to be calculated properly.
`ErrBigSignal`:	The signal is too large to be calculated properly, this can happen if the amplitude of the signal is saturated.
`ErrNoPulse`:	No pulse was detected or the detected signal could not be divided into separated pulses.
`ErrWlmMissing`:	The Wavelength Meter server is not active or no connection is possible.

ErrNotAvailable: This called function or the specified channel or array index is not available for this Wavelength Meter version.

Remarks

The device basically cannot measure the absolute power or energy. So, the power needs to be calibrated beforehand to convert the relative measurements into absolute values. The power calibration can be performed via menu “Operation | Calibration | Power calibration …”.

The measured power represents the power entering the device behind the output of the coupling fiber. Any losses caused by the fiber or by the collimation efficiency into the fiber can not be considered.

GetAmplitudeNum

The GetAmplitudeNum function returns the extremum points of the interferometer pattern, the absolute minimum and maximum as well as the fringes average amplitudes.

```
long GetAmplitudeNum(long Num, long Index, long A)
```

Parameters

Num

The signal channel number in versions with multichannel switch option or 2nd input channel. In versions without different channels use 1 only.

Index

Specifies the result meaning of request. Devices with two ccd arrays export two different result values, one for each ccd array. With devices with one ccd array (and one display chart) only, always use cMin1, cMax1 or cAvg1 here. Following Index values are available:

cMin1/2: The absolute minimum of the specified interference pattern.

cMax1/2: The absoulte maximum of the specified interference pattern.

cAvg1/2: The average height of the fringes of the specified interference pattern.

A

Reserved for future use.

Return Values

If the function succeeds and the Wavelength Meter is active it returns the corresponding extremum values of the interferometer patterns, else an error indication. For return-information, please look at "Error values of Set…-functions".

GetMinPeak, GetMaxPeak, GetMinPeak2, GetMaxPeak2

The GetMin…MaxPeak functions return the extremum points of the interferometer pattern of channel number 1. These functions principally are obsolete and included for backward compatibility only.

```
long GetMinPeak(long M1)
long GetMaxPeak(long X1)
long GetMinPeak2(long M2)
long GetMaxPeak2(long X2)
```

Parameters

M1, **X1**, **M2**, **X2**
Without meaning, simply use 0.

Return Values
If the functions succeed and the Wavelength Meter is active the functions return the corresponding extremum values of the interferometer patterns, else 0.

Remarks
These functions principally are obsolete and included for backward compatibility only. They can be replaced by using `GetAmplitudeNum`. The following calls but still are equivalent:

```
GetMinPeak(0);    and   GetAmplitudeNum(1, cMin1, 0);
GetMaxPeak(0);    and   GetAmplitudeNum(1, cMax1, 0);
GetMinPeak2(0);   and   GetAmplitudeNum(1, cMin2, 0);
GetMaxPeak2(0);   and   GetAmplitudeNum(1, cMax2, 0);
```

GetAvgPeak, GetAvgPeak2

The `GetAvgPeak` functions return the average amplitude of the interferometer pattern. These functions principally are obsolete and included for backward compatibility only.

```
long GetAvgPeak(long A1)
long GetAvgPeak2(long A2)
```

Parameters

A1, **A2**
Without meaning, simply use 0.

Return Values
If the functions succeed, the Wavelength Meter is active and the average is enabled using the `SetAvgPeak` function, the functions return the corresponding average amplitude values of the interferometer patterns. If the Wavelength Meter is not active they return `ErrWLMMissing` and if the average is not enabled using `SetAvgPeak` in USB1.1 versions they return `ErrNotAvailable`.

Remarks
Devices with two ccd arrays export two different average values, one for each ccd array, the second one can be obtained with `GetAvgPeak2`.

To calculate this average value, the interferometer raw data always needs to be transferred to the computer. In special cases this might lead to a slower repetition rate than with the fastest possible settings, especially for slow interfaces like USB1.1. To provide the maximum speed for usual measurements, this average calculation (and the raw data transfer) is disabled by default with USB1.1 devices. To enable the amplitude average please use the `SetAvgPeak` function. USB2 devices export the average always.

These functions principally are obsolete and included for backward compatibility only. They can be replaced by using `GetAmplitudeNum`. The following calls but still are equivalent:

```
GetAvgPeak(0);    and   GetAmplitudeNum(1, cAvg1, 0);
GetAvgPeak2(0);   and   GetAmplitudeNum(1, cAvg2, 0);
```

SetAvgPeak

The `SetAvgPeak` function dis-/ or enables the calculation and export of the interference pattern amplitude average.

long SetAvgPeak(long AP)

Parameters

AP
Use 1 to enable the average calculation and export and 0 to disable it.

Return Values
For return-information, please look at "Error values of `Set`…-functions".

Remarks
To calculate the interference pattern average amplitude, the interferometer raw data always needs to be transferred to the computer. In special cases this might lead to a slower repetition rate than with the fastest possible settings, especially for slow interfaces like USB1.1. To provide the maximum speed for usual measurements, the average calculation (and the raw data transfer) is disabled by default with USB1.1 devices. To enable the amplitude average please use this function. To obtain the average value then please use the `GetAmplitudeNum` function or use the `CallbackProc` method with the `cmiPatternAvg1…8` constants.

GetExternalInput

The `GetExternalInput` function returns data that previously has been transferred to the Wavelength Meter, either directly or from recorded files.

double GetExternalInput (long Index, double I)

Parameters

Index
Specifies the Index of the data as it was previously set. Possible values are 1 to 64.

I
Reserved for future use.

Return Values
If the function succeeds, the Wavelength Meter is active and a value was previously set for the given Index, this value is returned with double precision. Otherwise -1e100 is returned.

SetExternalInput

The `SetExternalInput` function transfers up to 64 external values to the Wavelength Meter server that can later be fetched synchronized with the measurements, on replaying recorded files for instance.

long SetExternalInput(long Index, double I)

Parameters

Index
Specifies the Index of the data as it was previously set. Up to 64 registers are available that can be addressed by Index directly (1 based).

I
The value to transfer (double precision floating point value). It should neither be NaN nor -1e100. If you want to store integer values please convert them to double beforehand.

Return Values
For return-information, please look at "Error values of `Set`…-functions".

Remarks

It is possible to push up to 64 values to the Wavelength Meter server that can later be read again. These values also are stored to recorded files, synchronized with the measurements when they appeared. This is an easy way to store a set of own data synchronuous with the Wavelength Meter measurements and access them later on replaying the recorded files. An other use is that a client application can read the values directly when an other client writes them. These values also are displayed by the belonging LongTerm program. The values can be fetched using `GetExternalInput` or via the Callback mechanism in combination with `cmiExternalInput`.

4.1.2.3 Operation related functions

GetOperationState

The `GetOperationState` function returns the actual state of measurement.

```
unsigned short GetOperationState(unsigned short Op)
```

Parameters

`Op`

Reserved for future use.

Return Values

If the function succeeds and the Wavelength Meter is active the function returns `cAdjustment` if the program is adjusting, `cMeasurement` if measuring, recording or replaying, else `cStop`.

Operation

The `Operation` function sets the program measurement action mode and enables loading and recording files.

```
long Operation(unsigned short Op)
```

Parameters

`Op`

Controls how a measurement or file accessing activity will be started or stopped. The following base values are available, they must not be combined among themselves:

Base Operation Constants	
`cCtrlStopAll:`	Stops all measurement activites, as well as adjustment, recording and replaying.
`cCtrlStartAdjustment:`	Starts the adjustment mode (data acquisition without further processing).
`cCtrlStartMeasurement:`	Starts the usual measurement mode.
`cCtrlStartRecord:`	Starts the measurement mode with simultaneous recording of all raw data to disk for later analysis.
`cCtrlStartReplay:`	Replays a previously recorded measurement by file.
`cCtrlStoreArray:`	Stores the currently visible measurement data set to disk for later analysis.
`cCtrlLoadArray:`	Loads a previously stored measurement data set.

The following additional flags can be combined with the base operation constants

cCtrlStartRecord, cCtrlStartReplay, cCtrlStoreArray and cCtrlLoadArray:

Operation Addition Constants	
cCtrlOverwrite:	Can be used with cCtrlStartRecord and cctrlStoreArray. Controls whether a possibly existing file will be overwritten. If not set, you will be asked to confirm whether a yet existing file with the same name shall be overwritten. Is without meaning if cCtrlFileDialog is set, too.
cCtrlFileDialog:	If used, you will be prompted by a file selection dialog box instead of using the information of the possibly previously called SetOperationFile function.
cCtrlFileASCII:	Can be used with cCtrlStartRecord and cctrlStoreArray. Controls the file type to be written, binary (smr and ltr files) or ASCII (smx and ltx files). Please also see chapter 4.4 “File formats”. Is ignored, too, if cCtrlFileDialog is set.

Return Values

For return-information, please look at "Error values of Set…-functions".

Remarks

For a file to be accessed without using cCtrlFileDialog, first the existing or desired file location and name must be published with the SetOperationFile function before the Operation function will be called.

SetOperationFile

The SetOperationFile function tells the Wavelength Meter server application about the file name used with the next called Operation function.

```
long SetOperationFile(char *Filename)
```

Parameters

Filename

The complete name of the file to be used, together with the drive and path portion. For file reading accesses this file must exist, else the Wavelength Meter server application will prompt an error message on calling the Operation function. If - for write accesses - the file yet exists, the cCtrlOverwrite flag should be specified with the Operation function, else a file overwrite confirmation message will be displayed.

Return Values

For return-information, please look at "Error values of Set…-functions".

Calibration

The Calibration function calibrates the Wavelength Meter or Laser Spectrum Analyser based on the light of a reference source.

```
long Calibration(long Type, long Unit, double Value, long Channel)
```

Parameters

`Type`

The laser type (or neon lamp) which is used for calibration:

`cHeNe633`: Red HeNe laser; the allowed wavelength range is 632.99 to 632.993 nm.

`cNeL`: Shipped neon lamp. This option is available only with Laser Spectrum Analysers and WS/5, WS/6 and some WS/7 Wavelength Meters.

`cOther`: Any other stabilized reference single mode laser. This option is available only with WS/7, WS/Ultimate and modern WS/5 and WS/6 Wavelength Meters. The allowed calibration wavelength range is:
450 to 900 nm with standard and UV Wavelength Meters
600 to 1750 nm with IR Wavelength Meters
632 to 2000 nm with IR2 Wavelength Meters

`cFreeHeNe`: A free running red HeHe laser. This option is available only with WS/7 Wavelength Meters. The needed calibration wavelength is a distinctive value for the special laser, it is pulished with the laser if the option is purchased.

`Unit`

The physical unit which the Value parameter is interpreted in. Available are `cReturnWavelengthVac`, `cReturnWavelengthAir`, `cReturnFrequency`, `cReturnWavenumber` and `cReturnPhotonEnergy`.

`Value`

The wavelength (resp. other) value the calibration is performed on. This value exactly must match the connected laser and is interpreted in the unit of the `Unit` parameter above.

`Channel`

The switch (multiplexer) channel used for calibration with versions which dispose of the switch option (Note: With channels other than 1, the switch mode must be set in advance.). With Wavelength Meters or Laser Spectrum Analysers without fiber switch option, 1 must be used here.

Return Values

For return-information, please look at "Error values of `Set`…-functions".

Remarks

The calibration only works if no measurement is active.

Please also have a look at chapter 2.2 "Calibration". The parameters used here reflect the options of the GUI calibration dialog. It is strongly recommended to adjust the maximum calibration pattern amplitude to reach 60 to 90 % of the chart detail height before calibration and to only use exactly known reference lasers, best if a stabilized reference HeNe laser.

TriggerMeasurement

The `TriggerMeasurement` function interrupts, continues or triggers the measurement loop.

`long TriggerMeasurement(long Action)`

Parameters

`Action`

`cCtrlMeasurementContinue`: Continues the measurement.

`cCtrlMeasurementInterrupt`:	Interrupts the measurement (the measurement mode but stays active).
`cCtrlMeasurementTriggerPoll`:	Single triggers the measurement polling loop.
`cCtrlMeasurementTriggerSuccess`:	Single triggers the measurement polling loop, but ensures to cover the next successive measurement event.

Return Values

For return-information, please look at "Error values of `Set`…-functions".

Remarks

This function does not have anything in common with starting or stopping the measurement or adjustment mode (see `Operation`). It just controls the flow of an active measurement once per loop circle. The action released with this function remains resident over a started or stopped measurement. If the flow is interrupted with this function (independent if in measurement mode or not), the next time the measurement is started it is started interrupted and will continue only if continued or triggered using this function.

There are two options to set single trigger events: `cCtrlMeasurementTriggerPoll` simply triggers the entire polling loop. Dependent on the triggering rate and the exposure setting (or pulse repetition rate) this can cause to not release a new measurement event if with the actual polling loop no new measurement data was available. If you trigger this way twice or more often per successive measurement cycle, at least one trigger will not be answered. To solve this you can use `cCtrlMeasurementTriggerSuccess`. With such triggers the measurement in any case runs until the next measurement data is available and then interrupts after calculating and exporting the measurement result.

SetBackground

The `SetBackground` function sets the Wavelength Meters background consideration state.

`long SetBackground(long BG)`

Parameters

`BG`

1 to activate the background consideration, 0 to deactivate it.

Return Values

For return-information, please look at "Error values of `Set`…-functions".

Remarks

Depending on the wavelength range and on the exposure of the measurements the ccd arrays can produce more or less characteristic noise, caused by blackbody background and ccd sensitivity characteristics. This effect raises in direction to IR and with increasing exposure, measurements with UV and standard range devices (192-1120 nm) with small exposure for instance do not require the background consideration. This characteristic noise can be reduced significant, increasing the quality of the measurements, if the darkness base information is recorded in advance to the measurements.

Activating the background consideration can take some time, up to several minutes, until all information is collected. If your device has builtin input shutters, the process will continue without user input requirement, devices without internal shutters but require to darken the entrance and acknowledge a belonging displayed message. Deactivating the background consideration takes no time.

4.1.2.4 State and parameter functions

GetResultMode

The `GetResultMode` function returns the result mode which actually is shown on the program surface.

`unsigned short GetResultMode(unsigned short RM)`

Parameters

`RM`
Reserved for future use.

Return Values
If the Wavelength Meter is active the function returns a value from `cReturnWavelengthVac` to `cReturnPhotonEnergy` (see `SetResultMode`) indicating the actual shown result values' unit, else 0.

Remarks
This value has nothing in common with the exported values of `GetFrequency` or `GetWavelength` and is irrelevant in relation to exported measurement results.

SetResultMode

The `SetResultMode` function sets the unit of the displayed result on the program surface.

`long SetResultMode(unsigned short RM)`

Parameters

`RM`

`cReturnWavelengthVac`:	Wavelength vacuum [nm]
`cReturnWavelengthAir`:	Wavelength standard air [nm]
`cReturnFrequency`:	Frequency [THz]
`cReturnWavenumber`:	Wavenumber [cm-1]
`cReturnPhotonEnergy`:	Photon energy [eV]

Return Values
For return-information, please look at "Error values of `Set`…-functions".

Remarks
This value has nothing in common with the exported values of `GetFrequency` or `GetWavelength` and is irrelevant in relation to exported measurement results.

GetRange

The `GetRange` function returns the wavelength range state of the program.

`unsigned short GetRange(unsigned short R)`

Parameters

`R`
Reserved for future use.

Return Values

If the function succeeds and the Wavelength Meter is active the function returns the actual wavelength range state as displayed in the program, else 0:

cRange_250_410:	250 - 410 nm (UV-versions only!)
cRange_250_425:	250 - 425 nm (UV-versions only!)
cRange_300_410:	300 - 410 nm (UV-versions only!)
cRange_350_500:	350 - 500 nm (a few non-IR/UV-versions only!)
cRange_400_725:	400 - 725 nm (non-IR-versions)
cRange_700_1100:	700 - 1100 nm (non-IR-versions)
cRange_900_1500:	900 - 1500 nm (IR-versions only!)
cRange_1100_1700:	1100 - 1700 nm (IR-versions only!)

Remarks

There are Wavelength Meters with and without measurement range preselection. Devices without do not show the Range control group in the left hand control groups bar of the main application window. For such devices this function (as well as SetRange) is without meaning.

SetRange

The SetRange function sets the wavelength range state of the program.

long SetRange(unsigned short R)

Parameters

R

cRange_250_410:	250 - 410 nm (UV-versions only!)
cRange_250_425:	250 - 425 nm (UV-versions only!)
cRange_300_410:	300 - 410 nm (UV-versions only!)
cRange_350_500:	350 - 500 nm (a few non-IR/UV-versions only!)
cRange_400_725:	400 - 725 nm (non-IR-versions)
cRange_700_1100:	700 - 1100 nm (non-IR-versions)
cRange_900_1500:	900 - 1500 nm (IR-versions only!)
cRange_1100_1700:	1100 - 1700 nm (IR-versions only!)

Return Values

For return-information, please look at "Error values of Set…-functions".

Remarks

There are Wavelength Meters with and without measurement range preselection. Devices without do not show the Range control group in the left hand control groups bar of the main application window. For such devices this function (as well as GetRange) is without meaning.

GetPulseMode

The GetPulseMode function returns the programs pulse mode setting if available.

unsigned short GetPulseMode(unsigned short PM)

Parameters

PM

Reserved for future use.

Return Values

If the function succeeds and the Wavelength Meter is active the function returns 0 for continuous mode, 1 for standard pulsed mode, for single pulsed mode (MLC version) and for internally triggered pulsed mode (special triggered version) and 2 for double pulsed mode (MLC version) and for externally triggered pulsed mode (special triggered version) if available in this Wavelength Meter version, else 0.

SetPulseMode

The `SetPulseMode` function sets the pulse or cw measurement mode.

```
long SetPulseMode(unsigned short PM)
```

Parameters

PM

0: Continuous Wave (cw) laser
1: Pulsed laser (standard version)
1: Single pulsed laser (MLC version)
1: Single pulsed laser (internally triggered) (special triggered pulse version)
2: Single pulsed laser (externally triggered) (special triggered pulse version)
2: Two single pulsed lasers simultaneously (double pulsed) and electrical TTL-triggered (MLC version)
3: Two single pulsed lasers simultaneously (double pulsed) and optical triggered (MLC version)

Return Values

For return-information, please look at "Error values of `Set`…-functions".

GetWideMode

The `GetWideMode` function returns the Wavelength Meters' measurement precision mode indicator.

```
unsigned short GetWideMode(unsigned short WM)
```

Parameters

WM

Reserved for future use.

Return Values

If the function succeeds and the Wavelength Meter is active the function returns 1 for selected wide mode setting and 2 for chosen grating analysis (in grating analysis versions only! Except for LSA!), else 0.

SetWideMode

The `SetWideMode` function sets the measurement precision.

```
long SetWideMode(unsigned short WM)
```

Parameters

WM

1 for wide line lasers with less accuracy and 2 for grating analysis (in grating analysis versions only!), 0 for fine lasers with full precision.

Return Values

For return-information, please look at "Error values of `Set`…-functions".

GetDisplayMode

The `GetDisplayMode` function returns the programs display mode setting if available.

```
long GetDisplayMode(long DM)
```

Parameters

DM

Reserved for future use.

Return Values

If the function succeeds and the Wavelength Meter is active and this function is available in this WLM version `GetDisplayMode` returns 1 if the signals in general are to be drawn, else 0.

Remarks

In Wavelength Meters with fiber switch option (MC) or double pulse option (MLC) the real display state but interferes with the per channel display state. The signals are drawn only when this general switch is on and the per channel display state, too.

SetDisplayMode

The `SetDisplayMode` function sets the interference pattern display modes.

```
long SetDisplayMode(long DM)
```

Parameters

DM

0: No interference pattern displayed
1: Generally display (see remarks)

Return Values

For return-information, please look at "Error values of `Set`…-functions".

Remarks

With `DM = 1` and devices with multichannel switch or MLC option (Multi Laser Control) that support double pulse measurements, in switch mode, resp. double pulse mode in MLC versions the channel display state depends on this general setting and also on the per channel display setting (see `Get-/SetSwitcherSettingStates`). The signals are drawn only when this general switch is on and the per channel display state, too.

GetFastMode

The `GetFastMode` function returns the Wavelength Meters fast mode state.

```
bool GetFastMode(bool FM)
```

Parameters

FM

Reserved for future use.

Return Values

If the function succeeds and the Wavelength Meter is active the function returns 1 for selected fast mode setting, else 0.

SetFastMode

The `SetFastMode` function sets the fast mode state.

```
long SetFastMode(bool FM)
```

Parameters

FM

True for faster calculation by fuzzy redrawing of the interference patterns, False for exact drawing. This does not affect the measurement precision.

Return Values

For return-information, please look at "Error values of `Set`…-functions".

GetBackground

The `GetBackground` function returns the Wavelength Meters background consideration state.

```
long GetBackground(long BG)
```

Parameters

BG

Reserved for future use.

Return Values

If the function succeeds and the Wavelength Meter is active the function returns 1 if the background (blackbody background and ccd characteristics) is recorded and its consideration in the measurements is active, else 0.

Remarks

Depending on the wavelength range and on the exposure of the measurements the ccd arrays can produce more or less characteristic noise, caused by blackbody background and ccd sensitivity characteristics. This effect raises in direction to IR and with increasing exposure. This characteristic noise can be reduced significantly, increasing the quality of the measurements, if the darkness base information is recorded in advance to the measurements. How to activate the background consideration please have a look at the `SetBackground` function on page 77.

GetAnalysisMode

The `GetAnalysisMode` function returns the Wavelength Meters analysis mode state.

```
bool GetAnalysisMode(bool AM)
```

Parameters

AM

Reserved for future use.

Return Values

If the function succeeds and the Wavelength Meter is active the function returns 1 if the analysis mode is available and active, else 0.

SetAnalysisMode

The `SetAnalysisMode` function sets the Wavelength Meters analysis mode state.

`long SetAnalysisMode(bool AM)`

Parameters

`AM`
True to activate the analysis mode, False to deactivate it.

Return Values
For return-information, please look at "Error values of `Set`…-functions".

Remarks
The analysis mode is available only with Laser- and High Definition Spectrum Analysers (LSA and HDSA) and Wavelength Meters with Resolver option. The analysis mode allows to calculate, display and export the spectrum of the measured signal.

GetLinewidthMode

The `GetLinewidthMode` function returns the Wavelength Meters linewidth estimation mode state.

`bool GetLinewidthMode(bool LM)`

Parameters

`LM`
Reserved for future use.

Return Values
If the function succeeds and the Wavelength Meter is active the function returns 1 if the linewidth estimation mode is available and active, else 0.

SetLinewidthMode

The `SetLinewidthMode` function sets the Wavelength Meters linewidth estimation mode state.

`long SetLinewidthMode(bool LM)`

Parameters

`LM`
True to activate the linewidth estimation mode, False to deactivate it.

Return Values
For return-information, please look at "Error values of `Set`…-functions".

Remarks
The linewidth estimation mode is an option and not each device consists of it. Test the return value of this `SetLinewidthMode` function to check whether your device has the linewidth estimation option included or not, or simply have a look after the GUI of the Wavelength Meter main application whether this option exists as selectable option.

GetDistanceMode

The `GetDistanceMode` function returns the Wavelength Meters lines distance mode state.

bool GetDistanceMode(bool DM)

Parameters

DM
Reserved for future use.

Return Values
If the function succeeds and the Wavelength Meter is active the function returns 1 if the lines distance mode is available and active, else 0.

SetDistanceMode

The `SetDistanceMode` function sets the Wavelength Meters lines distance mode state.

long SetDistanceMode(bool DM)

Parameters

DM
True to activate the lines distance mode, False to deactivate it.

Return Values
For return-information, please look at "Error values of `Set`…-functions".

Remarks
The lines distance calculation mode is an option and not each device consists of it. Test the return value of this `SetDistanceMode` function to check whether your device has this option included or not, or simply have a look after the GUI of the Wavelength Meter main application whether this option exists as selectable option.

GetIntervalMode

The `GetIntervalMode` function returns the Wavelength Meters interval mode state.

bool GetIntervalMode(bool IM)

Parameters

IM
Reserved for future use.

Return Values
If the function succeeds and the Wavelength Meter is active the function returns `true` if the interval mode is on, else `false`.

SetIntervalMode

The `SetIntervalMode` function sets the interval mode state.

long SetIntervalMode(bool IM)

Parameters

IM
`true` switches the interval mode on, `false` switches it off.

Return Values
For return-information, please look at "Error values of `Set`…-functions".

Remarks

The interval mode allows to modify the measurement repetition rate, resp. adds an interval before each consecutive measurement. To read or adjust the interval value (in ms) itself, please use the `GetInterval`/`SetInterval` functions.
For more manual control of the measurement timing please use the `TriggerMeasurement` function.

GetInterval

The `GetInterval` function returns the Wavelength Meters measurement interval.

`long GetInterval(long I)`

Parameters

`I`
Reserved for future use.

Return Values

If the function succeeds and the Wavelength Meter is active the function returns the interval setting (in ms), otherwise an indication about a missing Wavelength Meter server instance.

SetInterval

The `SetInterval` function sets the Wavelength Meter measurement interval.

`long SetInterval(long I)`

Parameters

`I`
The interval in ms. Possible values reach from 0 to 9'999'990.

Return Values

For return-information, please look at "Error values of `Set`…-functions".

Remarks

Using an interval allows to modify the measurement repetition rate, resp. adds an interval before each consecutive measurement. To activate or deactivate the interval mode please use the `GetIntervalMode`/`SetIntervalMode` functions.
For more manual control of the measurement timing please use the `TriggerMeasurement` function.

GetAutoCalMode

The `GetAutoCalMode` function returns the Wavelength Meters autocalibration mode state.

`long GetAutoCalMode(long ACM)`

Parameters

`ACM`
Reserved for future use.

Return Values

If the function succeeds and the Wavelength Meter is active the function returns 1 for selected autocalibration mode setting, else 0.

SetAutoCalMode

The SetAutoCalMode function sets the autocalibration mode state.

long SetAutoCalMode(long ACM)

Parameters

ACM
Using 1 switches the autocalibration mode on, 0 switches it off.

Return Values
For return-information, please look at "Error values of Set…-functions".

Remarks
The autocalibration rate can be retrieved/set using the Get-/SetAutoCalSetting functions.

GetAutoCalSetting

The GetAutoCalSetting function receives the calibration rate parameters of the autocalibration option.

long GetAutoCalSetting(long ACS, lref val, long Res1, lref Res2)

Parameters

ACS
Indicates the meaning of the val parameter. Possible are the following of the "cmi…"-constants, declared in the header files "wlmData.*" that are enclosed with the demo examples in the "Projects" subdirectory:

Value returned in val:
cmiAutoCalPeriod
cmiAutoCalUnit

Only one of these parameters are used with a single call to this function.

val
A pointer of type long. It receives the values spot to in the ACS parameter. With ACS = cmiAutoCalPeriod, val receives the autoalibration period or rate number. With ACS = cmiAutoCalUnit, val obtains the unit the rate number is interpreted in. Possible unit values are cACOnceOnStart, cACMeasurements, cACDays, cACHours and cACMinutes. With cACOnceOnStart, the software calibrates once each measurement start and the rate is without meaning.

Res1, Res2
Reserved for future use.

Return Values
This function returns 1 if the call was successful and ErrNotAvailable if ACS was set to a value which is not supported, resp. ErrWLMMissing if the Wavelength Meter is not active

Remarks
This function can only be used for versions with autocalibration functionality.

The val parameter is declared as lref, which is defined as long & in C++ versions and long * in plain C. With the address version in C++ the base variables of type long can be overhanded directly, the compiler cares for the pointer management. The indirection version which must be used in C systems (but also can be in C++) needs to overhand the references.

SetAutoCalSetting

The SetAutoCalSetting function sets the calibration rate parameters of the autocalibration function.

long SetAutoCalSetting(long ACS, long val, long Res1, long Res2)

Parameters

ACS

Indicates the meaning of the val parameter. Possible are the following of the "cmi…"-constants, declared in the header files "wlmData.*" that are enclosed with the demo examples in the "Projects" subdirectory:

Value in val:
cmiAutoCalPeriod
cmiAutoCalUnit

Only one of these parameters is used with a single call to this function.

val

Sets the values spot to in the ACS parameter. With ACS = cmiAutoCalPeriod, val determines the calibration period or rate number. With ACS = cmiAutoCalUnit, val specfies the unit the rate is interpreted in. Possible unit values are cACOnceOnStart, cACMeasurements, cACDays, cACHours and cACMinutes. With cACOnceOnStart, the software calibrates once each measurement start and the rate is without meaning.

Res1, Res2

Reserved for future use.

Return Values

For return-information, please have a look at "Error values of Set…-functions".

Remarks

This function can only be used for versions with autocalibration functionality.

If this function is used with an other ACS parameter than decribed here, the behaviour is undefined.

GetActiveChannel

The GetActiveChannel function returns the currently active measurement channel.

long GetActiveChannel(long Mode, lref Port, long Res1)

Parameters

Mode

Controls the interpretation of the return value. Following values are possible:

1 The channel is given in serial order, simply in the return value. This order is, first all channels at the front port and appended all channels at the rear port (if any).

2 The channel is given in separated order. The low word of the return value contains the channel number related to the specific port, the high order word contains the port number, 1 for the front, 2 for the rear port.

3 The channel is given in separated order. The return value contains the channel number related to the specific port and the `Port` parameter contains the port number.

A builtin neon lamp channel is treated like a rear port.

`Port`
A pointer of type long. It receives the port number if `Mode` is 3. 1 indicates the front, 2 the rear port.

`Res1`
Reserved for future use, please use zero.

Return Values
If the Wavelength Meter is active, this function returns the channel as described in the `Mode` parameter, `ErrNotAvailable` if `Mode` was set to a value which is not supported, resp. `0` if the Wavelength Meter is not active.

Remarks
The `Port` parameter is declared as `lref`, which is defined as `long &` in C++ versions and `long *` in plain C. With the address version in C++ the base variables of type `long` can be overhanded directly, the compiler cares for the pointer management. The indirection version which must be used in C systems (but also can be in C++) needs to overhand the references.

SetActiveChannel

The `SetActiveChannel` function sets the currently active measurement channel in non switch mode.

`long SetActiveChannel(long Mode, long Port, long CH, long Res1)`

Parameters

`Mode`
Controls the interpretation of the return value. Following values are possible:

1 The channel needs to be given in serial order in the `CH` parameter. This order is, first all channels at the front port and appended all channels at the rear port (if any).

2 The channel is given in separated order. The low word of the `CH` parameter must contain the channel number related to the specific port, the high order word the port number, 1 for the front, 2 for the rear port.

3 The channel again is treated in separated order. Overhand the channel number related to the specific port in the `CH` parameter and the front or rear port in the `Port` parameter, 1 for the front, 2 for the rear port.

A builtin neon lamp channel is treated like a rear port.

`Port`
If you have set `Mode` to 3, indicate the port here: 1 for the front, 2 for the rear port.

`CH`
Set the channel number here as described in the Mode parameter above.

Res1
Reserved for future use, please leave it zero.

Return Values
For return-information, please have a look at "Error values of `Set`…-functions".

Remarks
This function may only be used in non switch mode. In switch mode please use the SetSwitcherSignalStates function.

Example
Consider you have an 8 channel switch at the front port and a rear port for autocalibration. If you want to switch to the rear autocalibration port, you can use one of the following three conterminous calls:

```
SetActiveChannel(1, 0, 9, 0);
SetActiveChannel(2, 0, 0x00020001, 0);
SetActiveChannel(3, 2, 1, 0);
```

Channel 3 of a multichannel switch would be one of these:

```
SetActiveChannel(1, 0, 3, 0);
SetActiveChannel(2, 0, 0x00010003, 0);
SetActiveChannel(3, 1, 3, 0);
```

GetExposureNum

The `GetExposureNum` function returns the actual valid exposure values of a specific signal.

`long GetExposureNum(long num, long arr, long res)`

Parameters

num
The signal channel for devices with multi channel switch. Should be set to 1 for devices without multi channel switch.

arr
The ccd-array index for devices with more than one ccd array. Can be 1 or 2. Devices display the data for a ccd array inside an own chart each in the main application window. For devices with only one ccd array (WS/5, WS/6, WSD) only 1 is valid here.

res
Reserved for future use.

Return Values
If the function succeeds and the Wavelength Meter is active, the function returns the requested momentary exposure value, else one of the following error values:

`ErrWlmMissing`:	The Wavelength Meter is not active.
`ErrNotAvailable`:	The specified channel or array index is not available for this Wavelength Meter version.

Remarks
For devices with more than one ccd array the real exposure for the 2nd array (`arr` = 2) is interpreted in addition to the one of the 1st array. For this please also have a look after subitem **Exposure** (p. **26**). Usually LSA, WS/7, WS/8, WS Ultimate and WLM's with an additional grating spectrometer have two ccd arrays, WS/5, WS/6 and WSD have just one.

GetExposure, GetExposure2

The `GetExposure` functions return the actual valid exposure values of signal 1.

```
unsigned short GetExposure(unsigned short E)
unsigned short GetExposure2(unsigned short E2)
```

Parameters

E, **E2**
Reserved for future use.

Return Values

If the function succeeds and the Wavelength Meter is active, the functions return the momentary exposure values, else one of the following error values:

`ErrWlmMissing`: The Wavelength Meter is not active.

`ErrNotAvailable`: The called function is not available for this version of Wavelength Meter.

Remarks

`GetExposure` returns the exposure for the Fizeau interferometers or first order grating corresponding to the upper chart in the main program, `GetExposure2` returns the exposure for the wide interferometer(s) in versions with a 2nd ccd array corresponding to the lower chart, or the exposure for the grating analyser, if available.

For Wavelength Meters or Laser Spectrum Analysers with multi channel switch or builtin 2nd or calibration channel these functions are obsolete and should be replaced by `GetExposureNum` in your code.
The following pairs but still are equivalent:

```
GetExposure(0);     and    GetExposureNum(1, 1, 0);
GetExposure2(0);    and    GetExposureNum(1, 2, 0);
```

SetExposureNum

The `SetExposureNum` function sets the interferometers exposure values of a specific signal.

```
long SetExposureNum(long num, long arr, long E)
```

Parameters

num
The signal channel for devices with multi channel switch. Should be set to 1 for devices without multi channel switch.

arr
The ccd-array index for devices with more than one ccd array. Can be 1 or 2. Devices display the data for a ccd array inside an own chart each.

E
Integer exposure value to be set.

Return Values

For return-information, please look at "Error values of `Set`…-functions".

Remarks

Call `GetExposureRange` first (just once) to determine the range of allowed values.
For devices with more than one ccd array the real exposure for the 2nd array (`arr` = 2) is interpreted in addition to the one of the 1st array.

SetExposure, SetExposure2

The `SetExposure` functions set the interferometers exposure values of signal 1.

```
long SetExposure(unsigned short E)
long SetExposure2(unsigned short E2)
```

Parameters

E, **E2**
Integer exposure values to be set.

Return Values
For return-information, please look at "Error values of `Set`…-functions".

Remarks
Call `GetExposureRange` first (just once) to determine the range of allowed values.

For Wavelength Meters with multi channel switch or builtin 2nd or calibration channel these functions are obsolete and should be replaced by `SetExposureNum` in your code.
The following pairs are equivalent:

```
SetExposure(E);     and    SetExposureNum(1, 1, E);
SetExposure2(E);    and    SetExposureNum(1, 2, E);
```

GetExposureMode

The `GetExposureMode` function returns the state of exposure control of the first or only signal channel.

```
bool GetExposureMode(bool EM)
```

Parameters

EM
Reserved for future use.

Return Values
If the function succeeds and the Wavelength Meter is active the function returns 1 if the exposure of the first or only signal channel is set to automatic exposure control, else 0.

Remarks
For Wavelength Meters with multi channel switch or builtin 2nd or calibration channel this function is obsolete and should be replaced by `GetExposureModeNum` in your code.
The following pair but still is equivalent:

```
GetExposureMode(E);       and    GetExposureModeNum(1, E);
```

GetExposureModeNum

The `GetExposureModeNum` function returns the state of exposure control.

```
long GetExposureModeNum(long num, bool EM)
```

Parameters

num
The signal channel for devices with multi channel switch. Should be set to 1 for devices without multi channel switch.

EM
Reserved for future use.

Return Values
If the function succeeds and the Wavelength Meter is active the function returns 1 if the exposure mode of the specific is set to automatic exposure control, else 0.

SetExposureMode

The `SetExposureMode` function sets the mode of exposure control.

`long SetExposureMode(bool EM)`

Parameters

`EM`
True for automatic exposure control, False for manual control.

Return Values
For return-information, please look at "Error values of `Set`…-functions".

Remarks
For Wavelength Meters with multi channel switch or builtin 2nd or calibration channel this function is obsolete and should be replaced by `SetExposureModeNum` in your code.
The following pair but still is equivalent:
`SetExposureMode(E);` and `SetExposureModeNum(1, E);`

SetExposureModeNum

The `SetExposureModeNum` function sets the mode of exposure control of a specific signal channel.

`long SetExposureModeNum(long num, bool EM)`

Parameters

`num`
The signal channel for devices with multi channel switch. Should be set to 1 for devices without multi channel switch.
`EM`
True for automatic exposure control, False for manual control.

Return Values
For return-information, please look at "Error values of `Set`…-functions".

GetExposureRange

The `GetExposureRange` function returns the maximum and minimum of the possible exposure values of the active Wavelength Meter or Laser Spectrum Analyser version.

`long GetExposureRange(long ER)`

Parameters

`ER`
Specifies which value to return :

`cExpoMin`:	Minimum of the interferometers or gratings exposure
`cExpoMax`:	Maximum of the interferometers or gratings exposure
`cExpo2Min`:	Minimum of the wide interferometers or grating exposure if anyone of these available, depending on the device version
`cExpo2Max`:	Maximum of the wide interferometers or grating exposure if anyone of these available, depending on the device version

Return Values

If the function succeeds and the Wavelength Meter or Laser Spectrum Analyser is active the function returns the specified minimum or maximum if available in that version, else 0.

GetLinkState

The `GetLinkState` function returns the link state to the COM-Port.

```
bool GetLinkState(bool LS)
```

Parameters

LS

Reserved for future use.

Return Values

If the function succeeds and the Wavelength Meter is active the function returns 1 (or True) if linked to the COM-Port, else 0 (resp. False).

Remarks

The COM-Port parameters have to be set and requested manually with the program controls.

SetLinkState

The `SetLinkState` function sets the link state to the COM-Port.

```
long SetLinkState(bool LS)
```

Parameters

LS

'True' means connected to the COM-Port, 'False' means disconnected.

Return Values

For return-information, please look at "Error values of `Set`…-functions".

LinkSettingsDlg

The `LinkSettingsDlg` procedure displays a dialog box for setting up the COM-Port parameters for the wavelength exporting (server) and importing (client) capability.

```
void LinkSettingsDlg(void)
```

Parameters and Return Values

This procedure has no parameters and no return values.

Remarks

Using this procedure you can set up all COM-Port parameters for the wavelength exporting (server computer with Wavelength Meter main application running and with direct connection to the Wavelength Meter) and importing (client computer) capability. The parameters (except for the port number) must be chosen equally for both ports, the exporting port on the server computer and the importing port on the client computer. Parity and Flow control should be set to “none”. Please note that for adjusting these settings, the corresponding ports’ link state must be off (resp. “False”, see the `SetLinkState` function). And using this dialog outside of an extra thread will cause your program to behave unresponsive to user actions to other elements than this dialog as long as the dialog is active.

The port setting on the server side can easier be done using the Wavelength Meter main application (menu “Settings | COM Port Settings | Link Port …”), this `LinkSettingsDlg` function is more intended to also be able to adjust the corresponding settings on the client computer.

GetReduced

The `GetReduced` function returns the reduction state of the program surface.

`bool GetReduced(bool R)`

Parameters

`R`
Reserved for future use.

Return Values
If the function succeeds and the Wavelength Meter is active the function returns 1 if the program is reduced, else 0.

SetReduced

The `SetReduced` function sets the surface reduction state.

`long SetReduced(bool R)`

Parameters

`R`
True for reduced, False for enlarged.

Return Values
For return-information, please look at "Error values of `Set`…-functions".

GetScale

The `GetSale` function is obsolete. It still is implemented for compatibility means only.

`unsigned short GetScale(unsigned short S)`

Parameters

`S`
This parameter has no meaning.

Return Values
This function returns 0 always.

SetScale

The `SetScale` function is obsolete. It still is implemented for compatibility means only.

`long SetScale(unsigned short S)`

Parameters

`S`
This parameter has no meaning.

Return Values

If the Wavelength Meter is active, the function returns `ResERR_NotAvailable`, else `ResERR_WlmMissing`. For additional return-information, please look at "Error values of `Set`…-functions".

4.1.2.5 Switch functions (for the optional multichannel fiber switch)

GetSwitcherMode

The `GetSwitcherMode` function returns the general switch mode of the optional multichannel switch.

```
long GetSwitcherMode(long SM)
```

Parameters

SM

Reserved for future use.

Return Values

If the function succeeds and the Wavelength Meter is active the function returns 1 if the switch mode is on and 0 if it is off. Devices without multi channel switch option always return 0.

SetSwitcherMode

The `SetSwitcherMode` function generally switches the switch mode on or off.

```
long SetSwitcherMode(long SM)
```

Parameters

SM

Switch mode; use 1 to change to the switch mode, and 0 to switch it off.

Return Values

For return-information, please look at "Error values of `Set`…-functions".

Remarks

This function has effect with devices with multi channel switch option only.

GetSwitcherChannel

The `GetSwitcherChannel` function returns the currently active channel of the optional multichannel switch.

```
long GetSwitcherChannel(long CH)
```

Parameters

CH

Reserved for future use.

Return Values

If the function succeeds and the Wavelength Meter is active the function returns the currently active switch channel.

Remarks

The result follows the channel of the switching command, regardless whether the switch is connected

or not.

The current switch channel and the current measurement signal number are not synchronuous. The time shift inbetween depends on the different channels' exposure settings and usually is 1-2 measurement cycles. The true point in time of the measured signal is anywhere in the middle between two succeeding switching events. The timestamp of the switching command can be obtained on using `CallbackProcEx` or `WaitForWLMEventEx` (with the `Res1` parameter) instead of this function.

SetSwitcherChannel

The `SetSwitcherChannel` function sets the currently active channel of the optional multichannel switch in non-switch mode.

`long SetSwitcherChannel(long Signal)`

Parameters

`Signal`
The signal number to set active.

Return Values
For return-information, please look at "Error values of `Set`…-functions".

Remarks
In contrast to the `GetSwitcherChannel` function, which also reports about the active channel in switch mode, the `SetSwitcherChannel` function only has effect in non-switch mode. The switch state is set immediately to the desired channel immediately. Caused by the delay of the switch itself and by the logics of the usual measurement flow it but usually takes 1-2 measurement cycles to take effect. To modify the channels used in switch mode, use the `SetSwitcherSignalStates` function.

GetSwitcherSignalStates

The `GetSwitcherSignalStates` function returns the usage and display state of a specified switch channel.

`long GetSwitcherSignalStates(long Signal, lref Use, lref Show)`

Parameters

`Signal`
The signal number.

`Use`
A pointer of type long. Obtains the usage state of the specified channel, 1 if used, 0 if not.

`Show`
A pointer of type long. Obtains the display state of the specified channel, whether the interferometer pattern of this channel are displayed or not. This parameter is 1 if the signal is displayed, 0 if not. The real display state but also depends on the general display state (see `Get/SetDisplayMode`).

Return Values
For return-information, please look at "Error values of `Set`…-functions".

Remarks
This function works with devices with multichannel switch and in switch mode only. The results in non

switch mode might be undefined.

The `Use` and `Show` parameters are declared as `lref`, which is defined as `long &` in C++ versions and `long *` in plain C. With the address version in C++ the base variables of type `long` can be overhanded directly, the compiler cares for the pointer management. The indirection version which must be used in C systems (but also can be in C++) needs to overhand the references.

SetSwitcherSignal, SetSwitcherSignalStates

The `SetSwitcherSignal` and `SetSwitcherSignalStates` functions set the usage and display state of a specified switch channel.

```
long SetSwitcherSignal(long Signal, long Use, long Show)
long SetSwitcherSignalStates(long Signal, long Use, long Show)
```

Parameters

`Signal`
The signal number.

`Use`
Controls whether to use this channel or not. Set 1 to use the channel, 0 to not use the channel.

`Show`
Controls whether to display the interferometer pattern of this channel or not. Set 1 to display the channel, 0 to not display the channel. The real display state but also depends on the general display state (see `SetDisplayMode`).

Return Values

For return-information, please look at "Error values of `Set`…-functions".

Remarks

These functions have the desired effect with devices with multichannel switch and in switch mode only. The usage in non switch mode is undefined and can cause unpredictable results.

The `SetSwitcherSignal` function is obsolete and provided for backward compatibility only. Both these functions but still have the same functionality.

4.1.2.6 Deviation control and PID regulation functions

GetDeviationMode

The `GetDeviationMode` function returns the activity state of the Deviation output (Laser Control) or PID regulation functions.

```
bool GetDeviationMode(bool DM)
```

Parameters

`DM`
Reserved for future use and should be left zero.

Return Values

This function returns `True` if the Deviation output or PID regulation is active, `False` if not. This value reflects the main switch of the Deviation output or PID regulation functions. In the program it can be

found on the left top of the Deviation control/Laser control window, reached by menu "Settings | Deviation control …/", resp. "…Laser control …"

SetDeviationMode

The `SetDeviationMode` function sets the activity state of the Deviation output (Laser Control) or PID regulation functions.

`long SetDeviationMode(bool DM)`

Parameters

`DM`
Use `True` for activating the deviation output or PID regulation, `False` for deactivating.

Return Values
For return-information, please look at "Error values of `Set`…-functions".

GetDeviationReference

The `GetDeviationReference` function returns the reference value for the Deviation output function.

`double GetDeviationReference(double DR)`

Parameters

`DR`
Reserved for future use and should be left zero.

Return Values
This function returns the reference value (given in the unit set in the main program, nm, THz, cm^{-1} or eV) for the Deviation output function.

Remarks
In contrast to the corresponding Set…-function, this function can only be used for the simple Deviation output function, not for versions with PID regulation functionality. In versions with PID functionality please use the `GetPIDCourse` function.

SetDeviationReference

The `SetDeviationReference` function sets the constant reference value for the Deviation output or PID regulation function.

`long SetDeviationReference(double DR)`

Parameters

`DR`
The deviation reference wavelength (or other unit; depending on the main Deviation/PID unit setting). The value finally is cut to the precision which is displayed in the main program, usually one digit more than the standard calculation result has. Also it is limited to the measurement range of the specific device.

Return Values
For return-information, please look at "Error values of `Set`…-functions".

Remarks

This function can only be used to set a constant deviation reference. For setting a variable regulation course in PID versions please use the `SetPIDCourse` function.

GetDeviationSensitivity

The `GetDeviationSensitivity` function returns the dimension of the output sensitivity for the Deviation output function. This function is obsolete but still implemented for backward compatibility.

```
long GetDeviationSensitivity(long DS)
```

Parameters

DS

Reserved for future use and should be left zero.

Return Values

This function returns the dimension portion of the sensitivity for the Deviation output function. The value represents the exponent of the power of ten of the length in the denominator of the sensitivity. The base is nm (if wavelength as base unit is chosen, else THz, 10 cm-1, or eV), so the dimension would be –3 for pm for instance, for a sensitivty of 1 V/pm.

Remarks

This function is obsolete but still implemented for backward compatibility. For obtaining the deviation- as well as the PID output sensitivity (inverse laser amplifiation) please use the `GetPIDSetting` function.

SetDeviationSensitivity

The `SetDeviationSensitivity` function sets the dimension of the output sensitivity for the Deviation output function. This function is obsolete but still implemented for backward compatibility.

```
long SetDeviationSensitivity(long DS)
```

Parameters

DS

The dimension portion of the sensitivity for the Deviation output function. The value represents the exponent of the power of ten of the length in the denominator of the sensitivity. The base is nm (if wavelength as base unit is chosen, else THz, 10 cm-1, or eV), so the dimension would be –4 for 100 fm for instance, for a sensitivty of 1 V/100 fm.

Return Values

For return-information, please look at "Error values of `Set`…-functions".

Remarks

This function is obsolete but still implemented for backward compatibility. For setting the deviation- as well as the PID output sensitivity (inverse laser amplifiation) please use the `SetPIDSetting` function.

GetPIDCourse, GetPIDCourseNum

The `GetPIDCourse/Num` functions receive the reference value or PID regulation course of the PID regulation function.

```
long GetPIDCourse(sref PIDC)
long GetPIDCourseNum(long Port, sref PIDC)
```

Parameters

Port
Defines the port number for versions with more than one analog output port for regulations of more than just one laser at a time.

PIDC
A pointer of character array type which receives the PID course. The character array must be declared with 1024 Bytes size (including the terminating zero) and initialized.

Return Values
These functions return `ResERR_NoErr` if the call went successful, else `ResERR_WlmMissing` or `ResERR_ParmOutOfRange`.

Remarks
These functions can only be used for versions with PID regulation functionality. In versions with simple Deviation control only please use the `GetDeviationReference` function.

The `PIDC` parameter is declared as `sref`, what is defined as `char &` in C++ versions and `char *` in plain C. With the address version in C++ the initialized character array can be overhanded directly, the compiler cares for the pointer management. The indirection version, which must be used in C systems (but also can be in C++), needs to overhand the reference to the array.

The `GetPIDCourse` function is obsolete and included for backward compatibility only. The following calls but still are identical:

`GetPIDCourse(course)` and `GetPIDCourseNum(1, course)`

SetPIDCourse, SetPIDCourseNum

The `SetPIDCourse/Num` functions set the reference value or PID regulation course of the PID regulation function.

```
long SetPIDCourse(sref PIDC)
long SetPIDCourseNum(long Port, sref PIDC)
```

Parameters

Port
Defines the port number for versions with more than one analog output port for regulations of more than just one laser at a time.

PIDC
A pointer of character array type which receives the PID course. The character array can be declared with up to 1024 Bytes size (including the terminating zero). All mathematical elements can be used that are described in the PID settings section in chapter 3.

Return Values
For return-information, please look at "Error values of `Set`…-functions".

Remarks
This function can only be used for versions with PID regulation functionality. In versions with simple Deviation control only, please use the `SetDeviationReference` function to set a constant deviation reference.

To the declaration of the `sref` type of the `PIDC` parameter please have a look at the remarks section of the `GetPIDCourse/Num` function.

The `SetPIDCourse` function is obsolete and included for backward compatibility only. The following

calls but still are identical:

`SetPIDCourse(course)` and `SetPIDCourseNum(1, course)`

GetPIDSetting

The `GetPIDSetting` function receives the PID and sensitivity parameters of the PID regulation function.

`long GetPIDSetting(long PS, long Port, lref iVal, dref dVal)`

Parameters

`PS`

Indicates the meaning of the `iVal` and `dVal` parameters. Possible are the following of the "`cmi`…"-constants, declared in the header files "wlmData.*" that are enclosed with the demo examples in the "Projects" subdirectory:

Value returned in dVal:

`cmiPID_P`	`cmiPID_I`	`cmiPID_D`
`cmiPID_T`	`cmiPID_dt`	`cmiDeviation-SensitivityFactor`

Value returned in iVal:

`cmiDeviationUnit`	`cmiPIDUseTa`	`cmiDeviationPolarity`
`cmiDeviation-SensitivityDim`	`cmiPIDConstdt`	`cmiDeviationChannel`
`cmiPID_AutoClearHistory`		

Only one of the `iVal` and `dVal` parameters are used with a single call to this function.

`Port`

Defines the port number for versions with more than one analog output port for regulations of more than just one laser at a time.

`iVal`

A pointer of type long. It receives the values spot to the `iVal` section of the `PS` parameter.
With `PS` = `cmiPIDUseTa`, `iVal` after the call is 1 if the time constant t_a is used for the regulation calculation.
With `PS` = `cmiPIDConstdt`, `iVal` after the call is 1 if the timing behaviour is eliminated and dt fixed constant, else `iVal` is 0.
If `PS` is `cmiDeviationSensitivityDim`, `iVal` receives the dimension portion of the sensitivity. The value represents the exponent of the power of ten of the length in the denominator of the sensitivity. The base is nm (if wavelength as base unit is chosen, else THz, 10 cm-1, or eV), so the dimension would be –3 for pm for instance, for a sensitivty of 1 V/pm. This dimension still also can be obtained by a call to the `GetDeviationSensitivity` function.
With `PS` = `cmiDeviationPolarity`, `iVal` is set to 1 (positive) or –1 (negative).
With `PS` = `cmiDeviationUnit`, `iVal` holds the set unit of measurement which reflects the PID course unit and sensitivity unit base (a value from `cReturnWavelengthVac` to `cReturnPhotonEnergy`).
With `cmiPID_AutoClearHistory` it receives the automatic clearing setting of the regulation history of the given port.
If `PS` is `cmiDeviationChannel`, `iVal` will hold the channel number of the measurement channel of a belonging multichannel switch (if any) that is assigned to the given port, or 0 if port is no channel assigned.

dVal

A pointer of type `double`.
If `PS` is `cmiPID_P`, `cmiPID_I` or `cmiPID_D`, `dVal` obtains the corresponding PID regulation parameter.
With `PS` = `cmiPID_T`, `dVal` receives the time constant t_a (Note: whether this is finally used, depends on the setting of the UseTa switch, please also see at the `iVal` parameter).
With `cmiPID_dt` dVal receives dt (usage depends on `cmiConstdt`).
If `PS` is set to `cmiDeviationSensitivityFactor`, `dVal` obtains the sensitivity prefactor which can be from 1 to 9,99 with two digits precision.

Return Values

This function returns 1 if the call was successful and `ErrNotAvailable` if `PS` was set to a value which is not supported.

Remarks

This function can only be used for versions with PID regulation functionality.

The `iVal` and `dVal` parameters are declared as `lref` and `dref`, which are defined as `long &` and `double &` in C++ versions and `long *` and `double *` in plain C. With the address versions in C++ the base variables of type `long` and `double` can be overhanded directly, the compiler cares for the pointer management. The indirection versions which must be used in C systems (but also can be in C++) need to overhand the references.

SetPIDSetting

The `SetPIDSetting` function sets the PID and sensitivity parameters of the PID regulation function.

`long SetPIDSetting(long PS, long Port, long iVal, double dVal)`

Parameters

PS

Indicates the meaning of the `iVal` and `dVal` parameters. Possible are the following of the "`cmi`…"-constants, declared in the header files "wlmData.*" that are enclosed with the demo examples in the "Projects" subdirectory:

Value in dVal:

`cmiPID_P`	`cmiPID_I`	`cmiPID_D`
`cmiPID_T`	`cmiPID_dt`	`cmiDeviation-SensitivityFactor`

Value in iVal:

`cmiDeviationUnit`	`cmiPIDUseTa`	`cmiDeviationPolarity`
`cmiDeviation-SensitivityDim`	`cmiPIDConstdt`	`cmiDeviationChannel`
`cmiPID_AutoClearHistory`		

Only one of the `iVal` and `dVal` parameters are used with a single call to this function.

Port

Defines the port number for versions with more than one analog output port for regulations of more than just one laser at a time.

iVal

Sets the values spot to the `iVal` section of the `PS` parameter.
With `PS` = `cmiPIDUseTa`, `iVal` = 1 switches the time constant t_a usage for the regulation calculation on, `iVal` = 0 switches it off.

With PS = cmiPIDConstdt, iVal = 1 switches the constant dt consideration on for the regulation calculation, 0 switches it off.
If PS is cmiDeviationSensitivityDim, iVal is the dimension portion of the sensitivity. Please also have a look at the iVal description of the GetPIDSetting function.
With PS = cmiDeviationPolarity, iVal sets the output polarity, 1 (positive) or –1 (negative) and with PS = cmiDeviationUnit, iVal sets the unit of measurement (cReturnWavelengthVac to cReturnPhotonEnergy) which reflects the PID course unit and sensitivity unit base.
PS = cmiPID_AutoClearHistory sets the automatic regulation history clearing setting of the given port, iVal = 1 means setting it, 0 unsetting.
If PS is cmiDeviationChannel, iVal will assign the given measurement channel of a belonging multichannel switch (if any) to the given port, or unassign any with iVal = 0. Please note that assigning a channel that previously was assigned with an other port automatically releases it from its previous port which then will have no channel assigned. A port can have exactly one channel assigned or none, but not more. Vice versa also a channel can not be assigned to more than one port at a time. Before assigning a channel to a port while the general regulation mode already is active, you should be sure about the validity of the reference course or value. If a port with an invalid reference course or value gets a channel assigned, the regulation will be switched off automatically.

dVal

If PS is cmiPID_P, cmiPID_I or cmiPID_D, the function sets the corresponding PID regulation parameter given by dVal.
With PS = cmiPID_T, dVal represents the time constant t_a (Note: whether this is finally used, depends on the setting of the UseTa switch, please also see at the iVal parameter).
PS = cmiPID_dt sets dt, the constant timing consideration (Note: whether this is finally used, depends on the setting of cmiConstdt, please also see at the iVal parameter).
If PS is set to cmiDeviationSensitivityFactor, dVal must be the sensitivity prefactor which can reach from 1 to 9,99. Note: The program finally cuts the precision of this value to two digits.

Return Values

For return-information, please look at "Error values of Set…-functions".

Remarks

This function can only be used for versions with PID regulation functionality.

If this function is used with an other PS parameter than decribed here, the behaviour is undefined.

ClearPIDHistory

The ClearPIDistory function clears the PID integral history of a given regulation port.

```
long ClearPIDHistory(long Port)
```

Parameters

Port

Defines the port number for versions with more than one analog output port for regulations of more than just one laser at a time.

Return Values

This function always returns 1.

Remarks

This function can only be used for versions with PID regulation functionality.

Usually the integral history is cleared on any changes to the regulation course. This behaviour but

also can be switched off to be able to change the regulation course without clearing the history and the following effect of ugly regulation peaks (see SetPIDSetting function above) and the history therefor can be cleared manually at any desired time.

4.1.2.7 Pattern data functions

GetPatternItemCount, GetAnalysisItemCount

The `GetPatternItemCount` and `GetAnalysisItemCount` functions return the count of exported items per array accessible with `GetPattern`/`Num` and `GetPatternData`/`Num`, resp. `GetAnalysis` and `GetAnalysisData`.

```
long GetPatternItemCount(long Index)
long GetAnalysisItemCount(long Index)
```

Parameters

`Index`

Identifies the specific array. Possible values used with `GetPatternItemCount` are (depending on the WLM version):

`cSignal1Interferometers`:	The array received by the Fizeau interferometers or diffraction grating.
`cSignal1WideInterferometer`:	Additional long interferometer or grating array.
`cSignal1Grating`:	Only in Grating analyzing versions! The array received by spectrum analysis (grating precision).
`cSignal2Interferometers`:	Only in Double Pulse versions! The array received by the Fizeau interferometers for the 2nd pulse.
`cSignal2WideInterferometer`:	Only in Double Pulse versions! Additional long interferometer array for 2nd pulse.

With `GetAnalysisItemCount` only the following value is possible (with LSA and resolver versions):

`cSignalAnalysis`:	The array of one of the ordinates of the spectral data (x or y).

Return Values

If the functions succeed and the Wavelength Meter is active the functions return the count of items of the specific exported array, else 0.

Remarks

To access an array, you need to know its size and location. Its size is determined by the return value of `GetPatternItemCount`, resp. `GetAnalysisItemCount` multiplied with the return value of `GetPatternItemSize`, resp. `GetAnalysisItemSize`. Its location is returned by `GetPattern`/`Num`, resp. `GetAnalysis` if the array is uncovered with `SetPattern`/`SetAnalysis`. To increase the speed of measurement, arrays will not be exported unless ordered using `SetPattern`/`SetAnalysis` and in case of a spectrum (so, with `Index` = `cSignal1Grating` or `cSignalAnalysis`) only if the analysis or grating mode in the main program is active.

For raw pattern data this function needs to be called just once, the count doesn't change anymore.

With analysis data but this count should be determined each time before the data finally is accessed since it can change from measurement to measurement.

GetPatternItemSize, GetAnalysisItemSize

The `GetPatternItemSize` and `GetAnalysisItemSize` functions return the size of exported arrays' items accessible with `GetPattern`/`Num` and `GetPatternData`/`Num`, resp. `GetAnalysis` and `GetAnalysisData`.

```
long GetPatternItemSize(long Index)
long GetAnalysisItemSize(long Index)
```

Parameters

`Index`

Identifies the specific array. Possible values used with `GetPatternItemSize` are (depending on the WLM version):

`cSignal1Interferometers`:	The array received by the Fizeau interferometers or diffraction grating.
`cSignal1WideInterferometer`:	Additional long interferometer or grating array.
`cSignal1Grating`:	Only in Grating analyzing versions! The array received by spectrum analysis (grating precision).
`cSignal2Interferometers`:	Only in Double Pulse versions! The array received by the Fizeau interferometers for the 2nd pulse.
`cSignal2WideInterferometer`:	Only in Double Pulse versions! Additional long interferometer array for 2nd pulse.

With `GetAnalysisItemSize` only the following value is possible (with LSA and resolver versions):

`cSignalAnalysis`:	The array of one of the ordinates of the spectral data (x or y).

Return Values

If the function succeeds and the Wavelength Meter or Laser Spectrum Analyser software is active the functions return the size of single items of the specific exported array in Bytes, else 0. Possible values are 2 (short integer) and 4 (long integer on pattern data and single precision floating on analysis data) and 8 (double precision floating).

Remarks

Depending on hardware differences of various Wavelength Meter and Laser Spectrum Analysis versions the single items of an array use 2 (short integer) and 4 (long integer on pattern data and single precision floating on analysis data) and 8 (double precision floating) Bytes of memory. To access the data use directly working memory addressing functions or create an array of the desired datatype without initializing but pointing to this array.

To access an array, you need to know its size and location. Its size is determined by the return value of `GetPatternItemCount`, resp. `GetAnalysisItemCount` multiplied with the return value of `GetPatternItemSize`, resp. `GetAnalysisItemSize`. Its location is returned by `GetPattern`/`Num`, resp. `GetAnalysis` if the array is uncovered with `SetPattern`/`SetAnalysis`. To increase the speed of measurement, arrays will not be exported unless ordered using `SetPattern`/`SetAnalysis` and in case of a spectrum (so, with `Index` = `cSignal1Grating` or

cSignalAnalysis) only if the analysis or grating mode in the main program is active.

For a specific Index value this function needs to be called just once, the size of a given array type doesn't change anymore.

GetPattern, GetPatternNum, GetAnalysis

The GetPattern/Num and GetAnalysis functions return the memory location of an exported array.

```
long GetPattern(long Index)
long GetPatternNum(long Chn, long Index)
long GetAnalysis(long Index)
```

Parameters

Chn

Identifies the switch channel in versions with multichannel switch. Versions without switch must use 1 here or the GetPattern function (what is the same).

Index

Identifies the specific array. Possible values used with GetPattern/Num are (depending on the WLM version):

cSignal1Interferometers:	The array received by the Fizeau interferometers or diffraction grating.
cSignal1WideInterferometer:	Additional long interferometer or grating array.
cSignal1Grating:	Only in Grating analyzing versions! The array received by spectrum analysis (grating precision).
cSignal2Interferometers:	Only in Double Pulse versions! The array received by the Fizeau interferometers for the 2nd pulse.
cSignal2WideInterferometer:	Only in Double Pulse versions! Additional long interferometer array for 2nd pulse.

With GetAnalysis only the following values are possible (with LSA and resolver versions):

cSignalAnalysisX:	The x axis ordinate (wavelength) array of the spectral data.
cSignalAnalysisY:	The y axis ordinate (amplitude) array of the spectral data.

Return Values

If the functions succeed, the Wavelength Meter is active and the array is uncovered with SetPattern or SetAnalysis, they return the memory position of the first item of the specific exported array, else cPatternDisable, resp. cAnalysisDisable.

Remarks

To access an array, you need to know its size and location. Its size is determined by the return value of GetPatternItemCount, resp. GetAnalysisItemCount multiplied with the return value of GetPatternItemSize, resp. GetAnalysisItemSize. Its location is returned by GetPattern/Num, resp. GetAnalysis if the array is uncovered with SetPattern/SetAnalysis. To increase the speed of measurement, arrays will not be exported unless ordered using SetPattern/SetAnalysis and in case of a spectrum (so, with Index = cSignal1Grating or

cSignalAnalysis) only if the analysis or grating mode in the main program is active.

The pattern data is published right after drawing (if at all) in the main program. So, a specific measurements' data is available until the next measurement data (of the same switch channel) is processed and its data published. To help synchronizing this data with the corresponding measurement result, the publishing of the data can be noticed by the Mode parameter cmiPatternAnalysisWritten of a defined CallbackProc procedure or the WaitForWLMEvent function. The corresponding IntVal parameter is the current channel (signal number) of the multichannel switch (if any). In versions without switch IntVal is 1 always.

For a specific Chn and Index value this function needs to be called just once, the location of a given array doesn't change anymore.

SetPattern, SetAnalysis

The SetPattern and SetAnalysis functions control whether measured pattern or spectral analysis data arrays will be exported.

```
long SetPattern(long Index, long iEnable)
long SetAnalysis(long Index, long iEnable)
```

Parameters

Index

Identifies the specific array. Possible values used with SetPattern are (depending on the WLM version):

cSignal1Interferometers:	The array received by the Fizeau interferometers or diffraction grating.
cSignal1WideInterferometer:	Additional long interferometer or grating array.
cSignal1Grating:	Only in Grating analyzing versions! The array received by spectrum analysis (grating precision).
cSignal2Interferometers:	Only in Double Pulse versions! The array received by the Fizeau interferometers for the 2nd pulse.
cSignal2WideInterferometer:	Only in Double Pulse versions! Additional long interferometer array for 2nd pulse.

With SetAnalysis only the following value is possible (with LSA and resolver versions):

cSignalAnalysis:	The arrays of the spectral analysis data.

iEnable

Controls if the specified array (Index) shall be exported or not. Possible values are cPatternDisable and cPatternEnable (resp. cAnalysisDisable and cAnalysisEnable).

Return Values

For return-information, please look at "Error values of Set…-functions".

Remarks

To access an array, you need to know its size and location. Its size is determined by the return value of GetPatternItemCount, resp. GetAnalysisItemCount multiplied with the return value of GetPatternItemSize, resp. GetAnalysisItemSize. Its location is returned by

GetPattern/Num, resp. GetAnalysis if the array is uncovered with SetPattern/SetAnalysis. To increase the speed of measurement, arrays will not be exported unless ordered using SetPattern/SetAnalysis and in case of a spectrum (so, with Index = cSignal1Grating or cSignalAnalysis) only if also the analysis or grating mode in the main program is active.

GetPatternData, GetPatternDataNum, GetAnalysisData

The GetPatternData/Num and GetAnalysisData functions copy the interferometer pattern or spectral analysis data into an array or a reserved memory location.

```
long GetPatternData(long Index, DWORD PArray)
long GetPatternDataNum(long Chn, long Index, DWORD PArray)
long GetAnalysisData(long Index, DWORD PArray)
```

Parameters

Chn

Only available with GetPatternDataNum. Identifies the switch channel in versions with multichannel switch. Versions without switch must use 1 here or the GetPatternData function (what is the same).

Index

Identifies the specific data to copy. Possible values used with GetPatternData/Num are (depending on the WLM version):

cSignal1Interferometers:	The array received by the Fizeau interferometers or diffraction grating.
cSignal1WideInterferometer:	Additional long interferometer or grating array.
cSignal1Grating:	Only in Grating analyzing versions! The array received by spectrum analysis (grating precision).
cSignal2Interferometers:	Only in Double Pulse versions! The array received by the Fizeau interferometers for the 2nd pulse.
cSignal2WideInterferometer:	Only in Double Pulse versions! Additional long interferometer array for 2nd pulse.

With GetAnalysis only the following values are possible (with LSA and resolver versions):

cSignalAnalysisX:	The x axis ordinate (wavelength) array of the spectral data.
cSignalAnalysisY:	The y axis ordinate (amplitude) array of the spectral data.

PArray

Pointer to the destination array or the start of a memory location where the data shall be copied to. This array or memory location is needed to offer enough memory space for the requested data (see Remarks section).

Return Values

If the functions succeed, the Wavelength Meter is active and the specified array is uncovered with SetPattern, they return cPatternEnable else cPatternDisable (resp. cAnalysisEnable else cAnalysisDisable).

Remarks

To offer space for pattern or spectrum data to be copied, you need to know its size. The size of the copied data is determined by the return value of `GetPatternItemCount`, resp. `GetAnalysisItemCount` multiplied with the return value of `GetPatternItemSize`, resp. `GetAnalysisItemSize`. The data to be copied needs to be uncovered with `SetPattern`/`SetAnalysis`. To increase the speed of measurement, arrays will not be exported unless ordered using `SetPattern`/`SetAnalysis` and in case of a spectrum (so, with `Index` = `cSignal1Grating` or `cSignalAnalysis`) only if also the analysis or grating mode in the main program is active.

Instead of overhanding the pointer to any memory space you also can pass an array directly. In this case the `PArray` parameter needs to be declared as reference. To see how, please look at the declaration section of your used programming language (or closest to it) below inside this chapter or at the header file 'Data.*'.

The pattern (resp. spectrum) data is published right after drawing (if at all) in the main program. So, a specific measurements' data is available until the next measurement data (of the same switch channel) is processed and its data published. To help synchronizing this data with the corresponding measurement result, the publishing of the data can be noticed by the `Mode` parameter `cmiPatternAnalysisWritten` of a defined `CallbackProc` procedure or the `WaitForWLMEvent` function. The corresponding `IntVal` parameter is the current channel (signal number) of the multichannel switch (if any). In versions without switch `IntVal` is 1 always.

4.1.2.8 Other functions

ConvertUnit

The `ConvertUnit` function converts a given wavelength (or other) value into a representation of an other unit.

```
double ConvertUnit(double Val, long uFrom, long uTo)
```

Parameters

`Val`
The value to convert. Must be bigger than zero.

`uFrom`
The physical unit which the `Val` parameter is interpreted in. Available are `cReturnWavelengthVac` [nm], `cReturnWavelengthAir` [nm], `cReturnFrequency` [THz], `cReturnWavenumber` [1/cm] and `cReturnPhotonEnergy` [eV].

`uTo`
The unit to convert to. Available are the same units as with the `uFrom` parameter above.

Return Values

If the function succeeds it returns the `Val` parameter converted to the desired unit, if `Val` was smaller than zero, `Val` is returned nonconverted. Other return values are `ErrDiv0` (if `Val` was zero) and `ErrUnitNotAvailable` (if one of the unit parameters was out of range).

Remarks

This function has no effect on the wavemeter or its GUI application, it simply converts and returns the desired value.

ConvertDeltaUnit

The `ConvertDeltaUnit` function converts a given delta value relative to a base value into a representation of an other unit.

```
double ConvertDeltaUnit(double Base, double Delta, long uBase, long uFrom,
                        long uTo)
```

Parameters

Base
The base value which corresponds to the delta value to convert. Must be bigger than zero.

Delta
The delta value to convert.

uBase
The physical unit which the `Base` parameter is interpreted in. Available are `cReturnWavelengthVac` [nm], `cReturnWavelengthAir` [nm], `cReturnFrequency` [THz], `cReturnWavenumber` [1/cm] and `cReturnPhotonEnergy` [eV].

uFrom
The physical unit which the `Delta` parameter is interpreted in. Available are the same values as with the `uBase` parameter above.

uTo
The unit to convert `Delta` to. Available are the same units as with the `uBase` parameter above.

Return Values
If the function succeeds it returns `Delta` converted to the desired unit, if `Base` was smaller than zero, `Base` is returned. Other return values are `ErrDiv0` (if `Base` was zero) and `ErrUnitNotAvailable` (if one of the unit parameters was out of range).

Remarks
This function can be used to convert values such as linewidths or wavelength distances of two lasers or modes if the base wavelength (or other unit) is known.

This function has no effect on the wavemeter or its GUI application, it simply converts and returns the desired value.

4.1.3 Error value-constants of Set…-functions

Constant	Description
`ResERR_NoErr`:	Success, the value is set properly.
`ResERR_WlmMissing`:	There is no corresponding Wavelength Meter server instance active.
`ResERR_CouldNotSet`:	The Value should be accessible, but could not be set for any reason, please contact Angstrom.
`ResERR_ParmOutOfRange`:	The value to be set is out of its allowed range.
`ResERR_WlmOutOfResources`:	The Wavelength Meter is out of memory or

Constant	Description
	resources. The job could not be done. If this happens frequently, please contact Angstrom.
ResERR_WlmInternalError:	The Wavelength Meter raised an internal error, please contact Angstrom.
ResERR_NotAvailable:	This parameter setting is not available in this Wavelength Meter version.
ResERR_WlmBusy:	The Wavelength Meter was busy and the function returned without success. If this happens frequently, please contact Angstrom.
ResERR_NotInMeasurementMode:	This call is not allowed in measurement mode.
ResERR_OnlyInMeasurementMode:	This call is allowed in measurement mode only.
ResERR_ChannelNotAvailable:	The given channel was out of the available channels range. Wavemeters with multichannel switch option can have up to 8 channels available.
ResERR_ChannelTemporarilyNotAvailable:	The given channel is available generally, but the Wavelength Meter was not in switch mode.
ResERR_CalOptionNotAvailable:	This Wavelength Meter does not dispose of this calibration option.
ResERR_CalWavelengthOutOfRange:	The given calibration wavelength was out of its allowed range.
ResERR_BadCalibrationSignal:	The given wavelength did not match the connected calibration laser or its signal was of bad quality.
ResERR_UnitNotAvailable:	This was not a proper result unit.

4.1.4 Mode constants for Callback-export, the WaitForWLMEvent-function and for saved single measurement and long-term recording files

In the following table the items are marked with *e* if the option is used with `CallbackProc`-Export and `WaitForWLMEvent`. And an *f* indicates usage inside the measurement "frame" in saved files and if the option is irrelevant for further measurement processing it is shown by an *i*.

For Callback-Export and `WaitForWLMEvent` these constants refer to values stored in the `IntVal` parameter except where indicated by a *d* for `DblVal`.

Constant	Description
cmiResultMode:	Represents the unit the calculated result is displayed as on the WLM-server surface. *(efi)*
cmiRange:	The measurement range. See `Get`/`SetRange` above. *(ef)*
cmiPulseMode:	The actually used pulse/cw-mode. See `Get`/`SetPulseMode` above. *(ef)*
cmiWideMode:	The measurement precision. See `Get`/`SetWideMode` above. (*ef*)
cmiFastMode:	In fast mode the WLM-server application draws the interferometer pattern a bit more quickly because fuzzier. See `Get`/`SetFastMode` above. *(ef)*

Constant	Description
`cmiExposureMode`:	Indicates the exposure mode (automatic exposure adjustment while measuring or manual setting). See `Get`/`SetExposureMode` above. *(ef)*
`cmiExposureValue1` - `19`, `cmiExposureValue2` - `29`:	The belonging value is the exposure in [ms]. See `Get`/`SetExposure`/`2`/`Num`. *(ef)*
`cmiReduced`:	The server surface reduced mode (enlarged with interferometer pattern display and all options control elements, or reduced). In reduced mode the fast mode is switched on, which influences the measurements repetition rate. See `Get`/`SetReduced` above. *(ef)*
`cmiTemperature`:	The temperature value in °C. *(def)*
`cmiPressure`:	The pressure value (if any) in mbar. *(def)*
`cmiLink`:	RS232 COM-Port connection setting. See `Get`/`SetLinkState`. *(ef)*
`cmiOperation`:	The measurement mode. See `Operation` and `GetOperationState`. *(ef)*
`cmiDisplayMode`:	The interferometer pattern display mode. Means whether the signals are drawn in the charts and in double pulse versions also which. *(ef)*
`cmiSwitcherMode`:	The multichannel switch mode generally. *(ef)*
`cmiSwitcherChannel`:	The switch channel (in switch mode) which currently is active. This event is not synchronuous to the current measurement signal number. The switching timestamp is overhanded by the `Res1` parameter (in `…Ex`-versions only). *(ef)*
`cmiSwitcherSignal`:	The switch channel (or signal) state (used and shown). *(f)*
`cmiActiveChannel`:	The active switch channel in non switch mode. *(f)*
`cmiNowTick`:	Time correlating to a specific measurement calculation. Represents the interval elapsed since start of the measurement. *(fi)*
`cmiFrequency1`/`2`:	Not used anymore.
`cmiDLLAttach`:	Notification about a WLM server announcement. *(e)*
`cmiDLLDetach`:	Occurs when a WLM server is closed. *(e)*
`cmiVersion`/`0`:	The WLM or LSA main version. 5 to 9 possible. *(ef)*
`cmiVersion1`:	The WLM or LSA serial number. *(ef)*
`cmiAnalysisMode`:	Analysis mode in versions supporting spectral analysis. *(ef)*
`cmiLinewidthMode`:	Linewidth mode state in versions supporting linewidth estimation. *(ef)*
`cmiDistanceMode`:	Lines distance mode state in versions supporting this option. *(ef)*
`cmiAppearance`:	WLM-server visibility. See `Instantiate`. *(e)*
`cmiWavelength1` - `9`:	The calculated wavelength in [nm]. In double pulse and multichannel switch versions there are more than one. Refers to the `DblVal` parameter with `CallbackProc` and `WaitForWLMEvent`. *(def)*
`cmiLinewidth`:	The calculated linewidth in [nm]. Refers to the `DblVal` parameter with `CallbackProc` and `WaitForWLMEvent`. *(def)*
`cmiDistance`:	The distance between signal 1 and 2 in multichannel switch versions

Constant	Description
	with Diff option. *(def)*
cmiAnalogIn:	The analog input voltage in versions with analog input port. *(defi)*
cmiAnalogOut/1 - 8:	The analog output voltage in Laser Control and PID versions. *(defi)*
cmiPower - cmiPower + 7:	The signal power in µW. *(def)*
cmiMin1 - 19, cmiMin2 - 29:	The pattern minimum. *(ef)*
cmiMax1 - 19, cmiMax2 - 29:	The pattern maximum. *(ef)*
cmiAvg11 - 19, cmiAvg21 - 29:	The pattern peaks' average amplitude. *(ef)*
cmiDeviationMode:	The main Deviation/PID activity switch in Laser Control and PID versions. *(ef)*
cmiDeviationReference:	The constant Deviation/PID reference in Laser Control and PID versions. *(def)*
cmiDeviationUnit:	The Deviation/PID reference value or formula interpertation unit in Laser Control and PID versions. *(ef)*
cmiDeviationSensitivity:	Dimension portion of output sensitivity in Laser Control versions. *(ef)*
cmiPIDCourse:	The PID regulation course in PID versions. *(ef)*
cmiPIDUseT:	The time interpretation usage flag in PID regulation versions. *(ef)*
cmiPIDConstdt:	The time constant usage flag in PID regulation versions. *(ef)*
cmiPID_P, ...I, ...D, ...T, ...dt:	The P, I, D, T (ta) and dt parameters in PID regulation versions. *(def)*
cmiPID_AutoClearHistory:	The auto clearance flag of the integral history in case of mistakes and output overruns in PID regulation versions. *(ef)*
cmiDeviationSensitivity Dim:	Dimension portion of the output sensitivity in Laser Control and PID versions. *(ef)*
cmiDeviationSensitivity Factor:	Sensitivity prefactor in Laser Control and PID versions. *(def)*
cmiDeviationPolarity:	Voltage output polarity in Laser Control and PID versions. *(ef)*
cmiPatternAnalysisWritten:	Informs about pattern or analysis data being transferred to a client application. *(ei)*
cmiBackground:	Blackbody background and ccd array characterisics consideration. *(f)*
cmiExternalInput - cmiExternalInput + 63:	External user values transferred to the WLM server (64 possible). *(defi)*
cmiServerInitialized:	Notification about a finished WLM server initialization on start or about an uninitialization on exit. *(e)*

4.1.5 Return error value constants of GetFrequency… and GetWavelength…

Please see the declaration of GetFrequency1/2 and GetWavelength1/2/Num above (page *64)*.

4.1.6 Return error value constants of GetTemperature

Please see the declaration of `GetTemperature` above (page *66)*.

4.1.7 Importdeclarations

Note: *The wlmData.dll uses shared memory for interprocess communication. This requires that the server (the Wavelength Meter application) and the clients (any access and control programs, your own ones for instance) access the same dll file, not just identical copies of this file. The Wavelength Meter application accesses the dll in the System32 directory. If your application but uses its private copy inside its own directory for instance, the connection can't establish and called functions will return one of the error indicators* `ErrWLMMissing` *or* `ResERR_WlmMissing` *(see functions* `GetFrequencyNum`, `GetWavelengthNum` *on page 64 and chapter 4.1.3 "*Error value-constants of Set…-functions*", p. 110). On installation, the wlmData.dll file is installed to the System32 directory. All other programs should access the dll in the System32 directory, too. So we recommend to not create any other copies of this file on the computer where the Wavelength Meter is installed.*

4.1.7.1 C

The necessary header for C environments, 'wlmData.h', can be found in the subdirectory '\Projects\DataDemo\C' in the installation directory of the Wavelength Meter. The import-symbol files in COFF (wlmData.lib, MS style) and OMF (wlmData.OMF.lib, Borland Style) format are included, too. The used constants also are listed below in chapter 4.1.8 "Used Constants".

4.1.7.2 Pascal

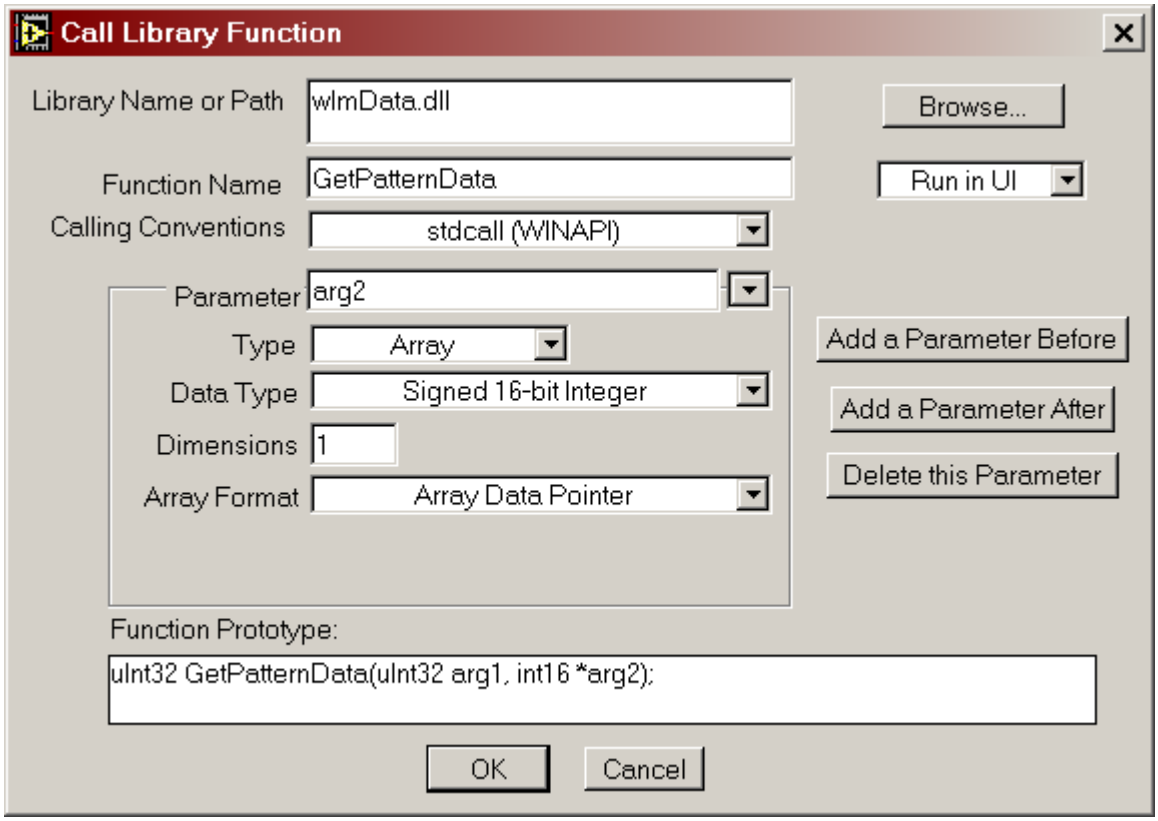
The required header file 'Data.pas' for Pascal environments like Delphi can be found in the subdirectory '\Projects\DataDemo\Delphi', the used constants also are listed below in chapter 4.1.8 "Used Constants".

4.1.7.3 Basic

Please find the header file 'Data.bas' for Basic environments in the subdirectory '\Projects\DataDemo\Visual Basic', the used constants also are listed below in chapter 4.1.8 "Used Constants".

4.1.7.4 LabView

In LabView all functions can be accessed creating a 'Call Library Function' knot for each function to include. Below you can find the required function declaration informations for LabView. All functions have to be declared with the library name 'wlmData.dll' and the 'stdcall' calling conventions and all arguments have to be passed as value except where indicated by '`(ref)`' (as reference). For extra declaration information please have a look at Importdeclarations header file for C environments. Additionally you can find a few small demo vi's in the subdirectory '\Projects\DataDemo\LabView', demonstrating the dll integration into LabView 5.1 and 8.5.



This image shows how to implement a wlmData.dll-library function within LabView (here version 5.1) at an example of the `GetPatternData` function. The 'Function Prototype'-box shows the syntax as it would be declared in C/C++.

```
void __stdcall CallbackProc(long Mode, long IntVal, double DblVal) ;
Data_API(long) WaitForWLMEvent(long &Mode, long &IntVal, double &DblVal);
```

Function Name	Return Type	Type Arg1	Type Arg2	Type Arg3	Type Arg4
Instantiate	Int32	Int32	Int32	Int32	Int32
WaitForWLMEvent	Int32	Int32 (ref)	Int32 (ref)	double (ref)	
WaitForWLMEventEx				Int32 (ref)	double (ref) **Arg5** Int32 (ref)
PresetWLMIndex	Int32	Int32			
ControlWLM	Int32	Int32	Int32 or String (psp)	Int32	
ControlWLMEx					Int32 **Arg5** Int32
SetMeasurementDelayMethod	Int32	Int32	Int32		
SetWLMPriority	Int32	Int32	Int32	Int32	
Get…-functions					

Function Name	Return Type	Type Arg1	Type Arg2	Type Arg3	Type Arg4
GetWLMVersion	Int32	Int32			
GetWLMIndex	Int32	Int32			
GetWLMCount	Int32	Int32			
GetFrequencyNum	double	Int32	double		
GetFrequency	double	double			
GetFrequency2	double	double			
GetWavelengthNum	double	Int32	double		
GetWavelength	double	double			
GetWavelength2	double	double			
GetCalWavelength	double	Int32	double		
GetLinewidth	double	Int32	double		
GetTemperature	double	double			
GetPressure	double	double			
GetExposure	uInt16	uInt16			
GetExposure2	uInt16	uInt16			
GetExposureNum	Int32	Int32	Int32	Int32	
GetMinPeak	Int32	Int32			
GetMinPeak2	Int32	Int32			
GetMaxPeak	Int32	Int32			
GetMaxPeak2	Int32	Int32			
GetAvgPeak	Int32	Int32			
GetAvgPeak2	Int32	Int32			
GetAmplitudeNum	Int32	Int32	Int32	Int32	
GetPowerNum	double	Int32	double		
GetExposureMode	Int8	Int8			
GetExposureModeNum	Int32	Int32	Int8		
GetExposureRange	Int32	Int32			
GetResultMode	uInt16	uInt16			
GetRange	uInt16	uInt16			
GetPulseMode	uInt16	uInt16			
GetWideMode	uInt16	uInt16			
GetFastMode	Int8	Int8			
GetDisplayMode	Int32	Int32			
GetSwitcherMode	Int32	Int32			
GetIntervalMode	Int8	Int8			
GetInterval	Int32	Int32			
GetAutoCalMode	Int32	Int32			
GetAutoCalSetting	Int32	Int32	Int32 (ref)	Int32	Int32 (ref)
GetSwitcherChannel	Int32	Int32			
GetActiveChannel	Int32	Int32	Int32 (ref)	Int32	
GetChannelsCount	Int32	Int32			
GetSwitcherSignalStates	Int32	Int32	Int32 (ref)	Int32 (ref)	
GetDeviationMode	Int8	Int8			
GetDeviationReference	double	double			

Function Name	Return Type	Type Arg1	Type Arg2	Type Arg3	Type Arg4
GetDeviationSensitivity	Int32	Int32			
GetDeviationSignal	double	double			
GetDeviationSignalNum	double	Int32	double		
GetPIDCourse	Int32	String (psp)			
GetPIDCourseNum	Int32	Int32	String (psp)		
GetPIDSetting	Int32	Int32	Int32	Int32 (ref)	double (ref)
GetReduced	Int8	Int8			
GetScale	uInt16	uInt16			
GetLinkState	Int8	Int8			
GetOperationState	uInt16	uInt16			
GetBackGround	Int32	Int32			
GetPatternItemCount	Int32	Int32			
GetPatternItemSize	Int32	Int32			
GetPattern	Int32	Int32			
GetPatternNum	Int32	Int32	Int32		
GetPatternData	Int32	Array uInt32 (ref)			
GetPatternDataNum	Int32	Int32	Array uInt32 (ref)		
GetAnalysisMode	Int8	Int8			
GetAnalysisItemCount	Int32	Int32			
GetAnalysisItemSize	Int32	Int32			
GetAnalysis	Int32	Int32			
GetAnalysisData	Int32	Array uInt32 (ref)			
GetLinewidthMode	Int8	Int8			
GetDistanceMode	Int8	Int8			
GetDistance	double	double			
GetAnalogIn	double	double			
GetExternalInput	double	Int32	double		
Set…-functions					
SetExposure	Int32	uInt16			
SetExposure2	Int32	uInt16			
SetExposureNum	Int32	Int32	Int32	Int32	
SetExposureMode	Int32	Int8			
SetExposureModeNum	Int32	Int32	Int8		
SetResultMode	Int32	uInt16			
SetRange	Int32	uInt16			
SetPulseMode	Int32	uInt16			
SetWideMode	Int32	uInt16			

Function Name	Return Type	Type Arg1	Type Arg2	Type Arg3	Type Arg4
SetFastMode	Int32	Int8			
SetDisplayMode	Int32	Int32			
SetSwitcherMode	Int32	Int32			
SetIntervalMode	Int32	Int8			
SetInterval	Int32	Int32			
SetAutoCalMode	Int32	Int32			
SetAnalysisMode	Int32	Int8			
SetLinewidthMode	Int32	Int8			
SetDistanceMode	Int32	Int8			
SetAutoCalSetting	Int32	Int32	Int32	Int32	Int32
SetSwitcherChannel	Int32	Int32			
SetActiveChannel	Int32	Int32	Int32	Int32	Int32
SetSwitcherSignal/States	Int32	Int32	Int32	Int32	
SetDeviationMode	Int32	Int8			
SetDeviationReference	Int32	double			
SetDeviationSensitivity	Int32	Int32			
SetDeviationSignal	Int32	double			
SetDeviationSignalNum	Int32	Int32	double		
SetPIDCourse	Int32	String (psp)			
SetPIDCourseNum	Int32	Int32	String (psp)		
SetPIDSetting	Int32	Int32	Int32	Int32	double
ClearPIDHistory	Int32	Int32			
SetReduced	Int32	Int8			
SetScale	Int32	uInt16			
SetLinkState	Int32	Int8			
LinkSettingsDlg					
SetAvgPeak	Int32	Int32			
Operation	Int32	uInt16			
Calibration	Int32	Int32	Int32	double	Int32
TriggerMeasurement	Int32	Int32			
SetBackGround	Int32	Int32			
SetOperationFile	Int32	String (psp)			
SetPattern	Int32	Int32	Int32		
SetAnalysis	Int32	Int32	Int32		
SetPressure	Int32	Int32	double		
SetExternalInput	Int32	Int32	double		
Other…-functions					
ConvertUnit	double	double	Int32	Int32	
ConvertDeltaUnit	double	double	double	Int32	Int32 **Arg5** Int32

4.1.8 Used constants

These are the constants meaningful for accessing the Wavelength Meter software interface wlmData.dll. They are declared in the header files 'Data.*' closest matching your development environment and can be found as part of the several example-projects, in subdirectories of "<WLM-InstDir>\Projects\", typically "C:\Programs\Angstrom\Wavelength Meter…\Projects\".

Instantiating Constants for 'RFC' parameter

```
cInstCheckForWLM = -1
cInstResetCalc = 0
cInstReturnMode = cInstResetCalc
cInstNotification = 1
cInstCopyPattern = 2
cInstCopyAnalysis = cInstCopyPattern
cInstControlWLM = 3
cInstControlDelay = 4
cInstControlPriority = 5
```

Notification Constants for 'Mode' parameter

```
cNotifyInstallCallback = 0
cNotifyRemoveCallback = 1
cNotifyInstallWaitEvent = 2
cNotifyRemoveWaitEvent = 3
cNotifyInstallCallbackEx = 4
cNotifyInstallWaitEventEx = 5
```

Result Error Constants of Set…-functions

```
ResERR_NoErr = 0
ResERR_WlmMissing = -1
ResERR_CouldNotSet = -2
ResERR_ParmOutOfRange = -3
ResERR_WlmOutOfResources = -4
ResERR_WlmInternalError = -5
ResERR_NotAvailable = -6
ResERR_WlmBusy = -7
ResERR_NotInMeasurementMode = -8
ResERR_OnlyInMeasurementMode = -9
ResERR_ChannelNotAvailable = -10
ResERR_ChannelTemporarilyNotAvailable = -11
ResERR_CalOptionNotAvailable = -12
ResERR_CalWavelengthOutOfRange = -13
ResERR_BadCalibrationSignal = -14
ResERR_UnitNotAvailable = -15
```

Mode Constants for Callback-Export and WaitForWLMEvent-function

```
cmiResultMode = 1
cmiRange = 2
cmiPulse = 3
cmiPulseMode = cmiPulse
cmiWideLine = 4
cmiWideMode = cmiWideLine
cmiFast = 5
cmiFastMode = cmiFast
cmiExposureMode = 6
cmiExposureValue1 = 7
cmiExposureValue2 = 8
cmiReduce = 12
cmiReduced = cmiReduce
cmiScale = 13
```

```
cmiTemperature = 14
cmiLink = 15
cmiOperation = 16
cmiDisplayMode = 17
cmiMin1 = 22
cmiMax1 = 23
cmiMin2 = 24
cmiMax2 = 25
cmiNowTick = 26
cmiFrequency1 = 28
cmiFrequency2 = 29
cmiDLLDetach = 30
cmiVersion = 31
cmiAnalysisMode = 32
cmiDeviationMode = 33
cmiDeviationReference = 34
cmiDeviationSensitivity = 35
cmiAppearance = 36
cmiAutoCalMode = 37
cmiWavelength1 = 42
cmiWavelength2 = 43
cmiLinewidth = 44
cmiLinewidthMode = 45
cmiAnalysis = 57
cmiAnalogIn = 66
cmiAnalogOut = 67
cmiDistance = 69
cmiWavelength3 = 90
cmiWavelength4 = 91
cmiWavelength5 = 92
cmiWavelength6 = 93
cmiWavelength7 = 94
cmiWavelength8 = 95
cmiVersion0 = cmiVersion
cmiVersion1 = 96
cmiDLLAttach = 121
cmiSwitcherSignal = 123
cmiSwitcherMode = 124
cmiExposureValue11 = cmiExposureValue1
cmiExposureValue12 = 125
cmiExposureValue13 = 126
cmiExposureValue14 = 127
cmiExposureValue15 = 128
cmiExposureValue16 = 129
cmiExposureValue17 = 130
cmiExposureValue18 = 131
cmiExposureValue21 = cmiExposureValue2
cmiExposureValue22 = 132
cmiExposureValue23 = 133
cmiExposureValue24 = 134
cmiExposureValue25 = 135
cmiExposureValue26 = 136
cmiExposureValue27 = 137
cmiExposureValue28 = 138
cmiPatternAverage = 139
cmiPatternAvg1 = 140
cmiPatternAvg2 = 141
cmiAnalogOut1 = cmiAnalogOut
cmiAnalogOut2 = 142
```

```
cmiMin11 = cmiMin1
cmiMin12 = 146
cmiMin13 = 147
cmiMin14 = 148
cmiMin15 = 149
cmiMin16 = 150
cmiMin17 = 151
cmiMin18 = 152
cmiMin21 = cmiMin2
cmiMin22 = 153
cmiMin23 = 154
cmiMin24 = 155
cmiMin25 = 156
cmiMin26 = 157
cmiMin27 = 158
cmiMin28 = 159
cmiMax11 = cmiMax1
cmiMax12 = 160
cmiMax13 = 161
cmiMax14 = 162
cmiMax15 = 163
cmiMax16 = 164
cmiMax17 = 165
cmiMax18 = 166
cmiMax21 = cmiMax2
cmiMax22 = 167
cmiMax23 = 168
cmiMax24 = 169
cmiMax25 = 170
cmiMax26 = 171
cmiMax27 = 172
cmiMax28 = 173
cmiAvg11 = cmiPatternAvg1
cmiAvg12 = 174
cmiAvg13 = 175
cmiAvg14 = 176
cmiAvg15 = 177
cmiAvg16 = 178
cmiAvg17 = 179
cmiAvg18 = 180
cmiAvg21 = cmiPatternAvg2
cmiAvg22 = 181
cmiAvg23 = 182
cmiAvg24 = 183
cmiAvg25 = 184
cmiAvg26 = 185
cmiAvg27 = 186
cmiAvg28 = 187
cmiPatternAnalysisWritten = 202
cmiSwitcherChannel = 203
cmiAnalogOut3 = 237
cmiAnalogOut4 = 238
cmiAnalogOut5 = 239
cmiAnalogOut6 = 240
cmiAnalogOut7 = 241
cmiAnalogOut8 = 242
cmiIntensity = 251
cmiPower = 267
cmiActiveChannel = 300
```

```
cmiPIDCourse = 1030
cmiPIDUseTa = 1031
cmiPIDUseT = cmiPIDUseTa
cmiPID_T = 1033
cmiPID_P = 1034
cmiPID_I = 1035
cmiPID_D = 1036
cmiDeviationSensitivityDim = 1040
cmiDeviationSensitivityFactor = 1037
cmiDeviationPolarity = 1038
cmiDeviationSensitivityEx = 1039
cmiDeviationUnit = 1041
cmiPIDConstdt = 1059
cmiPID_dt = 1060
cmiPID_AutoClearHistory = 1061
cmiAutoCalPeriod = 1120
cmiAutoCalUnit = 1121
cmiServerInitialized = 1124
cmiWavelength9 = 1130
cmiExposureValue19 = 1155
cmiExposureValue29 = 1180
cmiMin19 = 1205
cmiMin29 = 1230
cmiMax19 = 1255
cmiMax29 = 1280
cmiAvg19 = 1305
cmiAvg29 = 1330
cmiExternalInput = 1400
cmiPressure = 1465
cmiBackground = 1475
cmiDistanceMode = 1476
cmiIntervalMode = 1477
cmiInterval = 1478
```

WLM Control Mode Constants

```
cCtrlWLMShow = 1
cCtrlWLMHide = 2
cCtrlWLMExit = 3
cCtrlWLMWait = 0x0010
cCtrlWLMStartSilent = 0x0020
cCtrlWLMSilent = 0x0040
```

Operation Mode Constants

```
cStop = 0x00
cAdjustment = 0x0001
cMeasurement = 0x0002
```

Base Operation Constants

```
cCtrlStopAll = cStop
cCtrlStartAdjustment = cAdjustment
cCtrlStartMeasurement = cMeasurement
cCtrlStartRecord = 0x0004
cCtrlStartReplay = 0x0008
cCtrlStoreArray = 0x0010
cCtrlLoadArray = 0x0020
```

Additional Operation Flag Constants

```
cCtrlDontOverwrite = 0x0000
cCtrlOverwrite = 0x1000
```

```
cCtrlFileGiven = 0x0000
cCtrlFileDialog = 0x2000
cCtrlFileBinary = 0x0000
cCtrlFileASCII = 0x4000
```

Measurement Control Mode Constants

```
cCtrlMeasDelayRemove = 0
cCtrlMeasDelayGenerally = 1
cCtrlMeasDelayOnce = 2
cCtrlMeasDelayDenyUntil = 3
cCtrlMeasDelayIdleOnce = 4
cCtrlMeasDelayIdleEach = 5
cCtrlMeasDelayDefault = 6
```

Measurement triggering action constants

```
cCtrlMeasurementContinue = 0
cCtrlMeasurementInterrupt = 1
cCtrlMeasurementTriggerPoll = 2
cCtrlMeasurementTriggerSuccess = 3
```

Exposure Range Constants

```
cExpoMin = 0
cExpoMax = 1
cExpo2Min = 2
cExpo2Max = 3
```

Measurement Range Constants

```
cRange_250_410 = 4
cRange_250_425 = 0
cRange_300_410 = 3
cRange_350_500 = 5
cRange_400_725 = 1
cRange_700_1100 = 2
cRange_800_1300 = 6
cRange_900_1500 = cRange_800_1300
cRange_1100_1700 = 7
cRange_1100_1800 = cRange_1100_1700
```

Amplitude Constants

```
cMin1 = 0
cMin2 = 1
cMax1 = 2
cMax2 = 3
cAvg1 = 4
cAvg2 = 5
```

Unit Constants for Get-/SetResultMode, GetLinewidth and Calibration

```
cReturnWavelengthVac = 0
cReturnWavelengthAir = 1
cReturnFrequency = 2
cReturnWavenumber = 3
cReturnPhotonEnergy = 4
```

Power Unit constants

```
cPower_muW = 0
cPower_dBm = 1
```

Source Type Constants for Calibration

```
cHeNe633 = 0
```

```
cHeNe1152 = 0
cNeL = 1
cOther = 2
cFreeHeNe = 3
```

Unit Constants for Autocalibration

```
cACOnceOnStart = 0
cACMeasurements = 1
cACDays = 2
cACHours = 3
cACMinutes = 4
```

Pattern and Analysis Constants

```
cPatternDisable = 0
cPatternEnable = 1
cAnalysisDisable = cPatternDisable
cAnalysisEnable = cPatternEnable

cSignal1Interferometers = 0
cSignal1WideInterferometer = 1
cSignal1Grating = 1
cSignal2Interferometers = 2
cSignal2WideInterferometer = 3
cSignalAnalysis = 4
cSignalAnalysisX = cSignalAnalysis
cSignalAnalysisY = cSignalAnalysis + 1
```

Return error values of GetFrequency and GetWavelength

```
ErrNoValue = 0
ErrNoSignal = -1
ErrBadSignal = -2
ErrLowSignal = -3
ErrBigSignal = -4
ErrWlmMissing = -5
ErrNotAvailable = -6
ErrNoPulse = -8
InfNothingChanged = -7
ErrDiv0 = -13
ErrOutOfRange = -14
ErrUnitNotAvailable = -15
```

Return error values of GetTemperature and GetPressure

```
ErrTemperature = -1000
ErrTempNotMeasured = ErrTemperature + ErrNoValue
ErrTempNotAvailable = ErrTemperature + ErrNotAvailable
ErrTempWlmMissing = ErrTemperature + ErrWlmMissing
```

Return error values of GetDistance

(real errorvalues are ErrDistance combined with those of GetWavelength)

```
ErrDistance = -1000000000
ErrDistanceNotAvailable = ErrDistance + ErrNotAvailable
ErrDistanceWlmMissing = ErrDistance + ErrWlmMissing
```

Return flags of ControlWLMEx

(in combination with cCtrlShow or cCtrlHide, and cCtrlWait and Res = 1)

```
flServerStarted = 0x0001
flErrDeviceNotFound = 0x0002
flErrDriverError = 0x0004
flErrUSBError = 0x0008
```

```
flErrUnknownDeviceError = 0x0010
flErrWrongSN = 0x0020
flErrUnknownSN = 0x0040
flErrTemperatureError = 0x0080
flErrPressureError = 0x0100
flErrCancelledanually = 0x0200
flErrUnknownError = 0x1000
```

Note: *If possible, do not use the numerals of these constants directly, they may alter. Use the constant's names instead.*

4.2 Measurement examples

The measurement results and all program state values and settings are accessible for user's programs via calls of wlmData.dll-routines. You can find complete sourcecode examples in the subdirectory 'Projects' of your Wavelength Meter installation path. These examples exist for C/C++, Visual Basic, Delphi and LabView.

A complete control sample (DataDemo.exe compiled using Delphi 6) is available in the program path and over the startmenu and also a long-term measurement demo (LongTerm.exe). Both these, too, are shipped with complete sourcecodes. They can be called manually from within the installed directory or via "Startmenu | Programs | Angstrom | Wavelength Meter…", and it's also possible to get them launched automatically (with a started measurement or on start of the entire Wavelength Meter program). How to perform this, please have a look at "Start-Settings" and there at "Extra program" and "Command line".

- **DataDemo** (chapter 4.2.1) illustrates the receiving and setting of lots of program informations.
- **LongTerm** (chapter 4.2.2) illustrates a way to write a little long-term graphing program. It displays the calculated wavelength and the temperature inside the optical unit.
- **LongTerm (callback style)** (chapter 4.2.3) quite like the previous one but faster using the callback mechanism.
- **COM-Port-Client (callback style)** (chapter 4.3) shows how to access the measured wavelength on another computer connected via RS232 COM-Port.

4.2.1 Measurement Example "DataDemo"

DataDemo is made to illustrate the receiving and setting of all program informations.

Use the "Start" button to start observing the Wavelength Meter's states and settings. For the setting "only new frequency values" please see the description of the dll-function "`Instantiate`" in "External access", it guarantees every measurement to be checked just once. Anywhere where you can change a setting, the Wavelength Meter will respond to.Use the button "Pattern >>>" to open the second part of this demo view. When you select one of the GetPattern-checkboxes, the Wavelength Meter first has to measure a new pattern before it will be displayed.

Use the little thumbtack to toggle the demo's topmost state. In its topmost state, a window will appear on top of all other windows, even if it has lost the focus.

Note: Not all states, that are listed in the dropdown-boxes, are possible to be used with each Wavelength Meter version. The grey box below indicates the success of a done operation.

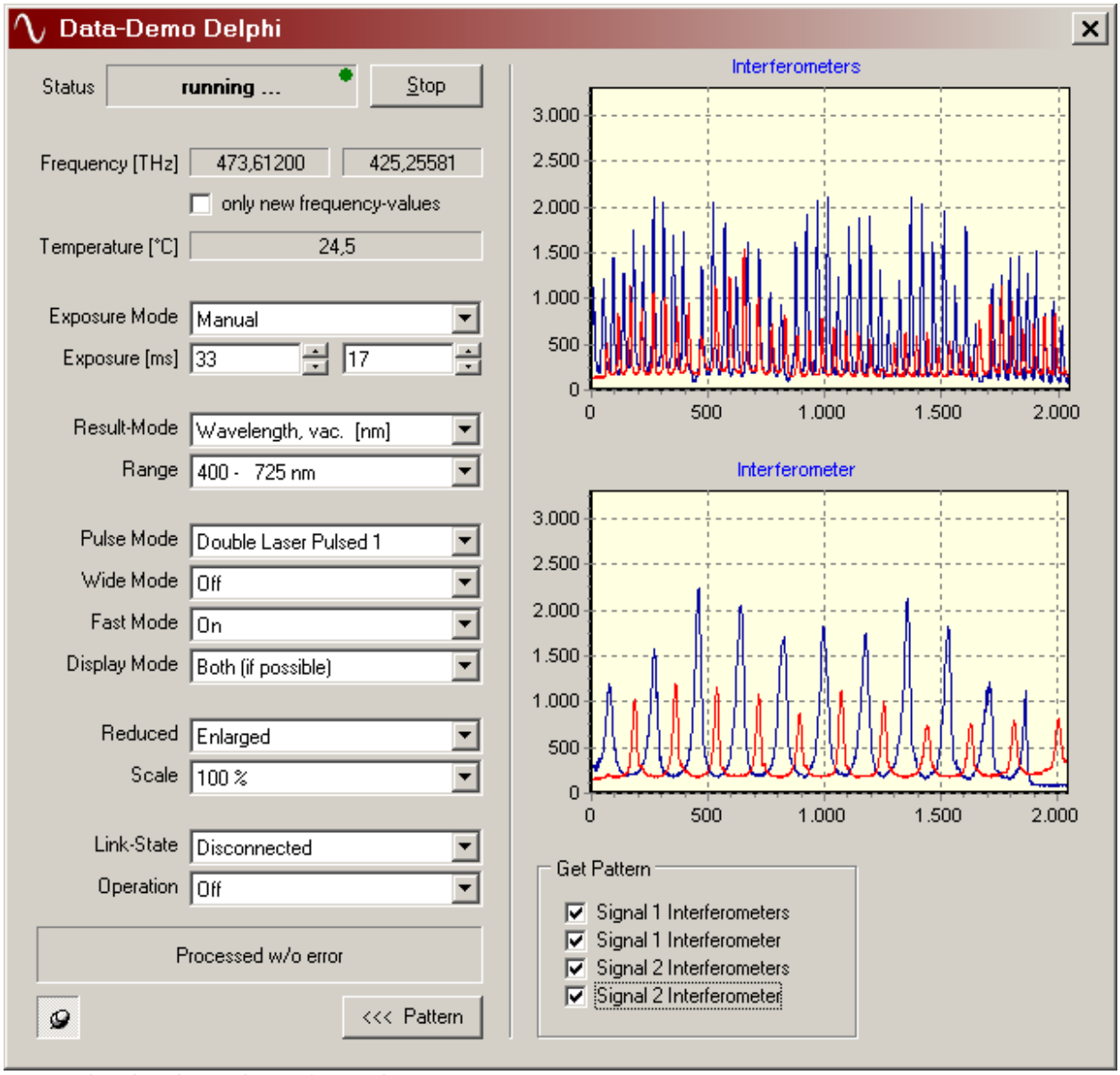
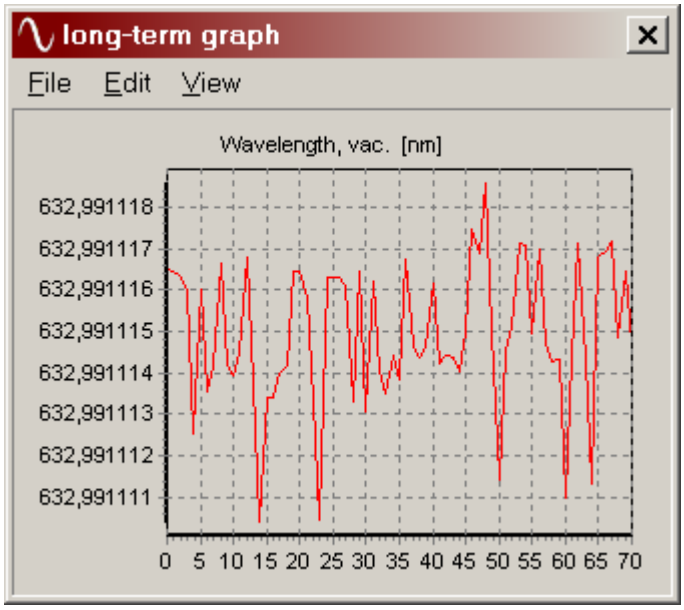


Image showing the options of a WS/7MLC.

4.2.2 Measurement Example "LongTerm"

LongTerm Demo illustrates a way to write a small long-term graphing program. It displays the calculated wavelength and the temperature inside the optical unit.



Short description

Use the menu option "Edit | Start/Stop recording" to toggle between these modes of getting the wavelength and temperature values. In order to spot immediately, this mode is turned on right after the program has started.

The menu "File" contains the possibilities to save and load a recording. These options are deactivated until the recording is stopped.

The displayed range of the wavelength chart is set automatically to best fit the chart. In menu "View | Range", you can change this behaviour to fix a specified wavelength range for a more detailed view.

Also in menu "View" (| Unit) you can set whether the wavelength (in nm) shall be displayed or the frequency in THz and if another chart shall be shown to hold the temperature.

Use menu "View | Always on top" to toggle the demo's topmost state. In its topmost state, a window will appear on top of all other windows, even if it has lost the focus.

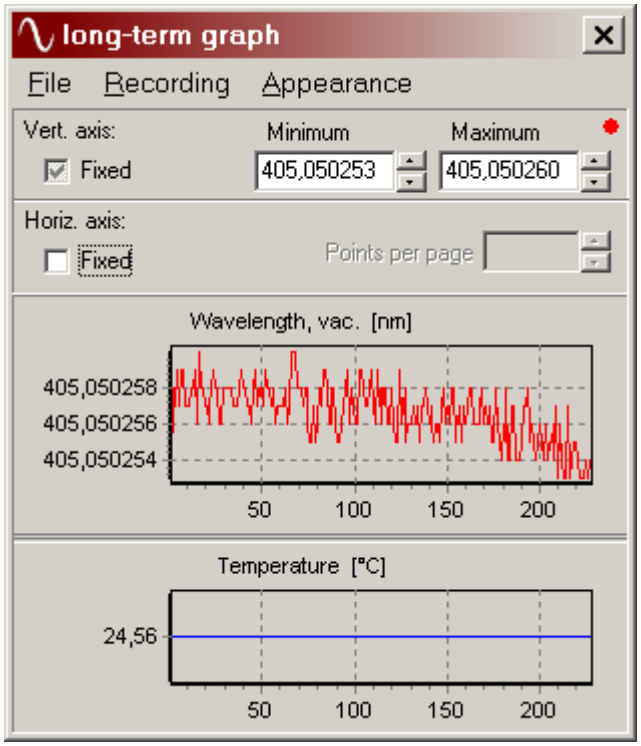
4.2.3 Measurement Example "LongTerm" (callback style)

The callback style LongTerm Demo illustrates how to write a small long-term graphing program using the timesaving callback mechanism. Like the simple version in the previous paragraph (4.2.2) it displays the calculated wavelength and the temperature inside the optical unit and enables saving this information to disc.

Short description

Use the menu option "Recording | Start/Stop" to toggle between these modes of receiving the wavelength and temperature values. In order to spot immediately, this mode is turned on right after the program has started. "Clear" removes all measurement points and also turns on the recording mode.

The menu "File" contains the possibilities to save and load a recording. These recorded longterm arrays (*.lta) are ASCII kind files consisting of the following information: The integer value in the first line represents the number of included measurement datasets to follow and each dataset itself consists of one line including the measurement timestamp in milliseconds, the vacuum wavelength in nm and the temperature in °C and all separated by tabstops.



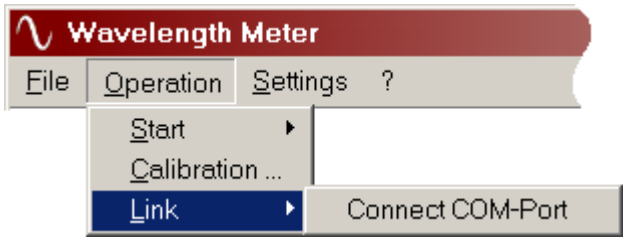
The displayed range of the wavelength chart is set automatically to best fit the chart. You can change this behaviour using the "Fixed"-checkboxes or by selecting a desired area with the mouse to fix a specified wavelength and horizontal range for a more detailed view. If the checkboxes had previously been set, unzooming restores the manually set or stored (red dots) values inside the edit fields. If the checkboxes previously had been unchecked, unzooming automates the axis' to best fit the entire graphs. Zooming/selecting is done by dragging down the mouse direction bottom right with the left button pushed and for unzooming the right mouse button is used to drag in the opposite direction. Having selected an area, it is possible to store the new edit values with pushing the red dots that appear beside.

In menu "Appearance" (| Chart 1) you can set whether the wavelength (in nm) shall be displayed or the frequency in THz and if another chart shall be shown (| Chart 2) to hold the temperature or some other values capable with your specific version.

Use menu "Appearance | Always on top" to toggle the demo's topmost state. In its topmost state, a window will appear on top of all other windows, even if it has lost the focus.

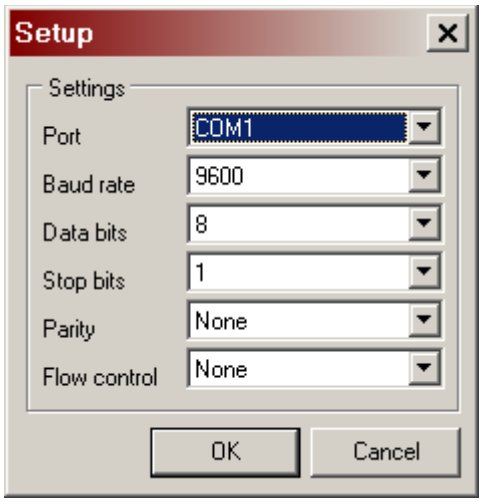
This demo uses the callback mechanism to be noticed about newly available measurement results and changed Wavelength Meters states. Each time new data is available, a user-defined callback procedure is called offering the concerning values. No time is wasted running in endless loops or unnecessary timer procedures.

4.3 Measurement result access (over COM-Port)



The program is able to export the calculated wavelength to the COM-port (RS232), accessible over the COM-port of another computer. Switch on this export with ‘Link | Connect COM-Port’ in the menu ‘Operation’. The program displays the connection state (Link On/Off) in the status bar.

You can also set the export to be started with the Wavelength Meter program or to be run while measuring, adjustable in the Start-Settings dialog window (menu ‘Settings | Extra Settings …’). The value by default is sent in decimal text based format followed by a carriage return. This style can be modified in the extra settings sheet "Various" (menu ‘Settings | Extra Settings’), what especially is needed for exporting the results in multichannel switch mode (if any). In switch mode the communication server expects the values with a guiding channel number followed by an “@” sign, please then modify the value mask to “*CH*@*VAL*”. For return value and error information please have a look at paragraph "GetFrequency, GetFrequency2, GetWavelength, GetWavelength2", p. 64.



The settings of the needed COM-port parameters can be found in menu ‘Settings | COM Port Settings | Link Port …’. It is possible to set all baud rates up to 115200, only the server and the client must use the same settings.

If you dispose of a Wavelength Meter version with switch box accessed by RS232 COM port and you want to use the switch and this functionality to export the measurement results via COM port simultaneously, you need to dispose of two COM ports and set the functionalities to different ports. If the ports are identical, the first used option causes the other to be disabled, saying that the port is occupied.

We also provide a very small demonstration file "ComTest.exe", that can be found within the subdirectory "\Com-Test\" of your installation directory. Please use this file on a different computer to test the COM-Port exporting capability of your Wavelength Meter.



Copy the files found in "\Com-Test\" to anywhere on the concerned computer and start ComTest.exe directly. Before opening the port, please ensure the COM-Port settings (using the "Settings" button) to be identical with these of the sending Wavelength Meter.

This project also is available as sourcecode (in the subfolder "\Projects\ComExampleCB\Delphi\") and shows a very easy mechanism how to access the COM-port to the Wavelength Meter server computer using the same wlmData.dll as described above inside this chapter.

Once WLMComServer.exe is running on your client computer, it also is possible to run all other wavelength retrieving programs, like the included LongTerm.exe for instance.

4.4 File formats

The Wavelength Meter main application is able to store and load single measurement files (menu "File | Save array …" and "Load array …") and long-term recordings (menu "Operation | Start | Recording …" and "Replay …"). Any recordable and saveable files are of the same structure, but old files (old style single measurement files *.sma and *.txt and old style recordings *.ltr) still can be replayed as well. The long-term recordings now are ltr- (binary) and ltx-files (ASCII), and the single measurement arrays are written to smr- (binary) and smx-files (ASCII). The only difference between the ltr- and smr-files is the count of recorded measurements, which is 1 in smr- and any in ltr-files, and so is the difference between the ASCII-type files. In each file any information is stored (e.g. analysis data but calculated measurement results as well).

File structure:

- All files are identical in structure.

- First there's a sequence of 10 Bytes for identifying the file:
  - Program identifier: 3 characters. (Should be "`WLM`".)
  - File type identifier: 3 characters. (Possibilities are: "`LTR`", "`LTX`", "`SMR`" and "`SMX`".)
  - File type ID identifier: 4 characters. (This versions file ID's each are "`ID03`".)

- Then there's a header sequence of 8 single informations (in this order):
  - File version revision: long. (This versions' file version revisions each are 4.)
  - Device version intended for: long.
  - Device revision intended for: long.
  - Count of measurements included in this file: long.
  - Bytes per interferometer pattern point: long.
  - Points per interferometer pattern: long.
  - Count of interferometer patterns per measurement: long.
  - Bytes per measurement for all patterns together: long.

- Then (possibly) a sequence of additional general information (not included in this version, but prepared for extensibility):
  - Sequence identifier: long. Here: `cmcGeneralInfo`.
  - Sequence size: long.
  - Count of general info items: long.
  - Per general info item:
    - General info item identifier: long. `cmg`…-constants (see data.h).
    - Item size: long.
    - Item value: variable.

- Then there's one frame sequence per measurement:
  - Measurement frame sequence identifier: long. `cmcMeasurementFrame`.
  - Frame size: long.
  - Number of this measurement: long.
  - Position of previous measurement frame: long. (in the first frame, -1 is used)
  - Then: a sequence of additional per frame information:
    - Per frame info sequence identifier: long. `cmcItemedData`.
    - Sequence size: long.
    - Count of items: long.
    - Then per item:
      - Item identifier: long. `cmi`…-constants (see data.h).
      - Item size: long.
      - Item value: variable.
  - Then: None up to more of the following:
  - The sequence of the raw interferometer pattern data:
    - Interferometer pattern data identifier: long. `cmcDataArrays`.
    - Size of entire pattern data: long.
    - Pattern data: varying on device version (see file header "sequence of 8 single informations" above).
  - The (possible) sequence of the spectral analysis data:

        - Spectral analysis data identifier: long. `cmcAnalysisArrays`.
        - Size of spectral analysis data: long.
        - Count of spectral analysis coordinates: long.
        - Count of items per coordinate: long. (typically: 2)
        - Per coordinate item:
            - Size of device internal datatype of item: long. (typically: 8)
        - Per coordinate:
            - The coordinate data: variable. (typically 2 times double)
    - Any count of possible future sequences of the structure:
        - Sequence identifier: long. `cmc`…-constants (see data.h).
        - Sequence size: long.
        - Count of sequence items: long.
        - Per sequence item:
            - Sequence item identifier: long. `cmcs`…-constants (see data.h).
            - Item size: long.
            - Item value: variable.

File items format:

- New binary and ASCII files are identical in format two by two.

- ASCII files:
    - Each single information is terminated by a carriage return and line feed, except for the following:
        - Different arrays of the interferometer pattern data (indicated by "Count of interferometer patterns per measurement" in the file header sequence) are stored parallel, what is: one or more items per row separated by tabulators.
        - Spectral analysis coordinates are stored in one row separated by tabulators.
    - Independent of system language settings, the decimal separator is a dot "." and no thousands separator is used.
    - Each single information uses the place its value at least needs (what also means that some size and datatype informations for single values included in this file are intended for structural equality of all files and do not matter for ASCII files' needs), except for the following:
        - Some size and count values are unknown the moment their place is reserved, so they can contain leading zero's, as there are:
            - Count of measurements per file (in the header sequence): uses 8 characters. (might be changed to 6)
            - Size of additional general information sequence: will use 6 or 8 characters.
            - Size of frame: uses 6 characters.
            - Size of per frame info sequence: uses 6 characters.
            - Count of per frame info items: uses 6 characters.
            - Size of interferometer pattern data: uses 6 characters.
            - Size of spectral analysis data: uses 6 characters.
            - Other size and count values of possible future sequences: will use 6 or 8 characters.

- Binary files:
    - Each single information is non-aligned and uses the space its datatype (where indicated) needs, except for the following:
        - The entering 10 Byte file-identifying sequence is aligned at stack space and uses 12 Bytes.

        - Some single values (especially those opened with the cmi…-constants) don't have a special datatype indicated above (variable). Their datatype is indicated by their size declarator, where 1 means: boolean, 2: short, 4: long and 8: double.

- Both file types:
    - Each sizing information is meant to reach from the position after the size value to the end of the spanned structure (including all carriage returns and line feeds in ASCII files).

5 Device information

The following informations are standard values for the referred device class. For your specific device the measurement parameters and conditions as well as the delivery list might be adapted due to specific needs. Please have a look at the shipped list of delivery or at the measurement and calibration certificate for possible adaptations to the following data. Additionally this chapter reflects the data valid for new devices of the specific device class at design time of the manual. So, if this manual is part of a software update, it might be possible that the following paragraphs differ from those of the originally shipped product. In such a case please refer to the “Device information” chapter of your initial printed manual.

5.1 Set of delivery

Optical unit:	1 piece
Collimator:	2 pieces
Fibers (5 m):	1 piece MM, 400 µ or 1 set of SM fibers (count and type depends on device type and offer)
Connection cable (optical unit – computer):	1 piece
Multichannel fiber switch:	1 piece (optional)
Connection cable (for the fiber switch unit):	1 piece (optional)
Power supply (fiber switch):	1 piece (optional)
Operation manual:	1 piece
USB flash drive with measurement software:	1 piece

5.2 Operating conditions

The Wavelength Meter is designed for indoor use under the following environmental conditions:

Air temperature:	15 … 28 °C
Humidity:	up to 80 %
Atmospheric pressure:	680 … 820 mm Hg

5.3 Technical Data (Ångstrom WS/7 IR Super Precision)

Measurement range:	630 … 1750 nm
Relative accuracy:	10^{-7}
Absolute accuracy:	60 MHz
Display resolution:	10 fm; 10 MHz; 0.0001 / cm
Exposure range:	1 … 9999 ms
Measurement repetition rate:	up to 1200 Hz
Required minimal light input:	• cw-operation mode: 0.06 … 15 µW (at 1s exposure) • pulsed-operation mode: 0.06 … 15 µJ
Coupling fiber diameter:	50 µm
Calibration:	Via stabilized He-Ne- or laser of any desired wavelength
Calibration period:	14 days (under Lab conditions)
Number of interferometers:	6

Dimensions:	35.2 x 11.8 x 19.8 cm³
Weight:	5600 g
Warm-up time:	no warm-up time, if there is no fluctuation of ambient temperature (laboratory condition), otherwise until thermal equilibrium has been reached
Display:	• Via PC-software: choice of indication: nm, THz, 1/cm, eV • data handling and full control with each programming environment • handling samples in C++, Delphi, Visual Basic, LabView
Supply:	Power consumption < 2.3 W; Connection directly via USB cable (no additional current supply necessary)
Extras available on request:	• Multi-Laser-Control software • AutoCalibration with multi-channel fiber switch • Multi-Laser measurements with multi-channel fiber switch • UV-versions: down to 192 nm • IR-versions: up to 2250 nm • Laser Control and PID Regulation software • Client specific cases and software

6 HighFinesse Information-Service

For further information please feel free to contact us.



HighFinesse GmbH
Laser and Electronic Systems

Auf der Morgenstelle 14d
72 076 Tübingen
Germany

Phone: +49 / (0) 70 71 / 96 85 15
Fax: +49 / (0) 70 71 / 96 85 17

http: www.HighFinesse.com
email: Info@HighFinesse.com
skype: highfinesse